

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



Nhóm 1

**BỘ LẬP LỊCH TÙY CHỈNH VÀ THUẬT
TOÁN CẤP PHÁT TÀI NGUYÊN CHO
KUBERNETES**

BÁO CÁO BÀI TẬP LỚN

Môn học: INT3310 1 - Quản trị mạng

HÀ NỘI - 2025

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**

NHÓM 1

**BỘ LẬP LỊCH TÙY CHỈNH VÀ THUẬT
TOÁN CẤP PHÁT TÀI NGUYÊN CHO
KUBERNETES**

BÁO CÁO BÀI TẬP LỚN

Môn học: INT3310 1 - Quản trị mạng

Giảng viên hướng dẫn: TS. Dương Lê Minh

Sinh viên thực hiện:
Huỳnh Tiến Dũng
Vũ Huy Hoàng
Nguyễn Đăng Quân
Nguyễn Hồng Quân
Nguyễn Chí Thanh

HÀ NỘI - 2025

LỜI CAM ĐOAN

Nhóm chúng em xin cam đoan những nội dung tìm hiểu, nghiên cứu, thực hành, kết quả thực nghiệm và số liệu hoàn toàn là trung thực và không sao chép hoặc sử dụng kết quả của những công trình, báo cáo tương tự. Nếu phát hiện có sự sao chép, nhóm em xin hoàn toàn chịu trách nhiệm.

Hà Nội, ngày 09 tháng 05 năm 2025

Nhóm sinh viên

TÓM TẮT

Tóm tắt: Trong bối cảnh các hệ thống ứng dụng ngày càng phức tạp và đòi hỏi cao về hiệu suất cũng như khả năng chịu lỗi, việc điều phối và phân bổ tài nguyên trong Kubernetes đóng vai trò then chốt. Bộ lập lịch mặc định của Kubernetes, kube-scheduler, tuy hiệu quả nhưng chủ yếu dựa trên yêu cầu tài nguyên tĩnh của pod. Điều này đôi khi chưa tối ưu trong các môi trường động với tải thay đổi liên tục. Dự án này giới thiệu một giải pháp bộ lập lịch Kubernetes tùy chỉnh, được phát triển nhằm mục tiêu cải thiện việc phân bổ pod bằng cách tích hợp các chỉ số nút (node metrics) theo thời gian thực và tôn trọng các ưu tiên affinity do pod định nghĩa.

Báo cáo này trình bày chi tiết về kiến trúc và quy trình hoạt động của bộ lập lịch tùy chỉnh. Nội dung tập trung vào:

- Kiến trúc tổng quan: Mô tả các thành phần chính bao gồm ứng dụng scheduler tùy chỉnh (viết bằng Golang), sự tương tác với Kubernetes API Server, và việc tích hợp với Prometheus để thu thập chỉ số thời gian thực từ các Node Exporter chạy trên mỗi nút.
- Quy trình lập lịch chi tiết: Đi sâu vào các giai đoạn của quá trình lập lịch, bao gồm cơ chế phát hiện và theo dõi pod cần lập lịch, giai đoạn lọc (Predicate) để chọn các nút tương thích dựa trên yêu cầu tài nguyên, taint/toleration, và node selector, giai đoạn tính điểm (Scoring) để ưu tiên các nút dựa trên bộ nhớ khả dụng, CPU rảnh (lấy từ Prometheus) và các quy tắc affinity, và cuối cùng là giai đoạn ràng buộc (Binding) để gán pod cho nút được chọn.
- Cấu hình và triển khai: Hướng dẫn các khía cạnh cấu hình cần thiết, bao gồm định danh bộ lập lịch cho pod, Kiểm soát Truy cập Dựa trên Vai trò (RBAC), triển khai bộ lập lịch, thiết lập Prometheus và Node Exporter, cùng các ví dụ minh họa và kiểm thử.
- Thực hành (Demo): Trình bày các bước cài đặt trên Google Kubernetes Engine (GKE) và so sánh hoạt động của bộ lập lịch tùy chỉnh với bộ lập lịch mặc định trong các kịch bản khác nhau như lập lịch dựa trên tài nguyên thực tế, node selector, taints/tolerations và node affinity.

Mục tiêu của dự án là cung cấp một giải pháp lập lịch thông minh hơn, linh hoạt hơn, giúp tối ưu hóa việc sử dụng tài nguyên và nâng cao hiệu suất hoạt động của các ứng dụng trên Kubernetes.

Từ khóa: Kubernetes, Custom Kubernetes Scheduler, Bộ lập lịch Kubernetes, Cấp phát tài nguyên, Prometheus, Node Exporter, Node Affinity, Taints and Tolerations, Kubernetes Scheduling.

MỤC LỤC

Lời cam đoan	i
Tóm tắt	ii
Mục lục	iii
Danh mục hình ảnh	vi
Bảng phân chia công việc	viii
Chương 1. Giới thiệu	1
1.1. Giới thiệu chung	1
1.1.1. Sự phát triển của kiến trúc phần mềm	1
1.1.2. Quy trình phát triển phần mềm và văn hóa DevOps	3
1.2. Container	5
1.2.1. Sự nhất quán về môi trường phát triển	5
1.2.2. Phân bổ tài nguyên	6
1.2.3. Container	8
1.2.4. Docker	9
1.3. Điều phối container	10
1.3.1. Hệ thống điều phối container	11
1.3.2. Kubernetes	11
1.3.3. Các hệ thống điều phối container khác	12
1.4. Kubernetes	13
1.4.1. Cơ bản về Kubernetes	13
1.4.2. Kiến trúc Kubernetes	15
1.4.3. Một số thành phần cơ bản của Kubernetes	18
1.4.4. Luồng hoạt động của Kubernetes	22

Chương 2. Bộ lập lịch Kubernetes	28
2.1. Giới thiệu chung	28
2.2. Khái niệm lập lịch trong Kubernetes	28
2.3. Trình lập lịch mặc định kube-scheduler	29
2.3.1. Kiến trúc	29
2.3.2. Quá trình lập lịch	31
2.4. Các phương pháp điều khiển lập lịch Pod	33
2.4.1. Node Selector	34
2.4.2. Node Affinity và Anti-Affinity	34
2.4.3. Taints và Tolerations	35
2.4.4. Pod Topology Spread Constraints	36
2.5. Các chiến lược nâng cao trong lập lịch Kubernetes	36
2.5.1. Topology Spread Constraints	36
2.5.2. Pod Priority và Preemption	37
2.5.3. Scheduling Policies và Scheduling Profiles (Plugin)	37
2.6. Ví dụ chi tiết về một quyết định lập lịch	38
2.7. Kết luận	39
Chương 3. Trình lập lịch Kubernetes tùy chỉnh	40
3.1. Kiến trúc của trình lập lịch Scheduler tùy chỉnh	40
3.1.1. Sơ đồ tổng quan	40
3.1.2. Các thành phần chính và vai trò của chúng	41
3.1.3. Luồng tương tác tổng thể khi phân lịch một Pod	44
3.2. Chi tiết quy trình lập lịch	45
3.2.1. Cơ chế phát hiện và theo dõi Pod	45

3.2.2. Giai đoạn lọc (Predicate Phase): Lọc các Node Tương thích ..	49
3.2.3. Giai đoạn tính điểm (Scoring Phase): Ưu tiên các Node	52
3.2.4. Giai đoạn ràng buộc (Binding Phase): Gán Pod cho Node	58
3.2.5. Báo cáo sự kiện	61
3.3. Cấu trúc mã nguồn tổng thể và các phần chính	63
3.3.1. Logic ứng dụng chính (scheduler/)	64
3.3.2. Các tệp manifests Kubernetes	70
3.4. Các khía cạnh cấu hình và triển khai	72
3.4.1. Nhận dạng Bộ lập lịch: Chỉ định Pod cho Lập lịch Tùy chỉnh .	72
3.4.2. Kiểm soát truy cập dựa trên Vai trò (RBAC)	73
3.4.3. Triển khai bộ lập lịch tùy chỉnh	76
3.4.4. Triển khai công cụ giám sát thông số (Prometheus & Node Exporter)	77
3.4.5. Kiểm thử và minh họa (scheduler/deployments/)	80
Chương 4. Demo	82
4.1. Set up Google Kubernetes Engine (GKE)	82
4.2. Default Scheduler vs Custom Scheduler	89
4.3. Node Selector	92
4.4. Taint and Tolerants	93
4.5. Node Affinity	94
Tài liệu tham khảo	96

DANH MỤC HÌNH ẢNH

Hình 1.1	Kiến trúc Monolith và kiến trúc Microservices	2
Hình 1.2	Mô hình phát triển thác nước	3
Hình 1.3	Luồng phát triển với Devops và các công nghệ liên quan	5
Hình 1.4	Cách docker chạy ứng dụng	6
Hình 1.5	Kiến trúc Virtual Machine và Container	8
Hình 1.6	Docker	10
Hình 1.7	Các hệ thống điều phối container phổ biến	13
Hình 1.8	Hệ thống Kubernetes	14
Hình 1.9	Kiến trúc Kubernetes	15
Hình 1.10	Node	17
Hình 1.11	Các thành phần của Kubernetes	18
Hình 1.12	Pod trong Kubernetes	19
Hình 1.13	Luồng khởi tạo của Pod	23
Hình 1.14	Lợi ích của việc sử dụng Kubernetes	24
Hình 4.1	Màn hình Google Cloud	82
Hình 4.2	Lựa chọn dự án	83
Hình 4.3	Truy cập vào dịch vụ cluster của GKE	83
Hình 4.4	Bật API Kubernetes	84
Hình 4.5	Tạo cluster mới	84
Hình 4.6	Thiết lập Cluster	85
Hình 4.7	Thiết lập kết nối đến master	85
Hình 4.8	Lấy mã kết nối	86
Hình 4.9	Truy cập bằng Google Cloud Shell	86
Hình 4.10	Sau khi chạy cái Pods liên quan tới Prometheus và Node Exporter	88
Hình 4.11	Web UI của Prometheus	88
Hình 4.12	Xem logs của trình lập lịch tùy chỉnh sau khi vừa deploy	89
Hình 4.13	2 nodes chạy 2 pods <code>sleep.yaml</code> và 1 node còn lại chạy pod <code>sysbench.yaml</code>	90
Hình 4.14	Node chạy <code>sysbench.yaml</code> có lượng memory requested thấp	90
Hình 4.15	Node chạy <code>sleep.yaml</code> có lượng memory requested cao	90
Hình 4.16	Thực tế lượng memory trống của các node	91
Hình 4.17	<code>testdefault.yaml</code> được chạy trên máy chạy <code>sysbench.yaml</code> (mặc dù máy này đang có rất ít memory trống), còn <code>testcustom.yaml</code> được chạy trên máy chạy <code>sleep.yaml</code> (máy đang có gần như toàn bộ memory trống)	91
Hình 4.18	Logs của custom scheduler	92
Hình 4.19	Logs của custom scheduler	93

Hình 4.20	Logs của custom scheduler	94
Hình 4.21	Logs của custom scheduler	95

BẢNG PHÂN CHIA CÔNG VIỆC

Thành viên	Nhiệm vụ	Đóng góp
Huỳnh Tiến Dũng (Nhóm trưởng) 21020007	- Lập kế hoạch, phân công công việc cho nhóm. - Tìm hiểu và cài đặt trình lập lịch custom. Trình bày phần Demo (Chương 3). - Chỉnh sửa slide. - Đại diện nhóm thuyết trình.	27%
Vũ Huy Hoàng 22021108	Tìm hiểu và trình bày về trình lập lịch Kubernetes (Chương 2).	17%
Nguyễn Đăng Quân 22021121	- Tìm hiểu vấn đề thực tế với Kubernetes, trình bày giới thiệu bài toán (Chương 1). - Hỗ trợ cài đặt trình lập lịch custom, lồng tiếng phần Demo. - Đại diện nhóm thuyết trình.	25%
Nguyễn Hồng Quân 22021122	Tìm hiểu và trình bày về trình lập lịch Kubernetes (Chương 2).	15%
Nguyễn Chí Thanh 22021123	Lên kế hoạch bố cục và chuẩn bị nội dung slide từ quyền báo cáo.	16%

Chương 1

Giới thiệu

1.1. Giới thiệu chung

1.1.1. Sự phát triển của kiến trúc phần mềm

Trong những giai đoạn đầu của công nghệ phần mềm, các ứng dụng thường được xây dựng theo mô hình monolithic (kiến trúc nguyên khối), nơi toàn bộ chức năng được tích hợp trong một chương trình duy nhất. Kiến trúc này đơn giản, phù hợp với các hệ thống nhỏ và đội ngũ phát triển hạn chế. Tuy nhiên, khi yêu cầu của người dùng ngày càng tăng và hệ thống ngày càng phức tạp, monolithic bộc lộ nhiều hạn chế:

- Tính độc lập cao dẫn đến hệ thống vô cùng kém linh hoạt, khó tích hợp, kết hợp với ứng dụng bên ngoài.
- Các thành phần phụ thuộc vào nhau, mức độ ràng buộc lớn, dẫn đến các tính năng khó phát triển độc lập, khó mở rộng và bảo trì phức tạp.
- Cập nhật một thành phần dù nhỏ vẫn phải build, deploy lại toàn bộ ứng dụng
- ...

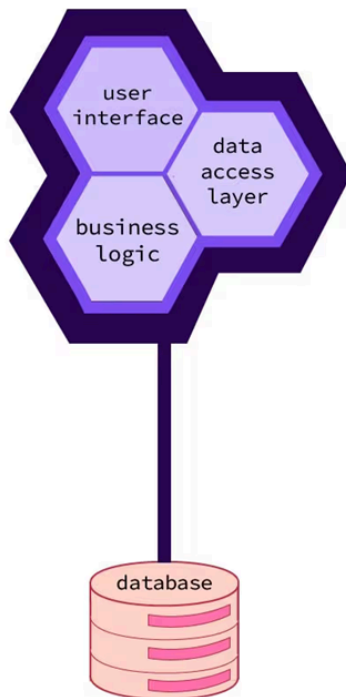
Để đáp ứng nhu cầu phát triển nhanh chóng và mở rộng linh hoạt, kiến trúc phần mềm đã tiến hóa dần qua các mô hình như layered architecture (kiến trúc phân lớp), event-driven architecture (kiến trúc hướng sự kiện), và đặc biệt là microservices architecture (kiến trúc dịch vụ nhỏ). Các kiến trúc ngày càng cải thiện hệ thống để dễ dàng mở rộng theo nhu cầu, cho phép các đội nhóm phát triển và triển khai độc lập từng phần nhỏ của ứng dụng.

Trong kiến trúc microservices, ứng dụng được chia thành nhiều dịch vụ nhỏ, mỗi microservice chạy như một tiến trình độc lập và giao tiếp với các dịch vụ khác thông qua các giao diện (API) được xác định rõ ràng và đơn giản. Mỗi dịch vụ

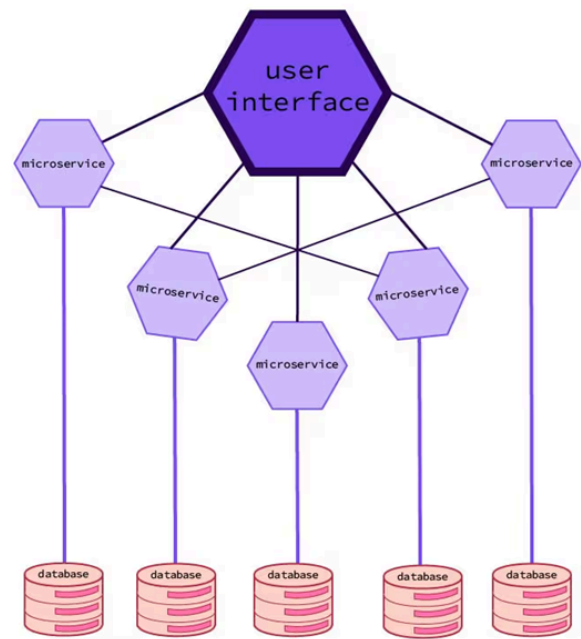
có thể được viết bằng ngôn ngữ phù hợp nhất để thực hiện nó, được coi là một ứng dụng con hoàn toàn độc lập, các thành phần khác bên ngoài chỉ gọi API, biết được đầu vào và đầu ra của ứng dụng. Giao thức được sử dụng có thể là các giao thức đồng bộ như HTTP thông qua API RESTful, hoặc các giao thức không đồng bộ như AMQP (Advanced Message Queueing Protocol). Các giao thức này đơn giản, dễ hiểu đối với nhà phát triển và không bị ràng buộc với bất kỳ ngôn ngữ lập trình cụ thể nào.

Mỗi microservice là một tiến trình độc lập với API ít thay đổi, theo đó có thể phát triển và triển khai chúng một cách độc lập. Thay đổi một microservice không yêu cầu cập nhật hoặc triển khai lại các microservice khác, với điều kiện API giữ nguyên.

monolithic architecture



microservices architecture



Hình 1.1 — Kiến trúc Monolith và kiến trúc Microservices

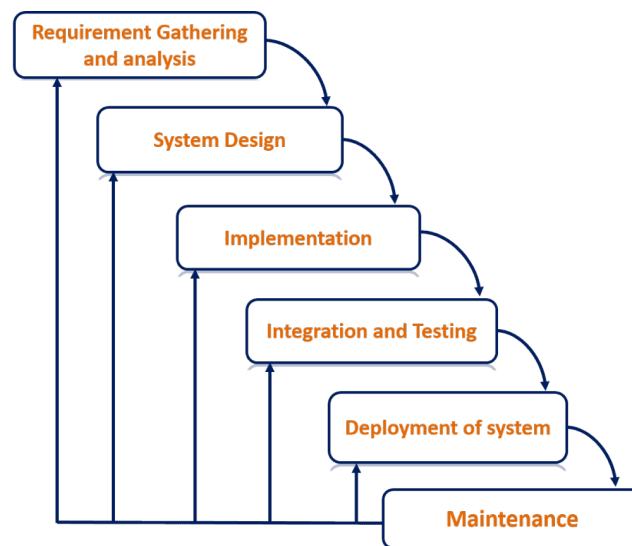
Với số lượng các dịch vụ cần được triển khai và các cơ sở hạ tầng dữ liệu, phần cứng ngày càng lớn, việc cấu hình, quản lý và duy trì hệ thống vận hành trở thành một thách thức lớn. Bài toán về việc phải deploy các thành phần của hệ thống ở những phần cứng nào sao cho tối ưu tài nguyên, tiết kiệm chi phí thậm chí là vô cùng khó giải, cần chuyên môn cao. Có thể nói, làm tất cả điều này một cách thủ công không hề dễ dàng. Ta kì vọng rằng ta có thể tự động hóa các

công việc như lập lịch để các thành phần đi vào server, tự động cấu hình, giám sát và xử lý sự cố. Đây chính là những công việc mà Kubernetes sẽ giải quyết

1.1.2. Quy trình phát triển phần mềm và văn hóa DevOps

Quy trình phát triển phần mềm là tập hợp các bước và phương pháp được tổ chức tuần tự hoặc linh hoạt nhằm thiết kế, xây dựng, kiểm thử và triển khai các sản phẩm phần mềm. Quy trình phát triển là yếu tố quan trọng đảm bảo sự thành công của dự án phần mềm, đảm bảo phần mềm được phát triển đúng yêu cầu, đạt chất lượng mong muốn và có khả năng bảo trì lâu dài. Cùng với sự phát triển của kiến trúc phần mềm, quy trình phát triển phần mềm cũng đã trải qua nhiều giai đoạn thay đổi nhằm đáp ứng yêu cầu tăng tốc độ phát hành sản phẩm, nâng cao chất lượng và đảm bảo khả năng thích ứng nhanh với biến động thị trường.

Ban đầu, các mô hình phát triển như Waterfall (mô hình thác nước) được áp dụng. Mô hình Waterfall chia quy trình phát triển thành các giai đoạn riêng biệt: phân tích yêu cầu, thiết kế, cài đặt, kiểm thử và triển khai. Các giai đoạn này được thực hiện tuần tự, chỉ chuyển sang bước tiếp theo khi hoàn thành bước trước. Tuy rất bài bản, đơn giản, dễ hiểu, dễ quản lý; song mô hình Waterfall chỉ tỏ ra phù hợp với những dự án nhỏ và có yêu cầu rõ ràng ngay từ đầu; mô hình này bộc lộ nhiều hạn chế trong môi trường thay đổi nhanh, khó thích ứng nếu có thay đổi yêu cầu sau mỗi giai đoạn.



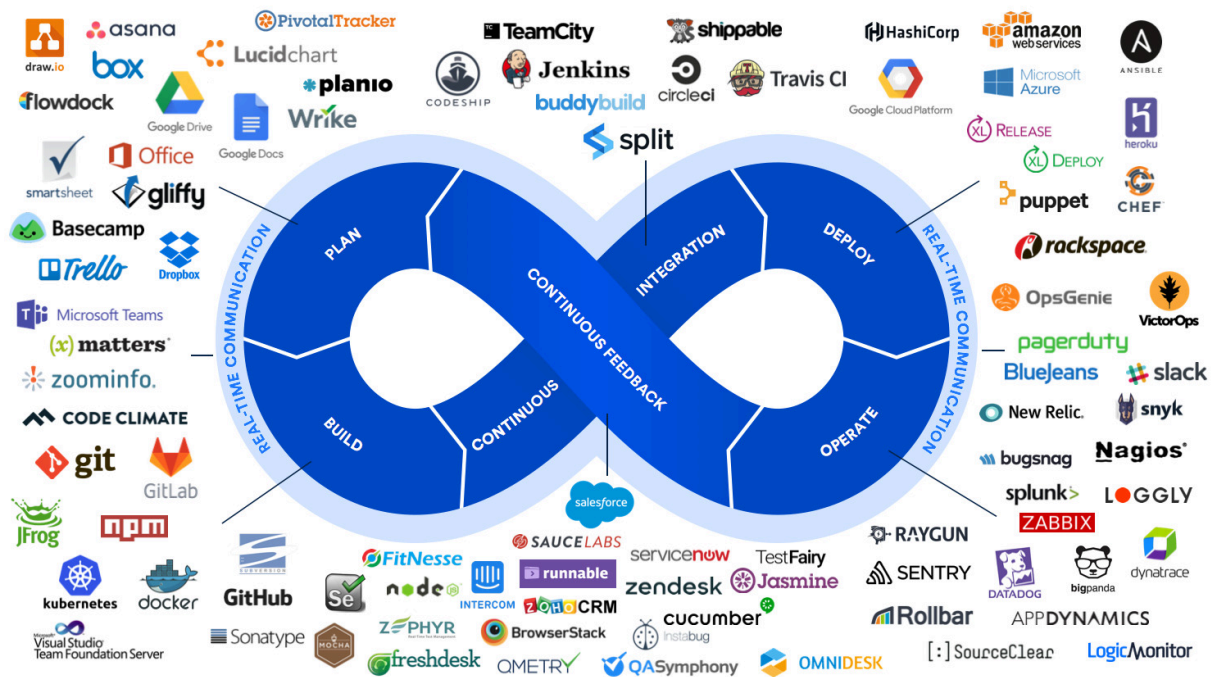
Hình 1.2 — Mô hình phát triển thác nước

Sự xuất hiện của các phương pháp phát triển linh hoạt (Agile) đã giúp quy trình phát triển phần mềm trở nên linh động hơn. Agile tiếp cận phát triển phần mềm theo hướng lặp (iterations) và gia tăng (incremental). Sản phẩm được xây dựng qua nhiều chu kỳ ngắn, với sự phản hồi liên tục từ khách hàng và cải tiến liên tục.

Trong các mô hình phát triển phần mềm truyền thống, sau khi các developers (nhà phát triển hoặc lập trình viên) hoàn thành việc lập trình, kiểm thử nội bộ và đóng gói phần mềm, sản phẩm sẽ được chuyển giao cho một bộ phận riêng biệt gọi là Operations Team (gọi tắt là Ops Team). Tuy nhiên, cách làm này cũng có một số vấn đề:

- Developers không hiểu hết môi trường production.
- Ops gặp khó khăn khi sản phẩm thiếu tài liệu hoặc chưa tối ưu.
- Quá trình bàn giao chậm, gây chậm trễ phát hành sản phẩm.

Trên nền tảng Agile, văn hóa DevOps ra đời nhằm hợp nhất phát triển phần mềm (Dev) và vận hành hệ thống (Ops). DevOps thúc đẩy việc tự động hóa các quy trình xây dựng (build), kiểm thử (test), triển khai (deploy) và giám sát (monitoring) phần mềm; từ đó rút ngắn chu kỳ phát hành phần mềm, giảm thiểu lỗi trong quá trình triển khai, tăng tính ổn định và khả năng phục hồi của hệ thống, đồng thời tạo môi trường hợp tác chặt chẽ hơn giữa các nhóm dev và ops. Trong bối cảnh đó, các công nghệ như container, orchestration, và hệ thống tự động hóa hạ tầng trở thành những thành phần cốt lõi hỗ trợ hiện thực hóa văn hóa DevOps trong thực tiễn.



Hình 1.3 — Luồng phát triển với Devops và các công nghệ liên quan

1.2. Container

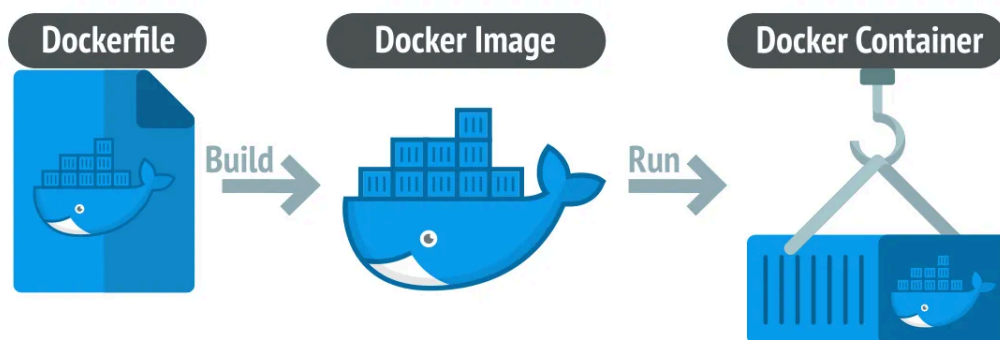
1.2.1. Sự nhất quán về môi trường phát triển

Trong thực tế phát triển phần mềm, sự khác biệt giữa môi trường phát triển (Development Environment), môi trường triển khai (Stage Environment) và môi trường sử dụng (Product Environment) là hiện tượng thường xuyên xảy ra; sự khác biệt này có thể đến từ nhiều yếu tố: Phần cứng, Hệ điều hành và phiên bản, Các thư viện, phần mềm hỗ trợ và cấu hình hệ thống, ...). Đây là nguyên nhân gây ra nhiều khó khăn và tốn kém thời gian trong quá trình phát triển và vận hành sản phẩm.

Trong quy trình truyền thống, môi trường production (môi trường vận hành chính thức) thường được quản lý bởi nhóm Operations (Ops), trong khi các developers (nhà phát triển) tự thiết lập môi trường riêng của mình để phục vụ việc lập trình. Hai môi trường này không chỉ khác biệt về cấu hình kỹ thuật mà còn khác biệt về mức độ ưu tiên: các quản trị viên hệ thống chú trọng việc cập nhật bản vá bảo mật và tính ổn định lâu dài, trong khi nhà phát triển lại tập trung vào tốc độ triển khai và thử nghiệm các tính năng mới.

Ngoài ra, môi trường của người sử dụng cuối — đặc biệt trong các ứng dụng phổ thông — còn biến động liên tục, chịu ảnh hưởng bởi thay đổi phần cứng, cập nhật hệ điều hành, hoặc thay đổi trình duyệt, nền tảng di động, v.v. Đây là yếu tố mà các nhà phát triển cần nghiên cứu kỹ và chuẩn bị phương án thích ứng linh hoạt.

Docker Container ra đời như một giải pháp hiệu quả để giải quyết bài toán về sự khác biệt môi trường giữa các giai đoạn phát triển, triển khai và vận hành. Docker tạo ra các container bằng cách đóng gói ứng dụng cùng với toàn bộ các thư viện, công cụ hỗ trợ và cấu hình cần thiết vào trong một đơn vị độc lập, nhẹ và di động. Nhờ container, ứng dụng có thể chạy ổn định trên bất kỳ môi trường nào có hỗ trợ công nghệ container, bất kể sự khác biệt về hệ điều hành, thư viện hệ thống hay phần cứng bên dưới.



Hình 1.4 — Cách docker chạy ứng dụng

Bằng cách cô lập môi trường vận hành của ứng dụng trong một lớp ảo hóa nhẹ, container đảm bảo rằng “những gì chạy được trên máy của lập trình viên cũng sẽ chạy được trên server production”. Điều này làm giảm đáng kể các lỗi do khác biệt môi trường, đồng thời đơn giản hóa quy trình phát triển, kiểm thử, triển khai và vận hành phần mềm.

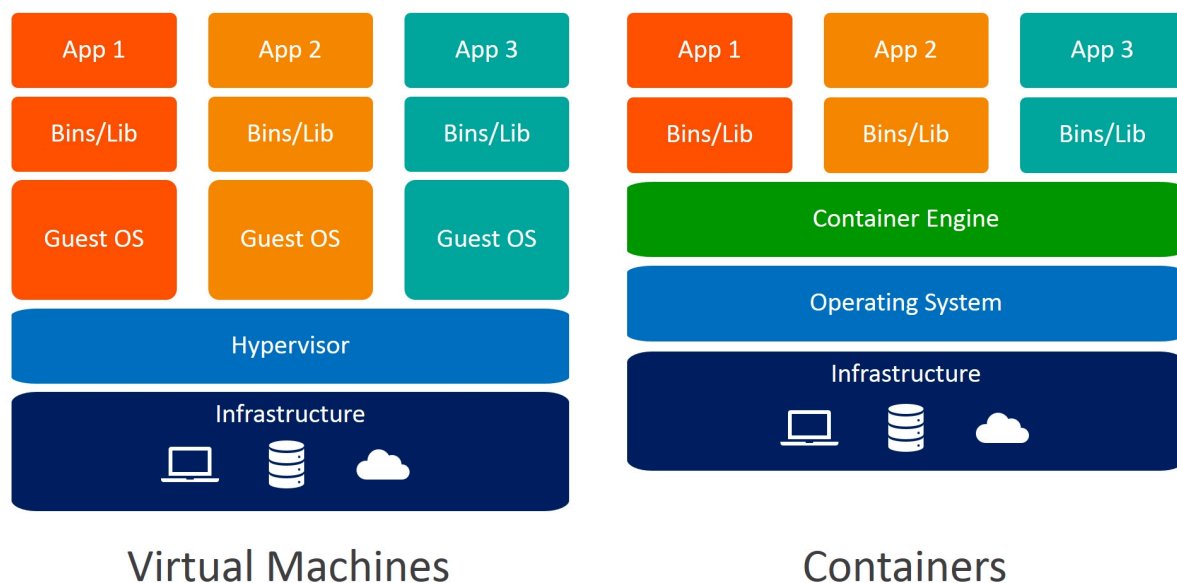
1.2.2. Phân bổ tài nguyên

Trong quá trình vận hành các hệ thống phần mềm, quản lý và phân bổ tài nguyên (bao gồm CPU, bộ nhớ RAM, băng thông mạng, và lưu trữ) là một vấn đề cốt lõi để đảm bảo hiệu suất và tính ổn định của dịch vụ. Ban đầu, các ứng dụng được chạy trên các máy chủ vật lý, chia sẻ tài nguyên chung của hệ thống. Không có cách nào để xác định ranh giới tài nguyên cho các ứng dụng trong máy chủ, điều này dẫn đến tình trạng phân bổ tài nguyên không đồng đều.

Một cách giải quyết đơn giản cho vấn đề cô lập ứng dụng là chạy mỗi ứng dụng trên một máy chủ vật lý riêng biệt. Tuy nhiên, giải pháp này không tối ưu và bộc lộ nhiều hạn chế: tài nguyên trên mỗi máy chủ thường không được tận dụng hết (ví dụ CPU và bộ nhớ còn dư thừa), dẫn đến lãng phí tài nguyên. Đồng thời, việc duy trì một số lượng lớn máy chủ vật lý gây ra chi phí rất cao về cả phần cứng, không gian lưu trữ, điện năng và công tác vận hành cho các tổ chức.

Một giải pháp khác là sử dụng công nghệ ảo hóa. Ảo hóa cho phép một máy chủ vật lý vận hành đồng thời nhiều Máy ảo (Virtual Machine – VM), mỗi máy ảo như một hệ thống độc lập với hệ điều hành và tài nguyên riêng, hoạt động bên trên một lớp phần cứng được ảo hóa. Nhờ đó, các ứng dụng có thể tận dụng tốt hơn tài nguyên phần cứng của máy chủ vật lý, đồng thời cho phép khả năng mở rộng linh hoạt, khi việc bổ sung hoặc cập nhật ứng dụng chỉ cần triển khai thêm máy ảo mới mà không yêu cầu thêm phần cứng vật lý. Công nghệ ảo hóa đã góp phần giảm chi phí đầu tư hạ tầng, nâng cao hiệu quả sử dụng tài nguyên và cho phép tổ chức xây dựng các cụm máy ảo như một tập hợp tài nguyên sẵn sàng cho nhiều ứng dụng khác nhau.

Khi một ứng dụng chỉ bao gồm một số lượng nhỏ các thành phần lớn, việc cấp phát một Máy ảo (Virtual Machine – VM) riêng cho mỗi thành phần và cô lập môi trường bằng cách cung cấp hệ điều hành riêng cho từng VM là hoàn toàn khả thi và hợp lý. Tuy nhiên, khi các thành phần của ứng dụng trở nên nhỏ hơn, đồng thời số lượng thành phần tăng lên, việc tiếp tục sử dụng nhiều VM độc lập trở nên kém hiệu quả. Mỗi VM cần vận hành một hệ điều hành riêng, chiếm dụng thêm bộ nhớ, tài nguyên CPU và dung lượng lưu trữ, dẫn đến lãng phí tài nguyên vật lý. Bên cạnh đó, việc mỗi VM yêu cầu quá trình cấu hình, cập nhật và quản lý riêng biệt cũng làm tăng đáng kể khối lượng công việc vận hành. Khi số lượng VM ngày càng lớn, tổ chức phải đầu tư thêm nguồn nhân lực cho việc quản trị hệ thống, gây tốn kém chi phí và làm giảm hiệu quả vận hành tổng thể.



Hình 1.5 — Kiến trúc Virtual Machine và Container

Sự xuất hiện của công nghệ Container đã thay đổi căn bản cách thức quản lý tài nguyên. Container cho phép chúng ta chạy nhiều dịch vụ khác nhau trên cùng một máy chủ vật lý, đồng thời vẫn đảm bảo mỗi dịch vụ có môi trường vận hành riêng biệt, cho phép giới hạn tài nguyên và được cô lập an toàn với các dịch vụ khác, tương tự như cách Máy ảo (VM) thực hiện. Tuy nhiên, khác với VM, container không cần hệ điều hành riêng cho mỗi đơn vị, do đó yêu cầu ít tài nguyên hơn rất nhiều, chúng ta cũng không cần khởi động toàn bộ hệ điều hành như khi khởi động một VM, việc khởi chạy container cũng nhanh hơn đáng kể. Một tiến trình trong container có thể khởi chạy ngay lập tức, giúp tăng tốc độ triển khai ứng dụng và tối ưu hoá hiệu suất sử dụng tài nguyên.

1.2.3. Container

Container là một công nghệ cung cấp phương thức đóng gói ứng dụng cùng với toàn bộ các thư viện, công cụ, và thiết lập môi trường cần thiết, đảm bảo rằng ứng dụng có thể chạy nhất quán và ổn định trên bất kỳ nền tảng nào hỗ trợ nó.

Khái niệm container hóa không phải là mới, mà đã được nghiên cứu từ những năm 1970 với các công nghệ như chroot trên Unix, cho phép cô lập hệ thống file cho một tiến trình. Sau đó, các bước tiến lớn hơn được thực hiện đưa việc cô lập tiến trình và tài nguyên lên một tầm cao mới. Đến năm 2013, công cụ Docker ra đời và đã làm cho container trở nên phổ biến nhờ việc đơn giản hóa quá trình

đóng gói, phân phối và vận hành các container, mở đường cho sự bùng nổ của container trong các hệ thống phần mềm hiện đại.

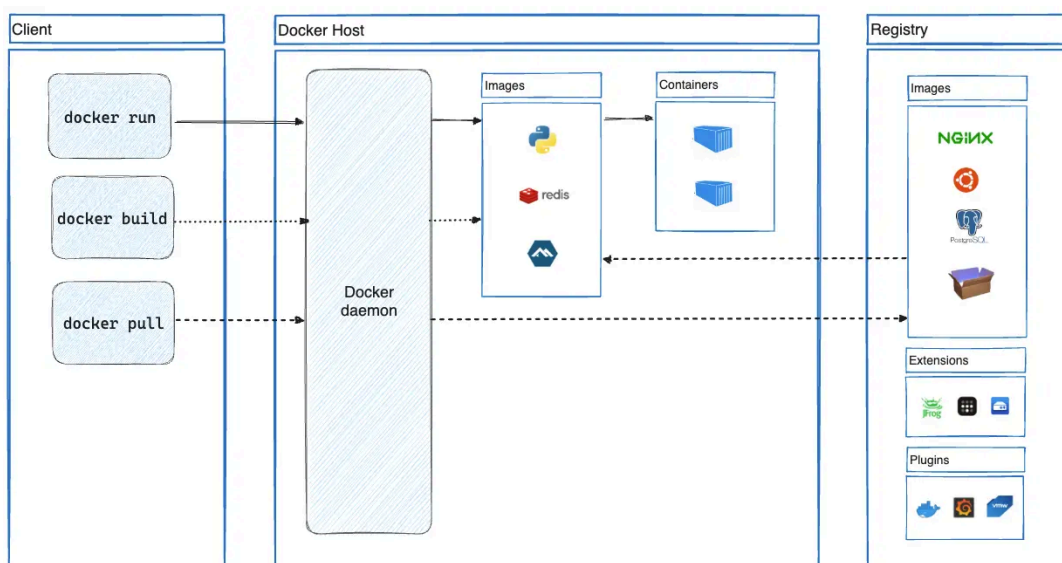
Khác với Máy ảo (Virtual Machine – VM), mỗi container không cần vận hành hệ điều hành riêng biệt. Thay vào đó, các tiến trình trong container chạy trực tiếp trên hệ điều hành của máy chủ vật lý, tương tự như các tiến trình thông thường, nhưng được cô lập với các tiến trình khác nhờ các cơ chế của hệ điều hành. Đối với mỗi tiến trình bên trong container, nó nhận thức như thể mình đang vận hành trên một hệ thống riêng biệt, với hệ điều hành và tài nguyên độc lập. Điều này giúp container nhẹ hơn, tiêu tốn ít tài nguyên hơn và có khả năng khởi động gần như tức thì, mang lại lợi thế lớn về tốc độ triển khai và hiệu quả sử dụng hạ tầng.

Một trong những cơ chế quan trọng đảm bảo sự cô lập và quản lý tài nguyên trong container là sự kết hợp giữa cgroups (control groups) và namespaces, hai tính năng cốt lõi của nhân (kernel) Linux. Cgroups cho phép giới hạn lượng tài nguyên hệ thống mà một container có thể tiêu thụ, bao gồm CPU, bộ nhớ RAM, băng thông mạng, và các tài nguyên khác. Namespaces đảm bảo rằng mỗi container có không gian riêng biệt về tiến trình, mạng, hệ thống tập tin và các tài nguyên khác, khiến các tiến trình bên trong container chỉ “nhìn thấy” những tài nguyên được cấp cho container đó. Nhờ sự kết hợp này, các container không thể chiếm dụng tài nguyên vượt mức cấu hình, và tiến trình trong container này cũng không thể làm ảnh hưởng đến tiến trình trong container khác, tương tự như khi chúng chạy trên các máy chủ vật lý riêng biệt.

1.2.4. Docker

Mặc dù công nghệ container đã xuất hiện từ khá lâu, nhưng chỉ đến khi nền tảng Docker ra đời, container mới thực sự trở nên phổ biến và được tiếp cận rộng rãi. Docker là hệ thống container đầu tiên cung cấp khả năng di chuyển container dễ dàng giữa các máy chủ khác nhau mà không cần phải lo lắng về sự khác biệt môi trường. Docker đã đơn giản hóa toàn bộ quy trình đóng gói ứng dụng, không chỉ bao gồm mã nguồn, thư viện và các phụ thuộc cần thiết, mà còn cả hệ thống tệp của môi trường vận hành, thành một gói di động thống nhất. Gói này, gọi là Docker Image, có thể được triển khai trên bất kỳ máy chủ nào có cài đặt Docker, bất kể nền tảng phần cứng hoặc hệ điều hành bên dưới.

Điểm mạnh lớn nhất của Docker là khả năng cô lập hoàn toàn môi trường vận hành cho ứng dụng. Khi một ứng dụng chạy bên trong Docker container, tiến trình chỉ nhìn thấy các thư viện, module và cấu hình được đóng gói bên trong container, hoàn toàn tách biệt với môi trường hệ điều hành bên ngoài. Nhờ đó, lập trình viên và tổ chức không còn phải lo lắng về việc ứng dụng có thể hoạt động khác nhau ở các môi trường phát triển, kiểm thử hay triển khai thực tế. Chỉ cần đóng gói ứng dụng vào một Docker container, chúng ta có thể mang nó đến bất cứ đâu và chạy ổn định mà không cần thiết lập lại môi trường.



Hình 1.6 — Docker

Một điểm khác biệt quan trọng giữa Docker Image và image của máy ảo (VM Image) là cách thức tổ chức theo cấu trúc lớp (layered architecture). Mỗi Docker Image được xây dựng từ nhiều lớp chồng lên nhau, và các lớp này có thể được chia sẻ hoặc tái sử dụng giữa các images khác nhau. Điều đó có nghĩa là khi triển khai một Docker Image mới, nếu máy chủ đã có sẵn một số lớp chung từ trước, chỉ những lớp mới cần tải xuống, giúp tiết kiệm đáng kể băng thông, thời gian và không gian lưu trữ.

1.3. Điều phối container

Khi số lượng thành phần ứng dụng trong hệ thống ngày càng tăng, việc quản lý và triển khai tất cả các thành phần đó trở nên ngày càng phức tạp. Google là một trong những công ty đầu tiên nhận ra nhu cầu cấp thiết về một phương pháp tốt hơn để triển khai và vận hành phần mềm cũng như cơ sở hạ tầng ở quy

mô toàn cầu. Với việc vận hành hàng trăm nghìn máy chủ trên khắp thế giới, Google đã phải đối mặt với những thách thức lớn trong việc quản lý hệ thống phân tán quy mô siêu lớn. Điều này đã thúc đẩy họ phát triển những giải pháp nội bộ nhằm tự động hóa và tối ưu hóa quá trình phát triển, triển khai và vận hành hàng nghìn thành phần phần mềm, đồng thời đảm bảo khả năng quản lý hiệu quả và kiểm soát chi phí ở quy mô chưa từng có.

1.3.1. Hệ thống điều phối container

Để giải quyết những thách thức trong việc vận hành một hệ thống khổng lồ gồm hàng trăm nghìn máy chủ, Google đã phát triển một hệ thống nội bộ có tên là Borg (sau này thay thế bởi Omega). Borg là nền tảng điều phối container quy mô lớn, được thiết kế để tự động hóa việc triển khai, quản lý và giám sát các ứng dụng chạy trên hạ tầng phân tán toàn cầu của Google.

Sau gần một thập kỷ vận hành nội bộ các hệ thống như Borg và Omega, vào năm 2014, Google đã quyết định chia sẻ kinh nghiệm của mình với cộng đồng bằng cách ra mắt Kubernetes – một nền tảng mã nguồn mở được xây dựng dựa trên những bài học thực tiễn thu thập từ việc vận hành các hệ thống quy mô lớn. Kubernetes kế thừa nhiều nguyên lý thiết kế và mô hình vận hành từ Borg, Omega cùng các hệ thống nội bộ khác của Google, nhưng được đơn giản hóa để phù hợp với nhu cầu triển khai container trong cộng đồng rộng lớn hơn.

1.3.2. Kubernetes

Kubernetes là một hệ thống phần mềm mã nguồn mở giúp tự động hóa việc triển khai, quản lý và mở rộng các ứng dụng container trên hệ thống máy chủ. Dựa trên các tính năng cô lập và quản lý tài nguyên của container của Linux, Kubernetes cho phép vận hành các ứng dụng mà không cần can thiệp sâu vào chi tiết nội bộ của từng ứng dụng, cũng như không cần triển khai thủ công lên từng máy chủ riêng lẻ.

Kubernetes cho phép vận hành các ứng dụng phần mềm trên hàng nghìn máy chủ (node) như thể chúng là một khối tài nguyên duy nhất. Bằng cách trừu tượng hóa toàn bộ hạ tầng phần cứng, Kubernetes biến cả trung tâm dữ liệu thành một “máy tính khổng lồ”, nơi các nhà phát triển chỉ cần triển khai thành phần ứng dụng mà không cần biết chi tiết máy chủ vật lý nào sẽ đảm nhận công việc đó. Khi triển khai một ứng dụng có nhiều thành phần, Kubernetes tự động chọn

máy chủ phù hợp cho từng thành phần, triển khai chúng, đồng thời thiết lập khả năng kết nối, giao tiếp dễ dàng giữa các thành phần trong toàn bộ hệ thống.

Một trong những đặc điểm nổi bật của Kubernetes là quy trình triển khai ứng dụng luôn nhất quán, bất kể quy mô của cụm (cluster). Cho dù cụm chỉ bao gồm vài nút hay hàng nghìn nút máy chủ, cách thức triển khai và vận hành ứng dụng vẫn hoàn toàn không thay đổi. Khi mở rộng quy mô cụm bằng cách bổ sung thêm các nút mới, Kubernetes đơn giản coi đây là sự gia tăng về tài nguyên tính toán khả dụng. Các tài nguyên mới sẽ tự động được tích hợp vào cụm, sẵn sàng để các ứng dụng đã triển khai sử dụng mà không cần thay đổi quy trình phát triển hay triển khai của người dùng.

Quay trở lại với vấn đề quy trình phát triển phần mềm, Kubernetes ra đời đã mở ra một cách tiếp cận mới, cho phép các nhà phát triển (developers) tự triển khai ứng dụng của họ và thực hiện các lần triển khai thường xuyên theo nhu cầu, mà không cần sự hỗ trợ trực tiếp từ đội ngũ vận hành (operations). Không chỉ mang lại lợi ích cho các nhà phát triển, Kubernetes còn hỗ trợ đội ngũ vận hành bằng cách tự động giám sát, phát hiện lỗi phần cứng, và lên lịch lại ứng dụng một cách tự động khi có sự cố xảy ra. Vai trò của các quản trị viên hệ thống (sysadmins) cũng dần dịch chuyển: thay vì giám sát chi tiết từng ứng dụng riêng lẻ, họ tập trung chủ yếu vào việc giám sát và quản lý Kubernetes cũng như cơ sở hạ tầng tổng thể, trong khi Kubernetes tự động đảm nhiệm phần vận hành ứng dụng.

Nói một cách đơn giản, Kubernetes giúp đội ngũ phát triển và vận hành tập trung hơn vào công việc phát triển sản phẩm và cải tiến dịch vụ, trong khi các vấn đề về hạ tầng phần cứng và việc vận hành ứng dụng thường ngày đã được hệ thống tự động hóa và tối ưu hóa.

1.3.3. Các hệ thống điều phối container khác

Ngoài Kubernetes — nền tảng phổ biến nhất hiện nay, còn có nhiều hệ thống điều phối khác như Docker Swarm, Apache Mesos với Marathon, Nomad của HashiCorp, và OpenShift. Mỗi hệ thống có những ưu điểm riêng, từ thiết kế đơn giản dễ sử dụng (Docker Swarm, Nomad) đến khả năng mở rộng mạnh mẽ cho hệ thống quy mô lớn (Kubernetes, Mesos).



Hình 1.7 — Các hệ thống điều phối container phổ biến

Các nền tảng này đều hướng tới mục tiêu chung là tối ưu hóa việc sử dụng tài nguyên hạ tầng, đảm bảo tính sẵn sàng cao cho ứng dụng, và giảm tải công việc vận hành thủ công cho các đội ngũ phát triển và quản trị hệ thống.

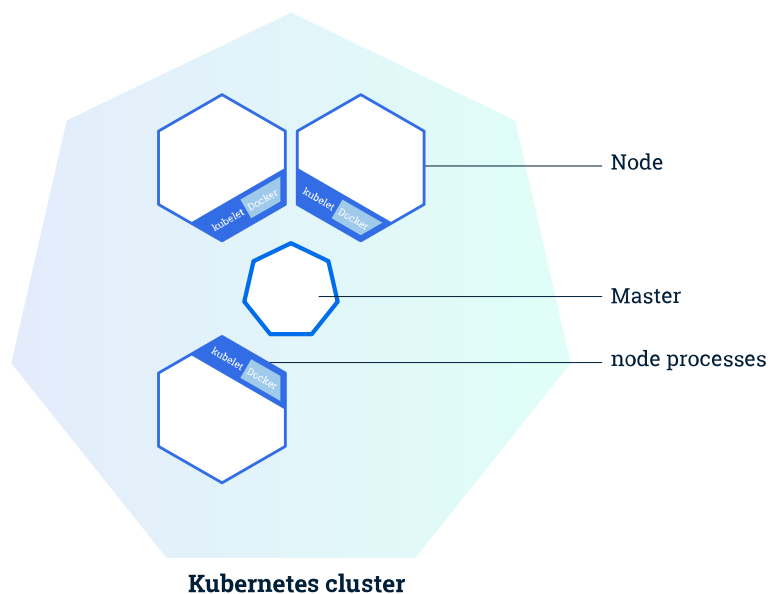
Trong khuôn khổ của bài báo cáo này, chúng tôi không đi sâu so sánh hoặc mô tả chi tiết về các hệ thống điều phối container khác ngoài Kubernetes. Các nội dung tiếp theo sẽ tập trung vào việc tìm hiểu và phân tích Kubernetes như một đại diện tiêu biểu cho mô hình điều phối container hiện đại.

1.4. Kubernetes

1.4.1. Cơ bản về Kubernetes

Kubernetes có thể được hình dung như một hệ điều hành cho toàn bộ cụm máy chủ. Thay vì yêu cầu các nhà phát triển phải tích hợp thủ công các dịch vụ cơ sở hạ tầng vào ứng dụng, Kubernetes cung cấp sẵn các dịch vụ như khám phá dịch vụ (service discovery), mở rộng quy mô tự động (autoscaling), cân bằng tải (load balancing), tự phục hồi (self-healing), ... Nhờ vậy, các nhà phát triển có thể tập trung hoàn toàn vào phát triển tính năng ứng dụng, thay vì tiêu tốn thời gian xử lý các vấn đề hạ tầng phức tạp.

Một hệ thống Kubernetes bao gồm một nút master (hoặc control plane) và một hoặc nhiều nút worker. Khi nhà phát triển gửi danh sách các ứng dụng cần triển khai tới nút master, Kubernetes tự động phân phối các ứng dụng này đến các nút worker trong cụm. Việc một thành phần ứng dụng chạy trên nút nào là chi tiết mà cả nhà phát triển lẫn quản trị viên hệ thống không cần quan tâm — Kubernetes tự động tối ưu hóa quá trình phân phối đó.



Hình 1.8 — Hệ thống Kubernetes

Về bản chất, Kubernetes sẽ quyết định vị trí chạy các container ứng dụng một cách tự động, cung cấp cho các thành phần ứng dụng thông tin cần thiết để tìm thấy nhau và duy trì trạng thái hoạt động ổn định. Việc ứng dụng không phụ thuộc vào vị trí vật lý trên cụm cho phép Kubernetes di chuyển container linh hoạt khi cần thiết, từ đó cải thiện đáng kể mức độ sử dụng tài nguyên so với phương pháp lập lịch thủ công truyền thống.

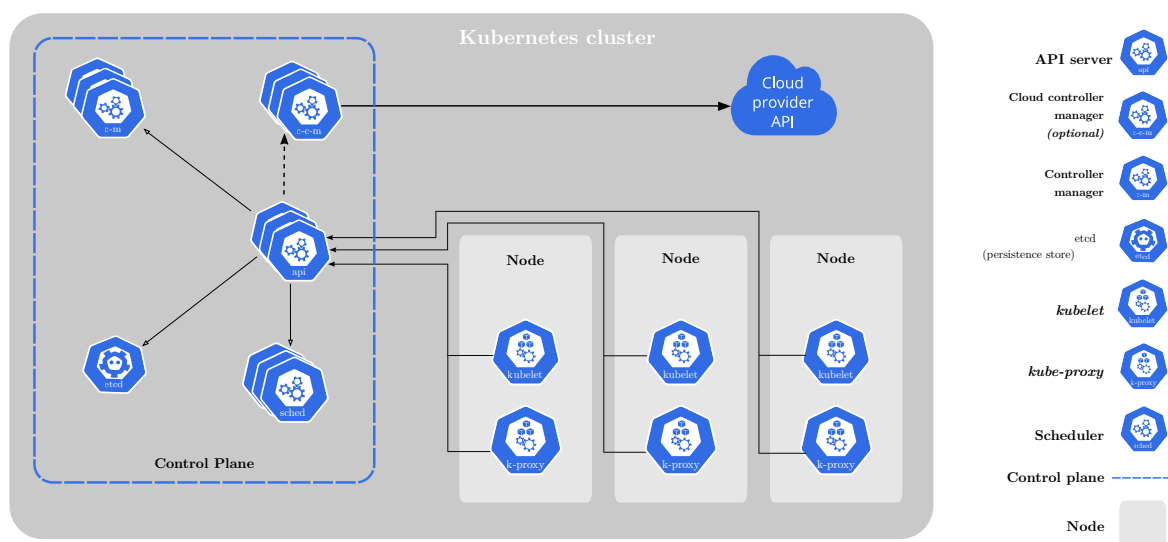
Khi cần thiết, nhà phát triển có thể yêu cầu Kubernetes triển khai một số ứng dụng được chỉ định trên cùng một nút worker. Nhưng dù các thành phần ứng dụng nằm trên cùng nút hay khác nút, Kubernetes đảm bảo rằng chúng luôn có thể liên lạc với nhau một cách ổn định nhờ hệ thống mạng tích hợp.

1.4.2. Kiến trúc Kubernetes

Kubernetes Cluster được thiết kế theo mô hình client-server, với các thành phần đảm nhiệm những vai trò riêng biệt nhằm cung cấp một môi trường quản lý ứng dụng container hóa hiệu quả. Thiết kế này hướng tới mục tiêu đảm bảo tính mở rộng (scalability), tính khả dụng cao (high availability) và tự động hóa trong triển khai, vận hành các ứng dụng.

Về tổng thể, kiến trúc của một Kubernetes Cluster bao gồm hai thành phần chính:

- Master Node: Lưu trữ Kubernetes Control Plane, quản lý trạng thái chung của cụm, chịu trách nhiệm điều phối (orchestration), lập lịch (scheduling) và giám sát toàn bộ hệ thống.
- Worker Node: Thực thi các container ứng dụng thực tế, nhận lệnh từ Control Plane và đảm bảo vận hành các workload theo yêu cầu.



Hình 1.9 — Kiến trúc Kubernetes

1.4.2.1. Control Plane

Control Plane là bộ phận chịu trách nhiệm điều khiển và quản lý toàn bộ Kubernetes Cluster, đảm bảo rằng cụm hoạt động ổn định và đúng theo trạng thái mong muốn. Control Plane điều phối việc triển khai container, giám sát trạng thái các thành phần, và phản ứng tự động khi có thay đổi trong hệ thống.

Các thành phần của Control Plane có thể được triển khai tập trung trên một nút master duy nhất, hoặc phân tán trên nhiều nút khác nhau để tăng tính sẵn sàng cao (high availability) và khả năng chịu lỗi.

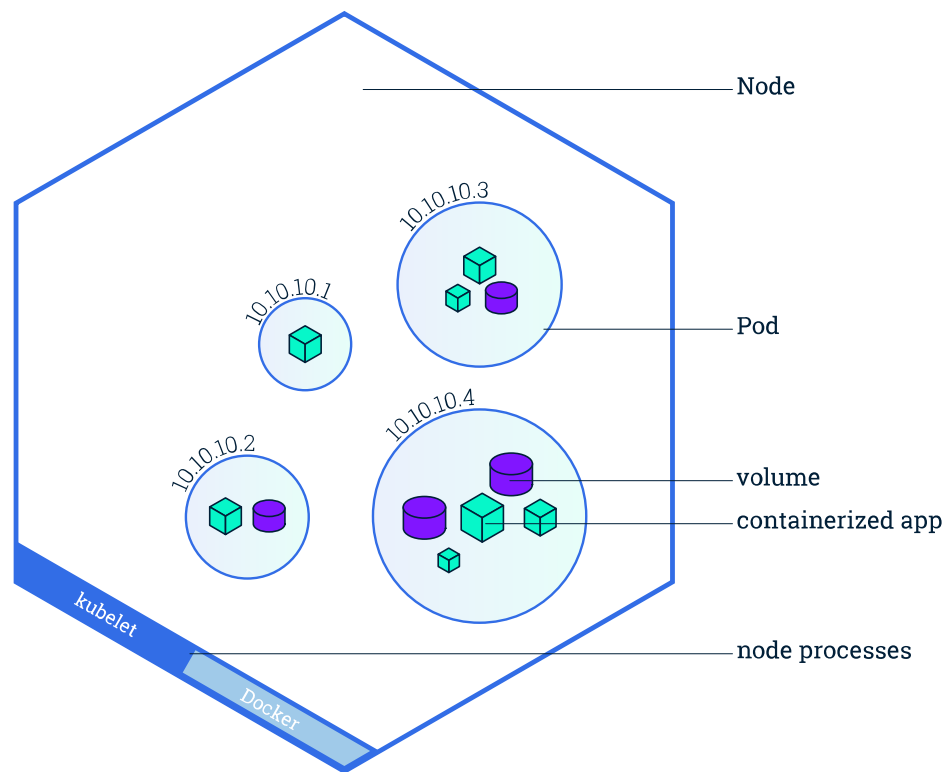
Các thành phần chính trong Control Plane bao gồm:

- kube-apiserver: Thành phần trung tâm, cung cấp giao diện API để các thành phần khác tương tác và quản lý cluster.
- kube-scheduler: Chịu trách nhiệm lập lịch, quyết định node nào sẽ chạy các pod mới dựa trên tài nguyên khả dụng và chính sách đã thiết lập.
- kube-controller-manager: Thực hiện các chức năng ở cấp độ cụm, chẳng hạn như sao chép các thành phần, theo dõi các nút worker, xử lý lỗi nút.
- etcd: Hệ thống lưu trữ key-value phân tán, lưu trữ toàn bộ trạng thái cấu hình và dữ liệu của cluster.
- cloud-controller-manager (nếu sử dụng môi trường cloud): Quản lý tương tác giữa Kubernetes và các dịch vụ của nhà cung cấp cloud như load balancer, volume lưu trữ,...

Mặc dù các thành phần của Control Plane chịu trách nhiệm nắm giữ và điều khiển trạng thái tổng thể của cụm, chúng không trực tiếp thực thi các ứng dụng mà người dùng triển khai. Việc chạy các container ứng dụng thực tế được giao cho các nút worker (worker nodes).

1.4.2.2. Node

Node là nơi thực thi các container ứng dụng trong một Kubernetes Cluster. Mỗi node là một máy worker có nhiệm vụ nhận chỉ thị từ Control Plane, khởi chạy và quản lý các container, theo dõi tài nguyên cục bộ, và đảm bảo các ứng dụng luôn vận hành ổn định.

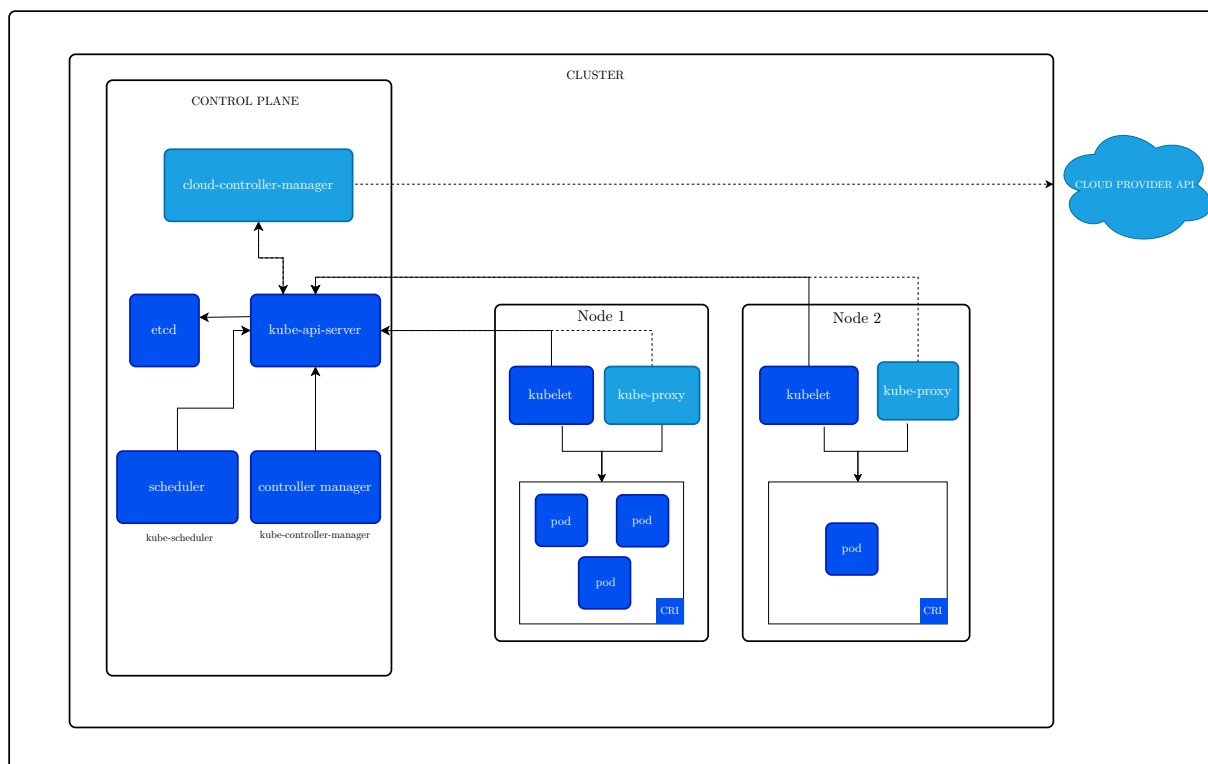


Hình 1.10 — Node

Các chức năng này được thực hiện thông qua các thành phần chính sau đây trên mỗi node:

- Container Runtime: Công cụ chạy các container (ví dụ: containerd, CRI-O, hoặc Docker).
- kubelet: Thành phần chịu trách nhiệm giao tiếp với Control Plane, nhận lệnh và quản lý các container trên node đó.
- kube-proxy: Quản lý mạng nội bộ, định tuyến lưu lượng đến đúng container, hỗ trợ cân bằng tải mạng.

1.4.3. Một số thành phần cơ bản của Kubernetes

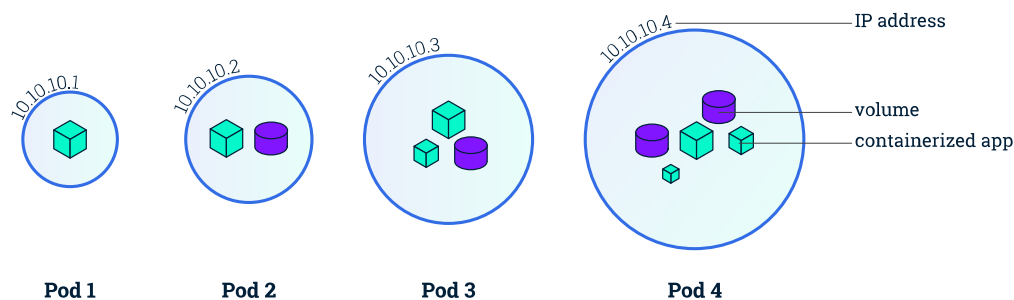


Hình 1.11 — Các thành phần của Kubernetes

1.4.3.1. Pod

Một Pod trong Kubernetes là nhóm gồm một hoặc nhiều container được triển khai cùng nhau, chia sẻ tài nguyên lưu trữ, tài nguyên mạng và một cấu hình chung xác định cách thức vận hành các container bên trong. Trong Kubernetes, Pod là đơn vị nhỏ nhất được triển khai và lập lịch. Khi người dùng gửi yêu cầu tạo ứng dụng, Kubernetes sinh ra các đối tượng Pod, và trình lập lịch (scheduler) sẽ quyết định vị trí triển khai Pod trong cụm.

Trong thực tế, một Pod thường chỉ chứa một container duy nhất. Tuy nhiên, khi một Pod chứa nhiều container, tất cả các container đó luôn chạy trên cùng một nút worker, chia sẻ cùng địa chỉ IP và không gian lưu trữ, và được quản lý như một khối thống nhất.



Hình 1.12 — Pod trong Kubernetes

Khái niệm Pod có thể được liên tưởng đến môi trường truyền thống, nơi các ứng dụng liên quan thường chạy chung trên cùng một máy vật lý hoặc máy ảo. Trong môi trường đám mây, Kubernetes sử dụng Pod để gom nhóm các container ứng dụng có liên quan vào một đơn vị logic, giúp tối ưu hóa khả năng vận hành, phối hợp và chia sẻ tài nguyên giữa các thành phần bên trong hệ thống.

1.4.3.2. Kubelet

Kubelet là một thành phần cốt lõi trong hệ thống Kubernetes, chạy trên mỗi worker node. Vai trò chính của kubelet là đảm bảo rằng các container ứng dụng trong các Pod được khởi chạy và duy trì trạng thái đúng với yêu cầu đã khai báo trong cluster.

Khi một Pod đã được trình lập lịch gán vào một node cụ thể, kubelet trên node đó sẽ:

- Theo dõi trạng thái các Pod được phân công cho mình bằng cách liên tục truy vấn thông tin,
- Kéo (pull) các container image cần thiết từ Registry,
- Tạo và quản lý container bằng cách sử dụng hệ thống Container Runtime cục bộ (ví dụ như containerd, CRI-O hoặc Docker),
- Giám sát sức khỏe (health check) và trạng thái hoạt động của các container (liveness, readiness),
- Báo cáo trạng thái Pod và container để cập nhật vào trạng thái chung của cluster.

Ngoài ra, kubelet còn chịu trách nhiệm:

- Quản lý PodSpec (cấu hình mô tả Pod) cục bộ,

- Đảm bảo rằng nếu một container thất bại (crash hoặc unhealthy), nó sẽ tự động khởi động lại theo chính sách khôi phục được khai báo (restartPolicy),
- Xử lý việc gắn kết các thành phần bộ nhớ lưu trữ theo yêu cầu của Pod.

Về cơ bản, kubelet đóng vai trò như “quản gia” của mỗi node worker: tiếp nhận mệnh lệnh triển khai từ Control Plane và đảm bảo rằng tài nguyên ứng dụng trên node đó vận hành đúng yêu cầu mong đợi.

Một điểm quan trọng cần lưu ý là kubelet chỉ quản lý các container được tạo ra bởi Kubernetes, không quản lý các container được khởi tạo thủ công bên ngoài hệ thống Kubernetes (ví dụ: container do người dùng tự tạo bằng lệnh docker run trực tiếp trên node). Điều này đảm bảo rằng kubelet chỉ chịu trách nhiệm đối với các workload do Kubernetes kiểm soát, từ đó duy trì tính nhất quán và độ tin cậy của cụm.

1.4.3.3. Kube API Server

Kube API Server cung cấp giao diện lập trình (API) chính thức và là cổng giao tiếp duy nhất giữa các thành phần bên trong cluster cũng như giữa người dùng với hệ thống Kubernetes. Do đó, Kube API Server đóng vai trò như trung tâm điều phối thông tin giữa các thành phần.

Mọi yêu cầu từ người dùng, các công cụ quản lý (như kubectl, dashboard), hoặc các thành phần nội bộ (như scheduler, controller) đều phải đi qua API Server. Do đó, Kube API Server thực hiện các chức năng chính sau:

- Xử lý và xác thực các yêu cầu (requests): Kube API Server tiếp nhận tất cả các yêu cầu (REST API calls) và thực hiện các bước xác thực (authentication), phân quyền truy cập (authorization), và xác minh hợp lệ yêu cầu (admission control).
- Truy cập và cập nhật trạng thái cluster: Kube API Server là cầu nối giữa người dùng và cơ sở dữ liệu etcd. Khi có thay đổi đối tượng (Pod, Node, Deployment, ConfigMap...), API Server ghi nhận thay đổi đó vào etcd, đồng thời cung cấp trạng thái cluster cho các thành phần khác khi có yêu cầu.
- Cung cấp cơ chế Watch: Ngoài việc trả lời các yêu cầu đọc/ghi, Kube API Server hỗ trợ cơ chế watch: các thành phần như trình lập lịch (scheduler) hoặc trình quản lý điều khiển (Controller Manager) có thể đăng ký theo dõi

các thay đổi tài nguyên, và nhận thông báo (notifications) khi có sự kiện mới (ví dụ Pod mới được tạo ra, Node bị lỗi).

- Đảm bảo tính nhất quán và khả năng mở rộng: Để hỗ trợ các cluster có quy mô lớn, Kube API Server được thiết kế để có thể triển khai nhiều instance (replica) đồng thời phía sau một load balancer. Các instance này đều truy cập etcd chung, đảm bảo tính nhất quán (consistency) và khả năng mở rộng (scalability) cho Control Plane.

1.4.3.4. etcd

etcd là một hệ thống lưu trữ dữ liệu dạng key-value phân tán được Kubernetes sử dụng để duy trì toàn bộ trạng thái của cluster. Đây là thành phần đóng vai trò bộ nhớ trung tâm cho Control Plane, đảm bảo rằng mọi thay đổi đối với các tài nguyên (như Pod, Node, Service, ConfigMap, Secret, v.v.) đều được ghi nhận một cách nhất quán và có thể phục hồi.

Với vị trí là nguồn dữ liệu duy nhất về trạng thái hệ thống, etcd đóng vai trò thiết yếu trong việc duy trì tính đồng bộ và khả năng phục hồi cho toàn bộ cụm Kubernetes.

1.4.3.5. Scheduler

Trình lập lịch (Kube Scheduler) là thành phần của Control Plane chịu trách nhiệm lập lịch (scheduling) — tức là quyết định Pod mới sẽ được chạy trên node nào trong cụm Kubernetes. Kube Scheduler giúp đảm bảo rằng workload được phân bổ hợp lý giữa các node, đáp ứng các yêu cầu về tài nguyên, hiệu năng, ràng buộc logic và chính sách vận hành.

Khi một Pod mới được tạo ra, nó sẽ chưa được gán node. Trình lập lịch sẽ phát hiện Pod này thông qua cơ chế watch của Kube API Server, sau đó thực hiện quy trình lập lịch gồm 3 bước chính

- Thực hiện quá trình lọc node (filtering), loại bỏ các node không đáp ứng yêu cầu (Không đủ CPU/RAM theo yêu cầu, Node nằm trong danh sách loại trừ của Pod, ...)
- Từ danh sách node khả thi, scheduler gán điểm số cho từng node theo các tiêu chí ưu tiên; lựa chọn node có điểm số cao nhất để gán cho Pod,
- Gửi yêu cầu binding thông qua API Server, sau đó kubelet trên node được chỉ định sẽ tiếp nhận Pod và thực thi

Kube Scheduler được thiết kế theo kiến trúc plugin, hỗ trợ tùy chỉnh toàn bộ quy trình lập lịch và hỗ trợ việc triển khai nhiều scheduler khác nhau cùng lúc trong một cụm. Kubernetes cung cấp nhiều plugin mặc định (như NodeResourcesFit, TaintToleration, NodeAffinity, InterPodAffinity...), người dùng có thể:

- Bật/tắt plugin,
- Cấu hình trọng số ưu tiên,
- Viết plugin mới bằng Go để mở rộng logic scheduling,
- Hoặc xây dựng trình lập lịch tùy chỉnh (custom scheduler) hoạt động song song với Kube Scheduler mặc định.

1.4.4. Luồng hoạt động của Kubernetes

Để triển khai một ứng dụng trong Kubernetes, bước đầu tiên là đóng gói ứng dụng thành một hoặc nhiều container image. Các image này sau đó được đẩy lên một image registry (như Docker Hub, Google Container Registry, hoặc registry nội bộ), để các node trong cụm có thể truy xuất khi cần triển khai.

Tiếp theo, người dùng gửi một mô tả chi tiết về ứng dụng đến Kubernetes API Server. Mô tả này thường được viết dưới dạng tệp YAML hoặc JSON, bao gồm:

- Thông tin về image container cho từng thành phần của ứng dụng,
- Mối quan hệ giữa các thành phần, yêu cầu về việc đồng vị trí (ví dụ: nhiều container cần chạy trong cùng một Pod),
- Số lượng bản sao (replica) cần triển khai cho mỗi thành phần,
- Các đối tượng dịch vụ (Service) cần tạo để expose ứng dụng nội bộ hoặc ra bên ngoài thông qua một địa chỉ IP ổn định,
- Các chỉ dẫn về phân phối tài nguyên, gán node, chính sách restart, liveness/readiness probe,...

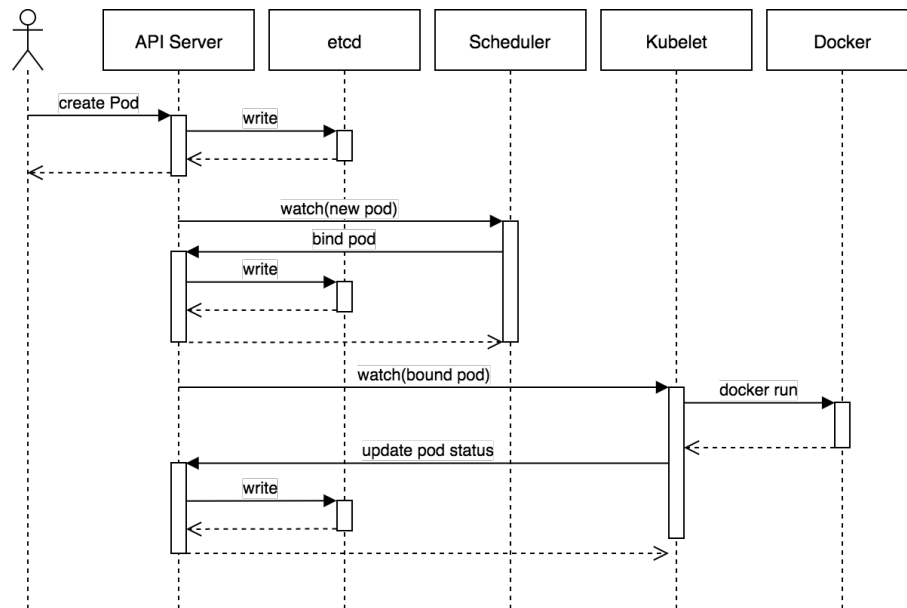
Luồng khởi động ứng dụng

Khi API Server tiếp nhận mô tả ứng dụng, Kubernetes thực hiện các bước sau:

1. Scheduler xác định vị trí triển khai:
 - Scheduler theo dõi các Pod mới chưa được gán node và đánh giá các node khả dụng dựa trên tài nguyên hiện có (CPU, RAM, v.v.), chính sách ràng buộc và ưu tiên.
 - Scheduler chọn node phù hợp và gán Pod đó thông qua Kube API Server.
2. Kubelet triển khai container:

- Kubelet trên mỗi node được giao Pod sẽ nhận thông tin qua API Server, sau đó tương tác với Container Runtime (như containerd hoặc Docker) để kéo image từ registry và khởi chạy các container theo cấu hình đã khai báo.

Ví dụ: Nếu một Pod chứa hai container (ví dụ: ứng dụng chính và container sidecar), thì cả hai sẽ được khởi chạy trên cùng một node, chia sẻ không gian mạng và tài nguyên lưu trữ.



Hình 1.13 — Luồng khởi tạo của Pod

Khi có nhiều bản sao của một Pod được yêu cầu (thông qua một đối tượng như Deployment), Kubernetes sẽ tự động lên lịch chúng trên các node khác nhau trong cluster để tối ưu hoá tài nguyên và tăng khả năng chịu lỗi.

Duy trì trạng thái ứng dụng

Sau khi ứng dụng được triển khai, Kubernetes liên tục theo dõi và đảm bảo rằng trạng thái thực tế của hệ thống khớp với trạng thái mô tả mong muốn. Ví dụ:

- Nếu được yêu cầu duy trì 5 bản sao của một Pod, Kubernetes sẽ tự động tạo lại bản sao nếu một Pod bị lỗi, bị dừng hoặc không phản hồi.
- Nếu một node gặp sự cố (mất kết nối, lỗi phần cứng), các Pod đang chạy trên node đó sẽ được tự động lên lịch lại trên các node khác còn khả dụng.

Các probe như liveness và readiness được sử dụng để theo dõi sức khỏe container và quyết định khi nào cần khởi động lại hoặc loại bỏ container không ổn định.

Mở rộng quy mô ứng dụng

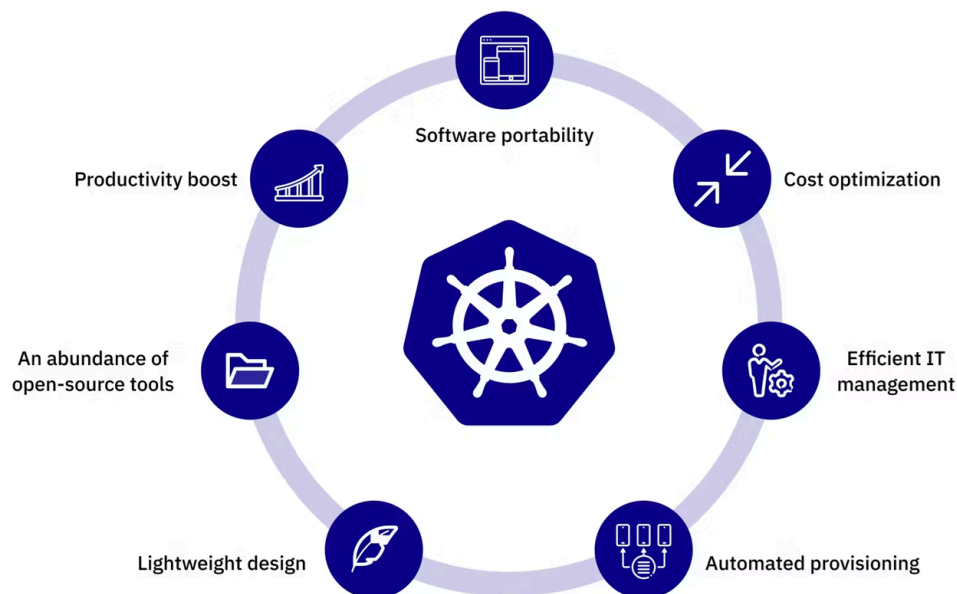
Khi ứng dụng đang chạy, người vận hành có thể thay đổi số lượng bản sao (replica) bất kỳ lúc nào thông qua API Server. Kubernetes sẽ tự động tạo thêm hoặc huỷ bỏ Pod để khớp với số lượng mong muốn. Ngoài việc mở rộng thủ công, Kubernetes cũng hỗ trợ tự động điều chỉnh số lượng bản sao dựa trên các chỉ số thời gian thực như:

- Mức sử dụng CPU hoặc bộ nhớ,
- Số lượng yêu cầu mỗi giây,
- Hoặc bất kỳ chỉ số tùy chỉnh nào khác được ứng dụng cung cấp thông qua hệ thống giám sát.

Cơ chế này được gọi là Horizontal Pod Autoscaler (HPA), giúp đảm bảo ứng dụng có thể mở rộng linh hoạt theo nhu cầu sử dụng thực tế.

1.4.4.1. Lợi ích của việc sử dụng Kubernetes

Dựa trên kiến trúc thiết kế và các tính năng vận hành, Kubernetes mang lại nhiều lợi ích thiết thực khi được áp dụng vào triển khai thực tế. Phần dưới đây sẽ tổng hợp và phân tích những giá trị cốt lõi mà Kubernetes đem lại trong quá trình vận hành hệ thống.



Hình 1.14 — Lợi ích của việc sử dụng Kubernetes

Đơn giản hóa triển khai ứng dụng

Khi một cụm Kubernetes đã được thiết lập trên toàn bộ hạ tầng máy chủ, nhóm vận hành không còn cần can thiệp thủ công vào quá trình triển khai ứng dụng. Nhờ đặc tính tự chứa của container (bao gồm mã nguồn, thư viện và môi trường thực thi), việc cài đặt thủ công trên từng máy chủ trở nên không cần thiết. Ứng dụng có thể được triển khai tự động trên bất kỳ node nào trong cụm chỉ với một bản mô tả YAML mà không đòi hỏi sự trợ giúp từ quản trị viên hệ thống.

Kubernetes trừu tượng hóa toàn bộ các node worker trong cụm như một khối tài nguyên tính toán thống nhất, giúp các nhà phát triển không cần quan tâm đến chi tiết phần cứng cụ thể của từng node. Họ có thể triển khai ứng dụng một cách độc lập, chỉ cần mô tả yêu cầu tài nguyên, số bản sao, và các đặc tả logic khác.

Trong các trường hợp đặc biệt khi phần cứng các node không giống nhau — chẳng hạn một số node sử dụng SSD trong khi số khác dùng HDD — Kubernetes vẫn cho phép điều khiển linh hoạt thông qua các chỉ dẫn, giúp đảm bảo ứng dụng được lên lịch đúng trên phần cứng mong muốn. Điều này loại bỏ hoàn toàn nhu cầu chỉ định thủ công node triển khai như trong cách vận hành truyền thống.

Tối ưu hóa sử dụng hạ tầng

Khi ứng dụng được triển khai thông qua Kubernetes, hệ thống có khả năng tự động chọn node phù hợp nhất để chạy mỗi Pod, dựa trên yêu cầu tài nguyên của Pod và mức sử dụng tài nguyên thực tế trên các node. Việc này tách rời hoàn toàn ứng dụng khỏi hạ tầng vật lý, giúp container có thể di chuyển linh hoạt trong cụm bất kỳ lúc nào.

Khả năng này không chỉ hỗ trợ đóng gói hiệu quả nhiều workload trên cùng một node (bin packing), mà còn tối ưu hóa việc sử dụng tài nguyên phần cứng một cách vượt trội so với thao tác thủ công. Trong khi con người khó có thể tính toán tổ hợp tối ưu cho hàng trăm ứng dụng và node, Kubernetes thực hiện điều đó một cách chính xác và tức thời.

Tự phục hồi và tăng tính sẵn sàng

Một trong những giá trị cốt lõi của Kubernetes là khả năng tự giám sát và tự phục hồi. Khi một Pod hoặc node gặp sự cố (chẳng hạn như container ngừng phản hồi hoặc toàn bộ node bị lỗi), Kubernetes sẽ tự động lên lịch lại Pod đó sang node khác trong cụm mà không cần sự can thiệp của con người.

Nhờ vậy, nhóm vận hành có thể tập trung vào việc sửa chữa phần cứng thay vì phải di chuyển ứng dụng bằng tay. Khi hệ thống có đủ năng lực dự phòng, Kubernetes thậm chí có thể duy trì trạng thái ổn định của ứng dụng mà không cần phản ứng khẩn cấp từ con người.

Tự động mở rộng

Kubernetes không chỉ duy trì trạng thái triển khai ổn định, mà còn có thể tự động điều chỉnh quy mô của ứng dụng dựa trên tải thực tế. Bằng cách tích hợp với Horizontal Pod Autoscaler (HPA) hoặc Cluster Autoscaler, hệ thống có thể tăng hoặc giảm số lượng bản sao của một ứng dụng theo thời gian thực, dựa trên các chỉ số như CPU, bộ nhớ, hoặc số lượng truy cập.

Trong môi trường đám mây, Kubernetes còn có khả năng mở rộng toàn bộ cụm bằng cách tự động thêm hoặc loại bỏ node thông qua API của nhà cung cấp, giúp hệ thống phản ứng linh hoạt với biến động tài nguyên mà không cần sự giám sát liên tục từ nhóm vận hành.

Mã nguồn mở

Kubernetes là một nền tảng mã nguồn mở được phát triển và duy trì bởi một cộng đồng rộng lớn, với sự tham gia của nhiều tập đoàn công nghệ hàng đầu. Việc là phần mềm mã nguồn mở mang lại những lợi ích quan trọng như:

- Khả năng kiểm soát và tùy chỉnh linh hoạt: Người dùng có thể tự điều chỉnh, mở rộng hoặc xây dựng các thành phần theo nhu cầu riêng, thay vì bị giới hạn bởi nhà cung cấp.
- Tính minh bạch và an toàn: Mã nguồn được công khai giúp tăng tính minh bạch, cho phép cộng đồng kiểm tra, phát hiện và xử lý lỗ hổng bảo mật kịp thời.
- Hệ sinh thái phong phú và hỗ trợ lâu dài: Hàng nghìn dự án và công cụ tương thích được phát triển xoay quanh Kubernetes, từ đó thúc đẩy sự đổi mới liên tục và cung cấp nhiều lựa chọn giải pháp cho người dùng.
- Không bị ràng buộc nhà cung cấp (vendor lock-in): Kubernetes cho phép triển khai trên nhiều nền tảng hạ tầng khác nhau (on-premises, multi-cloud, hybrid cloud) mà không phụ thuộc vào một nhà cung cấp đám mây duy nhất.

Với những lợi thế này, Kubernetes không chỉ là một công cụ kỹ thuật thuần túy, mà còn là nền tảng chiến lược giúp các tổ chức xây dựng các hệ thống ứng dụng hiện đại, mở và bền vững.

Chương 2

Bộ lập lịch Kubernetes

2.1. Giới thiệu chung

Trong hệ sinh thái điện toán đám mây hiện đại, Kubernetes đã nổi lên như một nền tảng quản lý container không thể thiếu cho các doanh nghiệp từ nhỏ đến lớn. Tại trung tâm của hệ thống phức tạp này, bộ lập lịch (Scheduler) đóng vai trò quan trọng như một bộ não điều phối, quyết định cách phân bổ tài nguyên tính toán một cách tối ưu nhất.

Bộ lập lịch không chỉ đơn thuần là một cơ chế phân bổ tài nguyên mà còn là một hệ thống ra quyết định phức tạp, cân nhắc hàng chục yếu tố khác nhau để đảm bảo rằng mỗi Pod được đặt vào Node phù hợp nhất. Điều này tương tự như một người quản lý nhà hàng giỏi biết cách sắp xếp khách hàng vào những bàn phù hợp dựa trên kích thước nhóm, yêu cầu đặc biệt, và trạng thái của nhà hàng tại thời điểm đó.

Sự phức tạp của bộ lập lịch xuất phát từ việc nó phải cân bằng giữa nhiều mục tiêu đôi khi xung đột nhau: tối đa hóa việc sử dụng tài nguyên để tiết kiệm chi phí, đảm bảo hiệu suất ứng dụng, duy trì tính sẵn sàng cao, và tuân thủ các ràng buộc về bảo mật, phần cứng cũng như chính sách kinh doanh.

Bộ lập lịch mặc định của Kubernetes có tên là kube-scheduler, tuy nhiên người dùng cũng có thể tùy chỉnh hoặc xây dựng các bộ lập lịch riêng phục vụ những nhu cầu đặc thù. Trong chương này, chúng ta sẽ đi sâu vào cơ chế hoạt động, các bước chính trong quy trình lập lịch, các yếu tố ảnh hưởng đến quyết định lập lịch và khả năng mở rộng thông qua cấu hình hoặc viết bộ lập lịch tùy chỉnh.

2.2. Khái niệm lập lịch trong Kubernetes

Lập lịch (scheduling) trong Kubernetes là quá trình xác định và gán một Pod chưa được gán Node đến một Node phù hợp nhất trong cluster để đảm bảo Pod

có thể chạy. Mỗi khi có một Pod mới được tạo ra (thường thông qua Deployment, StatefulSet hoặc trực tiếp qua API), hệ thống sẽ đặt nó ở trạng thái “chưa được lập lịch” (Pending) cho đến khi bộ lập lịch chọn một Node để chạy nó.

Trong một cụm Kubernetes, các Node thường có năng lực phần cứng khác nhau (CPU, RAM, ổ đĩa, GPU...) và đồng thời cũng có thể có các nhãn, taint, hoặc chính sách đặc thù. Chính vì vậy, không phải Pod nào cũng có thể chạy trên bất kỳ Node nào. Bộ lập lịch cần phải đảm bảo rằng Pod được chạy trên Node có đủ tài nguyên và đáp ứng các yêu cầu mà Pod đưa ra.

2.3. Trình lập lịch mặc định kube-scheduler

kube-scheduler là bộ lập lịch mặc định trong hệ thống Kubernetes, hoạt động như một thành phần độc lập thuộc Control Plane. Vai trò chính của nó là quyết định Node nào trong cụm (cluster) sẽ được gán để chạy một Pod mới.

Bất cứ khi nào một Pod được tạo ra mà chưa được gán Node (.spec.nodeName chưa có giá trị), kube-scheduler sẽ tự động phát hiện và bắt đầu quá trình lập lịch cho Pod đó.

2.3.1. Kiến trúc

Về mặt kiến trúc, kube-scheduler được tổ chức thành các thành phần chính sau:

2.3.1.1. Chính sách (Policy)

Hiện tại, cấu hình khởi chạy của chính sách lập lịch được sử dụng bởi kube-scheduler hỗ trợ ba định dạng: tệp cấu hình (configuration file), tham số dòng lệnh (command-line parameter), và ConfigMap. Chính sách lập lịch có thể được cấu hình để chỉ định các bộ lọc (predicates), các ưu tiên (priorities), các bộ mở rộng (extenders), và các plugin của Scheduler Framework mới nhất được sử dụng trong quá trình lập lịch chính.

2.3.1.2. Trình thông báo (Informer)

Khi khởi động, Kubernetes scheduler lấy dữ liệu cần thiết để lập lịch từ API server của Kubernetes thông qua một informer bằng cách sử dụng các API List và Watch. Dữ liệu này bao gồm các Pod, Node, Persistent Volumes (PV), và Persistent Volume Claim (PVC). Những dữ liệu này được tiền xử lý và lưu trữ dưới dạng dữ liệu được bộ nhớ đệm của Kubernetes scheduler.

2.3.1.3. Chuỗi quy trình lập lịch (Schedule Pipeline)

Kubernetes scheduler chèn các Pod cần được lập lịch vào một hàng đợi (queue) thông qua một informer. Các Pod này sẽ được lấy ra tuần tự từ hàng đợi và đưa vào chuỗi quy trình lập lịch.

Chuỗi quy trình lập lịch được chia thành ba luồng chính: luồng lập lịch (schedule thread), luồng chờ (wait thread), và luồng ràng buộc (bind thread).

2.3.1.3.1. Luồng lập lịch (Schedule Thread)

Luồng lập lịch được chia thành các giai đoạn sau:

- Tiền lọc (Pre-Filter): Chuẩn bị và kiểm tra dữ liệu trước khi lọc.
- Lọc (Filter): Lựa chọn các Node phù hợp với yêu cầu của Pod.
- Hậu lọc (Post-Filter): Xử lý sau khi lọc, thường kiểm tra các Node được chọn.
- Chấm điểm (Score): Chấm điểm và sắp xếp các Node đã chọn.
- Đặt trước (Reserve): Đặt Pod vào bộ nhớ đệm của Node được sắp xếp tối ưu, cho thấy Pod đã được gán vào Node đó. Điều này giúp các Pod tiếp theo trong hàng đợi lập lịch biết được Pod đã được gán vào Node nào khi thực hiện các bước lọc và chấm điểm.

2.3.1.3.2. Luồng chờ (Wait Thread)

Luồng chờ đảm nhận việc chờ đợi các tài nguyên liên quan đến Pod sẵn sàng, chẳng hạn như chờ việc tạo thành công Persistent Volume (PV) hoặc chờ việc lập lịch thành công của các Pod liên quan trong trường hợp Gang scheduling.

2.3.1.3.3. Luồng ràng buộc (Bind Thread)

Luồng ràng buộc đảm nhận việc lưu trữ vĩnh viễn các liên kết giữa Pod và Node trên API server của Kubernetes.

Các Pod được lập lịch theo tuần tự trong luồng lập lịch (schedule thread), nhưng trong các luồng chờ (wait thread) và ràng buộc (bind thread), quá trình lập lịch diễn ra một cách bất đồng bộ và song song. Điều này giúp tối ưu hóa hiệu suất lập lịch và tăng tốc độ triển khai Pod.

2.3.2. Quá trình lập lịch

Dưới đây là các giai đoạn cụ thể trong quy trình lập lịch của Kubernetes:

2.3.2.1. QueueSort

Giai đoạn đầu tiên trong quy trình lập lịch là QueueSort, nơi các Pod trong hàng đợi lập lịch được sắp xếp theo độ ưu tiên. Các Pod có PriorityClass cao hơn sẽ được xử lý trước. Kubernetes sử dụng cơ chế này để đảm bảo rằng các Pod quan trọng nhất được lập lịch và triển khai đầu tiên. Điều này giúp cho các ứng dụng có yêu cầu khẩn cấp hoặc cần độ sẵn sàng cao có thể được triển khai mà không bị chậm trễ bởi các Pod ít quan trọng hơn.

2.3.2.2. PreFilter

Sau khi các Pod được xếp thứ tự, Kubernetes thực hiện PreFilter, là một bước kiểm tra nhanh để xác định xem Pod có thể được lập lịch hay không, trước khi thực hiện các bước lọc chi tiết. Giai đoạn này giúp giảm thiểu thời gian và tài nguyên bằng cách kiểm tra các điều kiện đơn giản và quan trọng như việc liệu có bất kỳ Node nào trong cluster hay không, hoặc kiểm tra xem các tài nguyên cơ bản có đủ để khởi tạo Pod hay không. Đây là một bước quan trọng trong việc tối ưu hóa hiệu suất lập lịch, giúp giảm tải cho các bước lọc phức tạp hơn sau này.

2.3.2.3. Filter

Tiếp theo, giai đoạn Filter là nơi các Node không phù hợp sẽ bị loại bỏ. Đây là bước mà Kubernetes kiểm tra các điều kiện chi tiết hơn để đảm bảo rằng Pod chỉ được triển khai lên các Node đáp ứng các yêu cầu nhất định. Các bộ lọc phổ biến trong giai đoạn này bao gồm kiểm tra tài nguyên (CPU, bộ nhớ) có đủ để chạy Pod hay không, kiểm tra NodeAffinity, kiểm tra PodToleratesNodeTaints, và các điều kiện khác như trạng thái sẵn sàng của Node. Giai đoạn này có thể tốn kém về tài nguyên, do đó các bước PreFilter hiệu quả trước đó sẽ giúp giảm thiểu khối lượng công việc trong giai đoạn Filter.

2.3.2.4. PreScore

Sau khi loại bỏ các Node không phù hợp, giai đoạn PreScore sẽ bắt đầu tính toán các dữ liệu trung gian cần thiết cho bước chấm điểm (Scoring). Giai đoạn này chuẩn bị các thông tin về Node, như tài nguyên sử dụng, tình trạng của Node,

và các yếu tố khác, để phục vụ cho quá trình chấm điểm chính xác trong giai đoạn tiếp theo.

2.3.2.5. Score

Giai đoạn Score là nơi Kubernetes chấm điểm các Node khả thi dựa trên các yếu tố như tài nguyên sử dụng, khả năng đáp ứng yêu cầu của Pod, và các yếu tố khác. Các chiến lược chấm điểm có thể bao gồm các ưu tiên như tài nguyên ít sử dụng, sự cân bằng giữa các tài nguyên, hoặc các yêu cầu về NodeAffinity. Mỗi Node được gán một điểm số trong khoảng từ 0 đến 100, hoặc có thể là một thang điểm tùy chỉnh. Node có điểm cao nhất sẽ là lựa chọn ưu tiên cho việc lập lịch Pod.

2.3.2.6. NormalizeScore

Sau khi các Node được chấm điểm, bước NormalizeScore được thực hiện để chuẩn hóa điểm số giữa các plugin khác nhau. Bởi vì các plugin scoring có thể sử dụng các thang điểm khác nhau, quá trình chuẩn hóa giúp so sánh các Node một cách công bằng và nhất quán. Việc chuẩn hóa điểm số giúp các quyết định lập lịch trở nên chính xác hơn và đảm bảo tính đồng nhất trong quy trình.

2.3.2.7. Reserve

Giai đoạn Reserve là lúc Kubernetes đánh dấu tài nguyên của Node đã được chọn là “đã được đặt trước”. Điều này có nghĩa là các tài nguyên trên Node này sẽ không được sử dụng bởi các Pod khác cho đến khi quá trình lập lịch hoàn tất và Pod được triển khai. Giai đoạn này giúp đảm bảo rằng tài nguyên cần thiết cho Pod sẽ không bị chia sẻ hoặc xung đột trong quá trình triển khai.

2.3.2.8. Permit

Trong bước Permit, các plugin có thể can thiệp vào quá trình lập lịch và quyết định liệu có chấp nhận, từ chối hoặc trì hoãn việc lập lịch hay không. Đây là một cơ chế linh hoạt cho phép các plugin tùy chỉnh can thiệp vào quyết định lập lịch trước khi thực hiện bước PreBind. Ví dụ, plugin có thể yêu cầu một điều kiện bổ sung như đảm bảo rằng Pod không bị gán vào một Node chưa hoàn thành một công việc chuẩn bị nào đó.

2.3.2.9. PreBind

Trước khi Pod được gán vào Node đã chọn, giai đoạn PreBind thực hiện các công việc chuẩn bị cần thiết. Điều này có thể bao gồm các tác vụ như chuẩn bị các volume cần thiết cho Pod, thiết lập các tài nguyên bổ sung như mạng, hoặc xác thực thông tin về Node mà Pod sẽ được triển khai lên. Giai đoạn này đảm bảo rằng tất cả các điều kiện đã sẵn sàng trước khi quá trình gán Pod cho Node bắt đầu.

2.3.2.10. Bind

Giai đoạn cuối cùng trong quy trình lập lịch là Bind. Đây là bước xác nhận chính thức khi kube-scheduler gửi yêu cầu tới API server để gán Pod vào Node đã chọn. Quá trình này kết thúc việc lập lịch và đánh dấu Pod là đã được triển khai. Sau khi Bind thành công, Node đã được chọn sẽ bắt đầu triển khai Pod.

2.3.2.11. PostBind

Cuối cùng, sau khi Pod đã được gán vào Node, Kubernetes thực hiện PostBind, nơi các công việc hậu kỳ được thực hiện. Điều này có thể bao gồm việc cập nhật bộ nhớ cache nội bộ của hệ thống hoặc tiến hành các tác vụ giám sát để theo dõi tình trạng của Pod trên Node. Giai đoạn này giúp hoàn tất quá trình lập lịch và duy trì thông tin hệ thống luôn cập nhật.

2.4. Các phương pháp điều khiển lập lịch Pod

Trong Kubernetes, kube-scheduler thường đủ thông minh để tự động lựa chọn các Node phù hợp nhằm đảm bảo hiệu suất và sự phân bố hợp lý của các Pod trong cụm. Tuy nhiên, trong nhiều tình huống thực tế, người dùng – đặc biệt là các quản trị viên hệ thống hoặc các kỹ sư vận hành – muốn kiểm soát chính xác nơi một Pod sẽ chạy. Mục tiêu có thể là để đảm bảo tuân thủ các chính sách bảo mật, tận dụng phần cứng chuyên dụng như GPU hoặc SSD, phân tán ứng dụng theo vùng địa lý để tăng khả năng chống lỗi, hoặc cô lập workload để tránh can nhiễu. Kubernetes hỗ trợ nhiều công cụ và cơ chế để đạt được điều đó, từ các cách đơn giản như nodeSelector đến các cơ chế phức tạp như taints/tolerations và affinity/anti-affinity. Dưới đây là phân tích chi tiết từng cơ chế:

2.4.1. Node Selector

nodeSelector là cơ chế cơ bản và đơn giản nhất để kiểm soát việc lập lịch Pod. Nó hoạt động bằng cách chỉ định rằng Pod chỉ nên được lập lịch trên các Node có nhãn (label) phù hợp. Trong cấu hình Pod, thêm trường nodeSelector cùng với một tập các khóa-giá trị, ví dụ như:

```
...
nodeSelector:
  disktype: ssd
...
```

Kubernetes sẽ chỉ lập lịch Pod này lên các Node có nhãn disktype=ssd. Cách làm này phù hợp với các yêu cầu đơn giản như chỉ định một nhóm máy chuyên dụng để xử lý công việc nặng, hoặc gán workload nhạy cảm lên các Node riêng biệt.

Tuy nhiên, nodeSelector có những hạn chế. Nó chỉ hỗ trợ kiểm tra chính xác các nhãn theo kiểu “bằng nhau” (equality-based) và không thể kết hợp điều kiện phức tạp (ví dụ: nhãn A tồn tại nhưng nhãn B không tồn tại). Hơn nữa, các điều kiện là bắt buộc – nếu không có Node nào phù hợp, Pod sẽ không được lập lịch.

2.4.2. Node Affinity và Anti-Affinity

Để khắc phục hạn chế của nodeSelector, Kubernetes cung cấp nodeAffinity, một tính năng mạnh mẽ hơn, cho phép người dùng viết các ràng buộc phức tạp hơn dựa trên nhãn của Node. Node Affinity hỗ trợ hai loại điều kiện chính:

requiredDuringSchedulingIgnoredDuringExecution: đây là điều kiện bắt buộc. Nếu không có Node nào phù hợp, Pod sẽ không được lập lịch.

preferredDuringSchedulingIgnoredDuringExecution: đây là điều kiện ưu tiên. Nếu có Node phù hợp thì Pod sẽ được lập lịch theo ưu tiên này, còn không thì vẫn lập lịch bình thường.

Ví dụ, có thể viết điều kiện như sau:

```
...
requiredDuringSchedulingIgnoredDuringExecution:
  nodeSelectorTerms:
    - matchExpressions:
      - key: zone
        operator: In
```



```

values:
  - us-east-1a
  - us-east-1b
  ...

```

Khác với nodeSelector, nodeAffinity hỗ trợ các toán tử như In, NotIn, Exists, DoesNotExist, cho phép định nghĩa logic linh hoạt hơn.

Ngoài ra, còn có Pod affinity và anti-affinity – cho phép lập lịch các Pod dựa vào nhãn của các Pod khác. Ví dụ, có thể yêu cầu một Pod mới phải được lập lịch cùng Node với các Pod mang nhãn app=web (affinity), hoặc tránh các Node có nhiều Pod app=cache đang chạy (anti-affinity). Điều này rất hữu ích khi ta muốn:

Gom nhóm các Pod để giảm độ trễ trong giao tiếp giữa chúng.

Phân tán các bản sao Pod để giảm rủi ro mất mát do lỗi phần cứng.

2.4.3. Taints và Tolerations

Taints và Tolerations là cơ chế điều khiển lập lịch theo hướng loại trừ mặc định: đánh dấu (taint) một Node để ngăn không cho Pod được lập lịch lên đó, trừ khi Pod có “toleration” tương ứng. Điều này rất hữu ích trong các tình huống như:

Node chuyên dụng cho workload đặc biệt (như GPU, dữ liệu bảo mật).

Node đang trong quá trình bảo trì, không muốn Pod mới được lập lịch vào.

Một ví dụ về taint:

```
kubectl taint nodes node1 key=value:NoSchedule
```

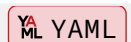


Pod nào không có tolerations tương ứng sẽ không được lập lịch lên node1. Trong manifest của Pod, ta thêm phần:

```

tolerations:
- key: "key"
  operator: "Equal"
  value: "value"
  effect: "NoSchedule"

```



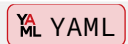
Kết hợp taints và affinity sẽ cho phép cô lập workload một cách linh hoạt và hiệu quả.

2.4.4. Pod Topology Spread Constraints

Đây là một tính năng nâng cao được giới thiệu để đảm bảo các Pod được phân bố đều trên các vùng hoặc nhóm Node, giúp tăng khả năng chống chịu lỗi. Ví dụ, nếu ta triển khai 10 bản sao Pod, ta có thể yêu cầu Kubernetes phân bố chúng đều trên 3 vùng khả dụng (Availability Zones), thay vì đặt tất cả lên một vùng.

Cấu hình sử dụng topologySpreadConstraints, ví dụ:

```
topologySpreadConstraints:
- maxSkew: 1
  topologyKey: topology.kubernetes.io/zone
  whenUnsatisfiable: ScheduleAnyway
  labelSelector:
    matchLabels:
      app: my-app
```



Điều này giúp đảm bảo rằng hệ thống phân tán của người dùng hoạt động ổn định và giảm thiểu tác động nếu một vùng (zone) bị lỗi.

Tóm lại, Kubernetes cung cấp một loạt công cụ đa dạng để điều khiển quá trình lập lịch Pod một cách chi tiết và có kiểm soát. Tùy vào yêu cầu kỹ thuật, hạ tầng và chính sách tổ chức, người dùng có thể chọn các công cụ phù hợp hoặc kết hợp nhiều cơ chế lại để đạt được hiệu quả tối ưu nhất cho việc vận hành ứng dụng trên Kubernetes.

2.5. Các chiến lược nâng cao trong lập lịch Kubernetes

Các chiến lược nâng cao trong lập lịch Kubernetes Trong Kubernetes, việc lập lịch không chỉ đơn giản là phân phối các Pod vào các Node, mà còn liên quan đến việc tối ưu hóa tài nguyên, giảm độ trễ, tăng khả năng chịu lỗi và bảo mật cho ứng dụng. Để đáp ứng các yêu cầu phức tạp và tăng tính linh hoạt trong việc triển khai ứng dụng, Kubernetes cung cấp một số chiến lược nâng cao trong lập lịch. Ba chiến lược quan trọng nhất trong số đó là Topology Spread Constraints, Pod Priority và Preemption, và Scheduling Policies và Scheduling Profiles (Plugin).

2.5.1. Topology Spread Constraints

Topology Spread Constraints giúp người dùng kiểm soát cách các Pod được phân phối trên các Node, vùng (availability zones), hay các khu vực lỗi trong cluster.

Mục đích chính của chiến lược này là giảm thiểu rủi ro khi một khu vực hoặc Node gặp sự cố, giúp ứng dụng duy trì tính khả dụng cao và chịu lỗi tốt hơn.

Khi ta triển khai một ứng dụng có yêu cầu độ sẵn sàng cao hoặc có thể gặp phải các sự cố hạ tầng, việc sử dụng Topology Spread Constraints sẽ giúp phân tán các Pod một cách hợp lý. Ví dụ, trong môi trường đa vùng dữ liệu, ta có thể cấu hình các Pod sao cho chúng được phân bố đều giữa các vùng để tránh tình trạng mất dịch vụ khi một vùng bị gián đoạn.

Kubernetes hỗ trợ cấu hình phân phối này qua các thuộc tính như `maxSkew`, `topologyKey`, và `whenUnsatisfiable`, giúp người dùng định nghĩa các ràng buộc về độ phân tán mà không làm giảm khả năng chịu lỗi của hệ thống. Các chiến lược này không chỉ giúp cải thiện khả năng chịu lỗi mà còn giúp tối ưu hóa hiệu suất và tài nguyên cho các ứng dụng phân tán.

2.5.2. Pod Priority và Preemption

Pod Priority và Preemption là hai cơ chế quan trọng giúp Kubernetes quản lý các Pod trong điều kiện tài nguyên hạn chế. Mỗi Pod trong Kubernetes có thể được gán một mức độ Priority, giúp xác định độ quan trọng của nó đối với hệ thống. Các Pod có Priority cao hơn sẽ có quyền sử dụng tài nguyên khi có sự tranh chấp tài nguyên.

Khi tài nguyên trong cluster không đủ để chạy tất cả các Pod, cơ chế Preemption sẽ được kích hoạt. Kubernetes sẽ hủy các Pod có Priority thấp hơn để tạo không gian cho các Pod có Priority cao hơn. Cơ chế này rất hữu ích trong các tình huống có tài nguyên hạn chế, nơi các dịch vụ quan trọng cần được ưu tiên để chạy.

Cơ chế Pod Priority và Preemption giúp người dùng kiểm soát việc phân phối tài nguyên giữa các Pod dựa trên mức độ quan trọng của chúng. Ví dụ, trong môi trường sản xuất, các dịch vụ cần thiết cho hoạt động của hệ thống (như dịch vụ cơ sở dữ liệu hoặc dịch vụ xử lý thanh toán) có thể được gán Priority cao để đảm bảo rằng chúng sẽ luôn có đủ tài nguyên ngay cả khi hệ thống bị thiếu tài nguyên.

2.5.3. Scheduling Policies và Scheduling Profiles (Plugin)

Scheduling Policies và Scheduling Profiles cung cấp một cách để tùy chỉnh quy trình lập lịch trong Kubernetes. Với Scheduling Policies, người dùng có thể cấu

hình các Predicates để lọc các Node phù hợp và Priorities để chấm điểm các Node, từ đó quyết định nơi nào phù hợp để triển khai Pod.

Trong khi đó, Scheduling Profiles cung cấp một cấu trúc mô-đun cho phép triển khai các Plugins tại các giai đoạn khác nhau trong quy trình lập lịch. Các giai đoạn này bao gồm: QueueSort, Filter, Score, Bind, Reserve, Permit, và nhiều giai đoạn khác. Người dùng có thể tùy chỉnh các giai đoạn lập lịch này để đáp ứng các yêu cầu đặc biệt về cách thức các Pod được lập lịch, từ việc lọc các Node đến việc tính điểm và quyết định việc gán Pod vào Node.

Cấu trúc của Scheduling Profiles giúp người dùng có thể triển khai nhiều bộ lập lịch với các chiến lược khác nhau, hoặc thậm chí triển khai các bộ lập lịch tùy chỉnh cho các tình huống đặc thù. Việc sử dụng Plugin trong Kubernetes giúp mở rộng khả năng lập lịch của hệ thống, đáp ứng những yêu cầu phức tạp hơn mà bộ lập lịch mặc định không thể xử lý.

Các chiến lược Scheduling Policies và Scheduling Profiles mang lại sự linh hoạt và khả năng mở rộng cao trong việc điều chỉnh hành vi lập lịch của Kubernetes. Chúng cho phép người dùng triển khai các giải pháp lập lịch tùy chỉnh để tối ưu hóa việc phân bổ tài nguyên, đáp ứng các yêu cầu đặc biệt của ứng dụng hoặc hạ tầng.

Các chiến lược nâng cao này không chỉ giúp Kubernetes trở thành một hệ thống lập lịch mạnh mẽ mà còn tăng cường khả năng mở rộng và chịu lỗi của các ứng dụng triển khai trên đó. Topology Spread Constraints giúp tối ưu hóa sự phân bổ tài nguyên, Pod Priority và Preemption giúp quản lý tài nguyên trong điều kiện thiếu hụt, và Scheduling Policies & Profiles cung cấp sự linh hoạt để tùy chỉnh hành vi lập lịch cho các yêu cầu đặc biệt.

2.6. Ví dụ chi tiết về một quyết định lập lịch

Giả sử chúng ta có một Pod cần 2 CPU cores và 4GB RAM, có nodeSelector yêu cầu disk-type=ssd, và có affinity ưu tiên gần các Pod có nhãn app=database. Cluster có 5 Node:

- Node A: 4 CPU, 16GB RAM, có nhãn disk-type=ssd, đang chạy 1 Pod với nhãn app=database
- Node B: 4 CPU, 8GB RAM, có nhãn disk-type=ssd, không có Pod app=database

- Node C: 8 CPU, 32GB RAM, không có nhãn `disk-type=ssd`
- Node D: 2 CPU, 16GB RAM, có nhãn `disk-type=ssd`
- Node E: 4 CPU, 16GB RAM, có nhãn `disk-type=ssd`, đang chạy 2 Pod với nhãn `app=database` nhưng CPU đang sử dụng cao (3.5/4 cores)

Trong quá trình lọc:

- Node C bị loại vì không có nhãn `disk-type=ssd`
- Node D bị loại vì không đủ CPU (cần 2 cores)

Trong quá trình chấm điểm:

- Node A: Điểm cao vì đủ tài nguyên, đáp ứng `nodeSelector`, và có affinity với database
- Node B: Điểm trung bình, đáp ứng `nodeSelector` nhưng không có affinity với database
- Node E: Điểm thấp vì mặc dù đáp ứng `nodeSelector` và có affinity mạnh với database, nhưng CPU đã gần đạt ngưỡng

Kết quả: Node A được chọn làm nơi đặt Pod.

2.7. Kết luận

Bộ lập lịch là một phần thiết yếu trong hoạt động của Kubernetes. Thông qua quá trình lọc và chấm điểm, kube-scheduler đảm bảo rằng mỗi Pod được đặt ở vị trí tối ưu nhất trong cluster. Việc hiểu rõ cơ chế hoạt động của kube-scheduler không chỉ giúp người dùng vận hành hệ thống hiệu quả mà còn mở ra khả năng tối ưu hóa hoặc tùy chỉnh phù hợp với đặc thù ứng dụng hoặc hạ tầng của doanh nghiệp.

Chương 3

Trình lập lịch Kubernetes tùy chỉnh

3.1. Kiến trúc của trình lập lịch Scheduler tùy chỉnh

Trong dự án của nhóm, chúng em đã phát triển một scheduler Kubernetes tùy chỉnh, có sẵn tại <https://github.com/HynDuf/custom-kubernetes-scheduler>, nhằm nâng cao khả năng phân bổ pod tiêu chuẩn trong cụm Kubernetes. Scheduler mặc định của Kubernetes chủ yếu xét đến các *yêu cầu tài nguyên* của pod. Scheduler tùy chỉnh của nhóm hướng tới một cách tiếp cận tinh vi hơn bằng cách tích hợp các chỉ số nút theo thời gian thực và tôn trọng các ưu tiên affinity do pod định nghĩa. Phần này trình bày kiến trúc cấp cao của hệ thống, nêu chi tiết các thành phần cốt lõi và cách chúng tương tác với nhau.

3.1.1. Sơ đồ tổng quan

Một sơ đồ khái niệm cho hệ thống scheduler tùy chỉnh của nhóm sẽ minh họa các thực thể và tương tác sau:

1. Cụm Kubernetes:

- **Kubernetes API Server:** Thành phần trung tâm của control plane.
- **Các Node:** Máy chủ làm việc nơi pod được thực thi.
 - **Node Exporter (DaemonSet):** Một instance Node Exporter chạy trên mỗi node, có nhiệm vụ thu thập các chỉ số mức hệ thống.
- **Các Pod:**
 - **Pod cần được phân lịch:** Các pod này được cấu hình rõ ràng với `spec.schedulerName: custom-scheduler`.
 - **Các Pod Khác:** Các pod được quản lý bởi scheduler mặc định hoặc scheduler tùy chỉnh khác.

2. **Monitoring Stack** (thường được triển khai trong namespace **monitoring**):

- **Prometheus Server:** Thành phần này tổng hợp các chỉ số từ tất cả Node Exporter và phục vụ chúng thông qua PromQL.

3. **Hệ thống Scheduler tùy chỉnh** (triển khai trong namespace **kube-system**):

- **Pod Scheduler Tùy Chỉnh:** Pod này chạy ứng dụng Go của nhóm, hiện thân của logic phân lịch tùy chỉnh.

Luồng tương tác chính (được minh họa bằng mũi tên trong sơ đồ):

1. Người dùng hoặc controller tự động tạo một Pod manifest có `schedulerName: custom-scheduler` và gửi nó đến **Kubernetes API Server**.
2. **Pod Scheduler tùy chỉnh** của nhóm liên tục theo dõi **Kubernetes API Server** để tìm các pod chờ được gán scheduler.
3. Khi phát hiện một pod như vậy, scheduler sẽ truy vấn **Kubernetes API Server** để lấy trạng thái và thông số hiện tại của tất cả node để lọc sơ bộ (predicate checks).
4. Sau đó, scheduler sẽ truy vấn **Prometheus Server** để lấy các chỉ số thời gian thực (ví dụ: bộ nhớ khả dụng, phần trăm CPU không hoạt động) của các node đã vượt qua bước lọc.
5. **Prometheus Server** trả lời yêu cầu này bằng dữ liệu đã thu thập từ các instance **Node Exporter** trên mỗi node.
6. Scheduler áp dụng thuật toán tính điểm. Logic này xét đến tài nguyên khả dụng (từ Prometheus) và các tùy chọn affinity do pod định nghĩa để xếp hạng các node tương thích và chọn node phù hợp nhất.
7. Scheduler thông báo quyết định của mình cho **Kubernetes API Server** bằng cách tạo đối tượng **Binding**, gán pod vào node đã chọn.
8. Kubelet trên node được chọn nhận biết việc gán này và bắt đầu chạy pod.

3.1.2. Các thành phần chính và vai trò của chúng

Hệ thống dựa vào nhiều thành phần liên kết chặt chẽ, mỗi thành phần đóng một vai trò riêng:

3.1.2.1. Trình lập lịch Scheduler tùy chỉnh (Golang)

1. **Mô tả:** Đây là thành phần cốt lõi của dự án, một ứng dụng Go triển khai thuật toán lập lịch tùy chỉnh. Mã nguồn được lưu tại <https://github.com/HynDuf/custom-kubernetes-scheduler>.
2. **Vị trí:** Chạy dưới dạng *Deployment* trong namespace `kube-system`, được định nghĩa trong tệp manifest `scheduler/custom-scheduler-deployment.yaml`.
3. **Vai trò:** Cài đặt của trình lập lịch tùy chỉnh.
4. **Trách nhiệm:**
 - Theo dõi liên tục Kubernetes API để phát hiện các pod ở trạng thái `Pending`, chưa có node (`spec.nodeName` rỗng), và có `schedulerName: custom-scheduler`.
 - Thực hiện kiểm tra predicate (xem xét yêu cầu tài nguyên, taint/toleration, node selector) để xác định danh sách node *có thể* chạy pod.
 - Truy vấn Prometheus để lấy chỉ số CPU và bộ nhớ thời gian thực của các node tương thích.
 - Tính toán điểm số cho mỗi node. Điểm số là tổ hợp có trọng số giữa tài nguyên khả dụng và mức độ phù hợp với các affinity do pod định nghĩa. Các trọng số có thể cấu hình qua biến môi trường.
 - Chọn node tương thích có điểm số cao nhất.
 - Tạo `Binding` để gán pod vào node được chọn, gửi đối tượng này đến API server.
 - Tạo đối tượng `Event` của Kubernetes để báo cáo quyết định phân lịch thành công hoặc lỗi gặp phải, nâng cao khả năng quan sát hệ thống.

3.1.2.2. Prometheus Server

1. **Khái niệm:** `Prometheus` là một bộ công cụ mã nguồn mở hỗ trợ việc giám sát và cảnh báo. Nó thu thập các chỉ số từ các mục tiêu được cấu hình theo chu kỳ, đánh giá các biểu thức rule, hiển thị kết quả và có thể kích hoạt cảnh báo.
2. **Vị trí:** Triển khai dưới dạng *Deployment* trong namespace `monitoring`, theo cấu hình trong `prometheus/prometheus-deployment.yaml`.

3. **Vai trò:** Thu thập, lưu trữ và cung cấp dữ liệu dạng chuỗi thời gian mô tả trạng thái các node trong cụm.

4. **Trách nhiệm trong hệ thống:**

- Thu thập dữ liệu định kỳ từ các mục tiêu, đặc biệt là các instance **Node Exporter**.
- Lưu trữ dữ liệu này trong cơ sở dữ liệu chuỗi thời gian.
- Cung cấp API HTTP hỗ trợ ngôn ngữ PromQL. Scheduler sử dụng API này để lấy các chỉ số như `node_memory_MemAvailable_bytes` và `avg by (instance)` (

3.1.2.3. Node Exporter

1. **Tóm tắt công cụ:** **Prometheus Node Exporter** là công cụ xuất dữ liệu hệ điều hành và phần cứng dành cho hệ điều hành *NIX, được viết bằng Go với khả năng mở rộng collector. Mục đích là cung cấp nhiều loại chỉ số của các node.

2. **Vị trí:** Triển khai dưới dạng DaemonSet, đảm bảo mỗi node đều có một instance chạy. Định nghĩa trong `node-exporter/node-exporter-daemonset.yml`.

3. **Vai trò:** Thu thập các chỉ số hệ điều hành và phần cứng chi tiết từ từng node.

4. **Trách nhiệm trong hệ thống:**

- Thu thập dữ liệu như mức sử dụng CPU, thống kê bộ nhớ (tổng, khả dụng, cache), I/O đĩa, và lưu lượng mạng.
- Cung cấp endpoint HTTP (thường là `/metrics` trên cổng 9100) để Prometheus có thể thu thập dữ liệu.

3.1.2.4. Kubernetes API Server

1. **Mô tả:** Thành phần cơ bản của control plane Kubernetes. Cung cấp API giao tiếp cho tất cả hoạt động trong cụm.

2. **Vai trò:** Là trung tâm quản lý cụm, lưu trữ trạng thái mong muốn và trạng thái hiện tại của tất cả đối tượng.

3. **Trách nhiệm (từ góc nhìn scheduler):**

- Lưu trữ và quản lý trạng thái của Pod, Node, Deployment, Service, và Event.
- Cung cấp cơ chế “watch” để scheduler theo dõi các pod cần được phân lịch.
- Cung cấp các API để liệt kê node, kiểm tra thông số kỹ thuật của node và liệt kê pod đang chạy.
- Nhận các `Binding` từ scheduler để chính thức gán pod vào node cụ thể.
- Lưu trữ và cung cấp các `Event` phục vụ việc quan sát quá trình phân lịch.

3.1.3. Luồng tương tác tổng thể khi phân lịch một Pod

Quá trình phân lịch một pod duy nhất sử dụng scheduler tùy chỉnh tuân theo chuỗi thao tác sau:

1. **Tạo và nhận diện Pod:** Người dùng hoặc controller gửi manifest pod đến API Server với `spec.schedulerName: custom-scheduler`. Pod ban đầu sẽ ở trạng thái `Pending`.
2. **Phát hiện bởi Scheduler:** Scheduler phát hiện pod mới chưa được phân lịch.
3. **Giai đoạn Predicate (Lọc):** Scheduler truy xuất danh sách node và áp dụng các kiểm tra để lọc các node tương thích:
 - Node đáp ứng yêu cầu CPU và bộ nhớ.
 - Node không có taint mà pod không thể tolerate.
 - Node khớp với `spec.nodeSelector` của pod.
4. **Thu thập chỉ số (Ưu tiên):** Với các node tương thích, scheduler truy vấn Prometheus để lấy dữ liệu mới nhất như `node_memory_MemAvailable_bytes` và CPU nhàn rỗi.
5. **Tính điểm Node:** Scheduler tính điểm tổng hợp cho mỗi node:
 - **Điểm Bộ Nhớ:** Dựa trên bộ nhớ khả dụng, chuẩn hóa.
 - **Điểm CPU:** Dựa trên mức nhàn rỗi của CPU, chuẩn hóa.

- **Điểm Affinity:** Nếu trong manifest của pod định nghĩa trường `preferredDuringSchedulingIgnoredDuringExecution`, các node phù hợp sẽ được cộng điểm tùy theo `weight`.
 - **Tổng điểm:** Kết hợp các điểm bằng trọng số (`SCORE_WEIGHT_MEM`, `SCORE_WEIGHT_CPU`, `SCORE_WEIGHT_AFFINITY`) qua biến môi trường.
6. **Chọn Node:** Node có điểm cao nhất sẽ được chọn. Nếu có điểm bằng nhau, ưu tiên node có bộ nhớ hoặc CPU cao hơn.
 7. **Thao tác ràng buộc (Binding Operation):** Bộ lập lịch tùy chỉnh tạo một đối tượng `Binding` trong Kubernetes. Đối tượng này liên kết pod (dựa trên tên và namespace của nó) với node được chọn. Sau đó, `Binding` này được gửi tới **Kubernetes API Server** để chính thức gán pod vào node tương ứng.
 8. **Thực thi Pod:** Sau khi Kubernetes API Server ghi nhận `Binding`, thành phần **Kubelet** đang chạy trên node mục tiêu sẽ phát hiện pod được giao và bắt đầu quá trình kéo image, khởi tạo container, và chạy pod theo cấu hình đã định.

3.2. Chi tiết quy trình lập lịch

Chi tiết quy trình lập lịch của bộ lập lịch Kubernetes tùy chỉnh của nhóm em là một chuỗi các hoạt động được phối hợp nhịp nhàng nhằm xác định node phù hợp nhất cho một pod đang chờ xử lý. Quá trình này được khởi xướng khi một pod được chỉ định cho bộ lập lịch này (`spec.schedulerName: custom-scheduler`) được phát hiện thông qua việc giám sát liên tục API Kubernetes. Quy trình này mạnh mẽ, tích hợp dữ liệu thời gian thực và các ưu tiên đã xác định, và có thể được chia thành nhiều giai đoạn riêng biệt: cơ chế phát hiện và theo dõi pod, giai đoạn vị từ (predicate) để lọc các node tương thích, giai đoạn tính điểm để ưu tiên các node này, giai đoạn ràng buộc (binding) để thực thi quyết định, và cuối cùng là báo cáo sự kiện toàn diện.

3.2.1. Cơ chế phát hiện và theo dõi Pod

Bước nền tảng trong chu trình lập lịch là việc xác định kịp thời và chính xác các pod thuộc phạm vi quản lý của bộ lập lịch tùy chỉnh của nhóm em. Điều này không đạt được thông qua việc thăm dò định kỳ, mà thay vào đó là một cơ chế theo dõi dựa trên sự kiện (event-driven watch mechanism) hiệu quả.

Trách nhiệm: Giai đoạn này chủ yếu được điều phối bởi hàm `watchUnscheduledPods` nằm trong tệp `scheduler/kubernetes.go`. Các pod được xác định bởi hàm này sau đó được tiêu thụ và xử lý bởi vòng lặp lập lịch chính trong `scheduler/main.go`.

Phân tích cơ chế:

- 1. Khởi tạo Kubernetes Client:** Khi khởi động, ứng dụng bộ lập lịch thiết lập một kết nối đến máy chủ API Kubernetes. Điều này được thực hiện bằng thư viện chính thức `client-go`. Hàm `initKubernetesClient` (trong `scheduler/kubernetes.go`) cố gắng sử dụng `rest.InClusterConfig()`, đây là phương thức tiêu chuẩn cho các ứng dụng chạy dưới dạng pod trong cụm Kubernetes để xác thực và kết nối với máy chủ API một cách an toàn.
- 2. Xác định Pod mục tiêu:** Bộ lập lịch tùy chỉnh của nhóm em được lập trình đặc biệt để tìm kiếm các pod thỏa mãn một tập hợp các tiêu chí chính xác, đảm bảo nó chỉ xử lý các pod dành cho logic của mình:
 - `spec.schedulerName` **phải** bằng chuỗi `"custom-scheduler"`. Giá trị này được định nghĩa là một hằng số `SchedulerName` trong `scheduler/kubernetes.go` và được tham chiếu trong toàn bộ mã nguồn và các tệp manifest Kubernetes (như `scheduler/deployments/testcustom.yaml`).
 - `spec.nodeName` **phải** là một chuỗi rỗng. Một trường `nodeName` rỗng biểu thị rằng hệ thống Kubernetes chưa gán pod cho bất kỳ node cụ thể nào.
 - `status.phase` **phải** là `v1.PodPending`. Trạng thái này cho biết pod đã được hệ thống Kubernetes chấp nhận nhưng đang chờ được lập lịch và thực thi sau đó.
- 3. Thiết lập theo dõi API (API Watch):** Hàm `watchUnscheduledPods` tận dụng `client-go` để thiết lập một “watch” trên các đối tượng Pod trên tất cả các không gian tên (namespaces) (`clientset.CoreV1().Pods("")`). Để tối ưu hóa điều này và giảm tải cho cả bộ lập lịch và máy chủ API, yêu cầu watch bao gồm một `fieldSelector`. Bộ chọn này, `fields.OneTermEqualSelector("spec.nodeName", "").String()`, hướng dẫn máy chủ API chỉ gửi các sự kiện cho các pod hiện chưa có node nào được gán.

```
// Đoạn mã liên quan từ scheduler/kubernetes.go minh họa thiết lập Watch
```



```
watcher, err := clientset.CoreV1().Pods("").Watch(ctx,
metav1.ListOptions{
    FieldSelector: fields.OneTermEqualSelector("spec.nodeName",
    "").String(),
})
```

Việc lọc phía máy chủ này rất quan trọng đối với hiệu quả trong các cụm lớn.

4. **Logic xử lý dựa trên sự kiện:** Bộ lập lịch không thăm dò máy chủ API. Thay vào đó, nó phản ứng với các sự kiện. Khi một Pod được tạo, hoặc một Pod hiện có được sửa đổi sao cho nó khớp với tiêu chí theo dõi, máy chủ API sẽ truyền một sự kiện (ví dụ: `watch.Added` cho các pod mới, `watch.Modified` nếu một pod hiện có trở nên chưa được lập lịch và khớp với tên bộ lập lịch) đến watcher được kết nối. Mã của nhóm em lặp qua các sự kiện này trong một vòng lặp:

- Nó ép kiểu đối tượng sự kiện thành `*v1.Pod`.
- Sau đó, nó xác minh lại rằng `pod.Spec.SchedulerName == SchedulerName`, `pod.Spec.NodeName == ""`, và `pod.Status.Phase == v1.PodPending`. Việc kiểm tra phía máy khách này là một biện pháp bảo vệ, mặc dù bộ lọc phía máy chủ phần lớn sẽ xử lý tiêu chí `nodeName`.

5. **Giao tiếp giữa các Goroutine qua Kênh (Channels):** Nếu một sự kiện liên quan đến một pod hoàn toàn khớp với tất cả các tiêu chí, một bản sao của đối tượng `v1.Pod` được tạo (để tránh tình trạng tranh chấp nếu đối tượng gốc trong bộ đệm watch được cập nhật) và được gửi đến một kênh Go có tên `podChannel`. Vòng lặp xử lý chính trong `scheduler/main.go` có một câu lệnh `select` liên tục lắng nghe các đối tượng `v1.Pod` mới đến trên `podChannel` này.

```
// Đoạn mã từ vòng lặp xử lý sự kiện trong scheduler/
kubernetes.go
```



```
if pod.Spec.SchedulerName == SchedulerName && pod.Spec.NodeName ==
"" && pod.Status.Phase == v1.PodPending {
    log.Printf("Found unscheduled pod for %s: %s/%s (Event: %s)", /
    * ... */)
    select {
```

```

case podChannel <- *pod: // Gửi một bản sao
case <-ctx.Done(): // Xử lý hủy context trong khi gửi
    // ...
    return
}
}

```

Tương tự, các lỗi không nghiêm trọng gặp phải trong quá trình watch (ví dụ: sự cố mạng tạm thời không dẫn đến việc đóng watch) được gửi đến một kênh `errChannel` có bộ đệm kích thước 1. Vòng lặp chính cũng chọn trên kênh này để ghi lại các cảnh báo như vậy. Bộ đệm ngăn goroutine watch bị chặn nếu vòng lặp chính tạm thời bận.

6. Khả năng phục hồi và quản lý lỗi trong cơ chế Watch: Cơ chế watch được thiết kế để có khả năng phục hồi:

- Toàn bộ hoạt động watch được bao bọc trong một vòng lặp bên ngoài. Nếu `watcher.ResultChan()` đóng lại (điều này có thể xảy ra nếu kết nối với máy chủ API bị gián đoạn hoặc nếu phiên bản tài nguyên trở nên quá cũ, được biểu thị bằng lỗi `metav1.StatusReasonGone`), mã sẽ ghi lại sự kiện và, sau một khoảng trễ ngắn (ví dụ: 1-5 giây), cố gắng thiết lập lại một watch mới. Điều này đảm bảo rằng bộ lập lịch có thể phục hồi từ các sự cố mạng tạm thời hoặc khởi động lại máy chủ API.
- Các lỗi watch cụ thể, như `http.StatusGone` (HTTP 410), kích hoạt ngay lập tức việc khởi động lại watch, vì chúng cho biết phiên bản tài nguyên của watch hiện tại không còn hợp lệ.

7. Ngắt hệ thống dễ dàng qua Context: Hàm `watchUnscheduledPods`, và thực sự hầu hết các hoạt động không đồng bộ trong bộ lập lịch của nhóm em, chấp nhận một `context.Context` (tên là `ctx`). Context này được tạo trong `scheduler/main.go` và bị hủy khi ứng dụng nhận được tín hiệu tắt (SIGINT hoặc SIGTERM). Các Goroutine thường xuyên kiểm tra `ctx.Done()`. Nếu context bị hủy, chúng sẽ dọn dẹp (ví dụ: dừng watcher, đóng các kênh) và kết thúc một cách mềm mại, ngăn chặn rò rỉ tài nguyên hoặc thoát đột ngột.

Cơ chế watch dựa trên sự kiện, có khả năng phục hồi này đảm bảo rằng bộ lập lịch tùy chỉnh của nhóm em nhận biết một cách hiệu quả và kịp thời các Pod và yêu cầu của chúng.

3.2.2. Giai đoạn lọc (Predicate Phase): Lọc các Node Tương thích

Sau khi một pod chưa được lập lịch được nhận từ cơ chế watch, bộ lập lịch trước tiên phải xác định một tập hợp con các node trong cụm nơi pod **có thể** chạy một cách khả thi. Đây là giai đoạn vị từ, áp dụng một loạt các ràng buộc cứng, không thể thương lượng. Chỉ những node thỏa mãn tất cả các kiểm tra vị từ mới được chuyển sang giai đoạn tính điểm tiếp theo.

Trách nhiệm: Logic lọc quan trọng này chủ yếu được đóng gói trong hàm `predicateChecks` trong `scheduler/kubernetes.go`. Hàm này được hỗ trợ bởi một số hàm trợ giúp như `calculateNodeUsage`, `calculatePodResourceRequests`, `checkNodeResources`, và `checkNodeTaints`.

Phân tích cơ chế: Hàm `predicateChecks` đánh giá một cách có hệ thống từng node trong cụm (hoặc một danh sách đã được lọc trước dựa trên `pod.Spec.NodeName` hoặc `pod.Spec.NodeSelector`) dựa trên các yêu cầu của pod cần lập lịch.

1. Tạo danh sách Node ứng cử viên ban đầu:

- **Yêu cầu Node rõ ràng (`pod.Spec.NodeName`):** Nếu đặc tả của pod bao gồm một `nodeName` không rỗng, giai đoạn vị từ chỉ xem xét node được chỉ định này. Nếu node này không vượt qua bất kỳ kiểm tra nào sau đó, pod không thể được lập lịch bởi bộ lập lịch này.
- **Bộ chọn Node (`pod.Spec.NodeSelector`):** Nếu `pod.Spec.NodeSelector` (một map các cặp khóa-giá trị) được định nghĩa, bộ lập lịch chỉ liệt kê những node có nhãn khớp với tất cả các mục trong `NodeSelector`. Điều này đạt được bằng cách chuyển đổi `MatchLabels` thành một `labels.Selector` và sử dụng nó trong `ListOptions` của `client-go`.
- **Trường hợp mặc định:** Nếu cả `nodeName` và `nodeSelector` đều không được chỉ định, tất cả các node trong cụm ban đầu được coi là ứng cử viên.

2. Trạng thái Node không thể lập lịch (`node.Spec.Unschedulable`):

- Trường `Unschedulable` của mỗi node ứng cử viên được kiểm tra. Nếu trường boolean này là `true`, nó biểu thị rằng một quản trị viên đã cách ly (cordon) node, với ý định không cho nó nhận các pod mới. Các node

như vậy thường bị loại trừ bởi các vị từ, và bộ lập lịch của nhóm em tuân thủ quy ước này.

3. Kiểm tra tính sẵn có của Tài nguyên (CPU và Bộ nhớ): Đây là một quy trình nhiều bước để đảm bảo node có đủ tài nguyên.

- **Tính toán nhu cầu tài nguyên của Pod:** Hàm trợ giúp `calculatePodResourceRequests` lặp qua tất cả các container được định nghĩa trong `pod.Spec.Containers`. Đối với mỗi container, nó cộng các giá trị được chỉ định trong `container.Resources.Requests[v1.ResourceCPU]` và `container.Resources.Requests[v1.ResourceMemory]`. Chúng đại diện cho lượng CPU (tính bằng cores/millicores) và bộ nhớ (tính bằng byte) mà pod đảm bảo cần. Kết quả là các đối tượng `resource.Quantity`, hỗ trợ các phép tính số học cho các giá trị tài nguyên (ví dụ: “500m” cho CPU, “1Gi” cho bộ nhớ).
- **Tính toán mức sử dụng hiện tại của Node:** Hàm trợ giúp `calculateNodeUsage` xác định các tài nguyên đã được cam kết trên một node ứng cử viên. Nó liệt kê tất cả các pod hiện đang được gán cho node đó (tức là `existingPod.Spec.NodeName == node.Name`) mà không ở trạng thái cuối (Failed hoặc Succeeded). Đối với mỗi pod hiện có như vậy, nó cộng dồn các `requests` CPU và bộ nhớ của các container của chúng. Tổng này đại diện cho tổng tài nguyên được yêu cầu bởi các pod đã có trên node đó.
- **So sánh nhu cầu của Pod với tài nguyên khả dụng của Node:** Hàm trợ giúp `checkNodeResources` thực hiện so sánh cuối cùng. Nó sử dụng `node.Status.Allocatable`, đại diện cho lượng tài nguyên trên một node có sẵn cho các pod (sau khi tính đến các daemon hệ thống và tài nguyên dành riêng cho kubelet).
 - `availableCPUOnNode = node.Status.Allocatable.Cpu() - currentTotalCPURequestsOnNode`
 - `availableMemoryOnNode = node.Status.Allocatable.Memory() - currentTotalMemoryRequestsOnNode`
 - Pod chỉ có thể được lập lịch nếu thỏa mãn đồng thời 2 tiêu chí `newPodCPURequest <= availableCPUOnNode` (có đủ CPU) và `newPodMemoryRequest <= availableMemoryOnNode` (có đủ bộ nhớ).

- Các phép so sánh và trừ được thực hiện bằng các phương thức được cung cấp bởi kiểu `resource.Quantity`, đảm bảo xử lý chính xác các đơn vị tài nguyên.

4. Khả năng tương thích giữa Taint và Toleration: Kiểm tra này đảm bảo rằng pod có thể chịu đựng bất kỳ hạn chế nào (taint) được đặt trên node.

- Hàm trợ giúp `checkNodeTaints` lặp qua từng `v1.Taint` trong `node.Spec.Taints`.
- Đối với mỗi taint, nó kiểm tra `Effect` của nó:
 - Nếu `effect` là `v1.TaintEffectNoSchedule` hoặc `v1.TaintEffectNoExecute`, bộ lập lịch phải tìm một `v1.Toleration` tương ứng trong `pod.Spec.Tolerations`.
 - Hàm trợ giúp `tolerationsTolerateTaint` (bên trong sử dụng `toleration.ToleratesTaint()`) kiểm tra xem bất kỳ toleration nào của pod có “khớp” với taint không. Một sự khớp thường có nghĩa là:
 - `Key` khớp (hoặc toleration không có key, khớp với tất cả các key).
 - `Operator` là `v1.TolerationOpExists` (khớp với bất kỳ giá trị nào cho key), hoặc `Operator` là `v1.TolerationOpEqual` và `Value` cũng khớp.
 - `Effect` của toleration khớp với effect của taint (hoặc toleration không có effect nào được chỉ định, khớp với tất cả các effect).
 - Nếu một taint `NoSchedule` hoặc `NoExecute` được tìm thấy trên node mà pod không có toleration tương ứng, node đó không vượt qua kiểm tra vị từ.
- **Xử lý trì hoãn các Taint `PreferNoSchedule`**: Quan trọng là, nếu một taint có effect `v1.TaintEffectPreferNoSchedule`, giai đoạn vị từ của bộ lập lịch của nhóm em **không** lọc bỏ node đó. Loại taint này cho biết một sự ưu tiên, không phải là một ràng buộc cứng. Các node như vậy được chuyển đến giai đoạn tính điểm, nơi chúng có thể nhận điểm thấp hơn do sự ưu tiên này, nhưng vẫn có thể được chọn nếu không có node tương thích, không bị taint nào tốt hơn.

5. Yêu cầu Node Affinity/Anti-Affinity:

- Mặc dù trọng tâm chính của bộ lập lịch của nhóm em đối với affinity là `preferredDuringSchedulingIgnoredDuringExecution` (được xử lý trong giai đoạn tính điểm), một giai đoạn vị từ đầy đủ cũng sẽ đánh giá `requiredDuringSchedulingIgnoredDuringExecution` từ `pod.Spec.Affinity.NodeAffinity`. Điều này sẽ hoạt động giống như một dạng biểu cảm hơn của `nodeSelector`, nơi các node phải khớp với các biểu thức nhân phức tạp để vượt qua. Việc triển khai hiện tại của nhóm em ngầm bao gồm việc khớp nhân đơn giản thông qua `pod.Spec.NodeSelector`.

Kết quả Giai đoạn Lọc và Báo cáo Sự kiện:

- Nếu một node vượt qua thành công tất cả các kiểm tra vị từ ở trên, nó được thêm vào danh sách “các node tương thích”.
- Nếu, sau khi đánh giá tất cả các node ứng cử viên, danh sách các node tương thích này trống, điều đó có nghĩa là pod không thể được lập lịch trên bất kỳ node nào trong cụm dựa trên các yêu cầu cứng của nó. Trong trường hợp này, `predicateChecks` trả về một lỗi.
- Hàm gọi (thường là `schedulePod` trong `scheduler/processor.go`) sau đó sẽ sử dụng `postEvent` (từ `scheduler/kubernetes.go`) để tạo một `Event` Kubernetes liên quan đến pod. Sự kiện này sẽ có:
 - `Reason`: “FailedScheduling”
 - `Type`: “Warning”
 - `Message`: Một thông báo mô tả, ví dụ: “pod (namespace/pod-name) không vượt qua kiểm tra vị từ trên tất cả các node. Không đủ CPU trên node-X; Pod không chịu được taint Y trên node-Z.” Điều này cung cấp cho quản trị viên lý do rõ ràng cho việc lập lịch thất bại.

Chỉ những node xuất hiện từ giai đoạn vị từ nghiêm ngặt này mới được coi là ứng cử viên khả thi cho giai đoạn tính điểm tinh tế hơn tiếp theo.

3.2.3. Giai đoạn tính điểm (Scoring Phase): Ưu tiên các Node

Sau khi giai đoạn vị từ đã tạo ra một danh sách các node tương thích (tức là các node nơi pod **có thể** chạy), giai đoạn tính điểm, còn được gọi là giai đoạn ưu tiên hóa, sẽ xếp hạng các node này từ phù hợp nhất đến ít phù hợp nhất. Đây là nơi trí tuệ cốt lõi của bộ lập lịch tùy chỉnh của nhóm em phát huy tác dụng,

vì nó kết hợp các số liệu thời gian thực và các ưu tiên đã xác định để đưa ra lựa chọn tối ưu.

Trách nhiệm: Logic tính điểm cốt lõi được triển khai trong hàm `ScoreNodes` và các hàm trợ giúp liên quan của nó trong `scheduler/scoring.go`. Hàm `getBestNode` trong `scheduler/getBestNode.go` đóng vai trò trung gian, gọi `ScoreNodes` với các node tương thích và chi tiết pod. Hoạt động của giai đoạn này bị ảnh hưởng đáng kể bởi struct `ScoringConfig` (chứa `WeightMem`, `WeightCPU`, `WeightAffinity`), được điền vào trong `scheduler/main.go` bằng cách đọc các biến môi trường (ví dụ: `SCORE_WEIGHT_MEM`) được định nghĩa trong tệp YAML triển khai của bộ lập lịch (`scheduler/custom-scheduler-deployment.yaml`).

Phân tích cơ chế:

1. **Thu thập số liệu Node trong thời gian thực từ Prometheus:** Đây là bước quan trọng đầu tiên trong việc tính điểm, vì nó cung cấp dữ liệu đồng thời gian thực.

- **Hàm:** `WorkspaceNodeMetrics` (trong `scheduler/scoring.go`) chịu trách nhiệm. Nó chấp nhận một map các tên node tương thích để đảm bảo số liệu chỉ được lấy và xử lý cho các node liên quan.
- **Điểm cuối Prometheus (Prometheus Endpoint):** Nó nhắm mục tiêu dịch vụ Prometheus, có địa chỉ được xác định bởi hằng số `prometheusService` (ví dụ: `http://prometheus-service.monitoring.svc.cluster.local:8080`, được cấu hình để trỏ đến `prometheus-service` trong không gian tên `monitoring`).
- **Các truy vấn PromQL Cụ thể:** Bộ lập lịch của nhóm em sử dụng các truy vấn PromQL chính xác để lấy dữ liệu tài nguyên có ý nghĩa:
 - **Bộ nhớ khả dụng:** Truy vấn: `node_memory_MemAvailable_bytes` (từ hằng số `memPromQL`). Số liệu này được ưu tiên hơn `node_memory_MemFree_bytes` vì `MemAvailable` cung cấp ước tính bộ nhớ có sẵn để khởi chạy ứng dụng mới, không cần swap, và bao gồm cả slab memory có thể thu hồi. Đây là một chỉ số chính xác hơn về tính sẵn có thực tế từ góc độ của kernel.

- ▶ **Trung bình số core CPU “rảnh”:** Truy vấn được sử dụng:

```
avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[1m]))
```

(từ hằng số `cpuIdlePromQL`). Truy vấn này tính toán:

- `node_cpu_seconds_total{mode="idle"}`: Một bộ đếm tổng thời gian (tính bằng giây) CPU đã ở trạng thái rảnh, trên mỗi lõi CPU.
 - `rate(...[1m])`: Tính toán tốc độ tăng trung bình mỗi giây của bộ đếm này trong 1 phút qua. Điều này thực chất cho biết tỷ lệ thời gian mỗi lõi CPU rảnh trong phút đó (ví dụ: 0.5 có nghĩa là 50% rảnh).
 - `avg by (instance) (...)`: Tính trung bình các tỷ lệ rảnh trên mỗi lõi này trên tất cả các lõi của một node nhất định (được xác định bằng nhãn `instance`), cho ra một giá trị đại diện cho số lượng lõi “tương đương rảnh” trung bình trên node đó. Ví dụ, trên một node 8 lõi, giá trị 4.0 từ truy vấn này có nghĩa là, trung bình, có 4 lõi CPU rảnh trong phút vừa qua.
- **Thực thi truy vấn:** Hàm trợ giúp `queryPrometheus` xử lý yêu cầu HTTP GET thực tế đến điểm cuối `/api/v1/query` của Prometheus. Nó đảm bảo mã hóa URL đúng cách cho chuỗi truy vấn PromQL. Nó mong đợi một phản hồi JSON và giải mã nó thành các struct Go `MetricResponse`, `Data`, và `Result`, được định nghĩa trong `scheduler/scoring.go` để khớp với cấu trúc phản hồi API của Prometheus.
 - **Ánh xạ số liệu Prometheus tới Node của Kubernetes:** Số liệu Prometheus thường sử dụng nhãn `instance` (ví dụ: `node-exporter-ip:port`). Hệ thống của nhóm em cần ánh xạ điều này tới tên node Kubernetes. Hàm `getNodeNameFromMetric`, cùng với `parseInstanceLabel`, xử lý việc này. Đối với các số liệu bắt nguồn từ `node-exporter` (chạy dưới dạng DaemonSet), cấu hình scrape của Prometheus trong `prometheus/config-map.yaml` (cụ thể là job `kubernetes-pods` khám phá `node-exporter`) sử dụng các quy tắc gán lại nhãn (relabeling). Các quy tắc này lấy `__meta_kubernetes_pod_node_name` (tên node Kubernetes nơi pod

`node-exporter` đang chạy) và đặt nó làm nhãn `instance` trên các số liệu được scrape. Do đó, nhãn `instance` trực tiếp cung cấp tên node Kubernetes. `parseInstanceLabel` cũng xử lý việc loại bỏ hậu tố cổng nếu có.

- **Phân tích giá trị số liệu:** Prometheus trả về các giá trị số liệu dưới dạng một cặp: `[<timestamp>, "<value_string>"]`. Hàm trợ giúp `parseMetricValue` trích xuất `"<value_string>"` và chuyển đổi nó thành `float64` để tính toán.
- **Khả năng phục hồi trong việc lấy số liệu:** `WorkspaceNodeMetrics` được thiết kế để có phần nào khả năng phục hồi. Nếu việc lấy số liệu cho một loại (ví dụ: bộ nhớ) thất bại, nhưng số liệu CPU thành công, nó sẽ ghi lại một cảnh báo và tiếp tục. Điểm số cho loại số liệu bị lỗi sẽ thực chất bằng không cho tất cả các node. Tuy nhiên, nếu **tất cả** việc lấy số liệu đều thất bại (cả bộ nhớ và CPU đều trả về lỗi hoặc không có dữ liệu cho các node tương thích), `WorkspaceNodeMetrics` sẽ trả về một lỗi, sau đó `ScoreNodes` sẽ truyền đi, thường dẫn đến thất bại trong việc lập lịch hoặc phương án dự phòng. Hệ thống hiện tại không có xử lý tình vi cho độ trễ của Prometheus ngoài các timeout HTTP tiêu chuẩn, đây là một lĩnh vực tiềm năng để cải tiến trong tương lai.

2. Tính toán các thành phần điểm riêng lẻ cho mỗi node tương thích:

Sau khi lấy số liệu, mỗi node tương thích được tính điểm trên nhiều khía cạnh:

- **Tính toán điểm bộ nhớ:**
 - ▶ Giá trị thô `node_memory_MemAvailable_bytes` cho node được lấy.
 - ▶ Giá trị thô này sau đó được chuyển cho hàm chuẩn hóa `normalizeScore(value, maxValue)`. Giá trị `maxValue` ở đây là giá trị `MemAvailable` cao nhất được quan sát trên **tất cả** các node tương thích mà số liệu bộ nhớ đã được truy xuất thành công.
 - ▶ `normalizeScore` tính toán $(value / maxValue) * 10$. Điều này điều chỉnh tính sẵn có của bộ nhớ theo thang điểm 0-10. Một node có nhiều bộ nhớ khả dụng nhất sẽ nhận được điểm 10 (hoặc gần bằng), trong khi một node có rất ít (hoặc thiếu dữ liệu, dẫn đến `value = 0`) sẽ nhận điểm 0. Chuẩn hóa là rất quan trọng vì nó đưa các số liệu khác nhau (byte bộ nhớ, số lỗi CPU) về một thang đo chung, có thể

so sánh được trước khi áp dụng trọng số. Thang điểm 0-10 là một quy ước phổ biến trong việc tính điểm của bộ lập lịch Kubernetes.

- **Tính toán điểm CPU:**

- Giá trị số lõi CPU rảnh trung bình thô cho node được sử dụng.
- Giá trị này cũng được chuẩn hóa tương tự bằng cách sử dụng `normalizeScore(value, maxValue)`, trong đó `maxValue` là giá trị số lõi CPU rảnh trung bình cao nhất được quan sát trong số các node tương thích. Điều này cũng cho kết quả là một điểm số 0-10.

- **Tính toán điểm Node Affinity:**

- Hàm `calculateAffinityScore(node, pod)` được gọi. Hàm này kiểm tra trường `preferredDuringSchedulingIgnoredDuringExecution` trong `pod.Spec.Affinity.NodeAffinity`. Trường này chứa một danh sách các cấu trúc giống như `WeightedPodAffinityTerm`, mỗi cấu trúc có một `weight` (số nguyên từ 1-100) và một `preference` (một `NodeSelectorTerm`).
- Đối với mỗi `preferredTerm` trong danh sách:
 - Nếu `preference.MatchExpressions` được định nghĩa, các biểu thức này (ví dụ: `key: zone, operator: In, values: [a, b]`) được chuyển đổi thành một đối tượng `labels.Selector` bằng cách sử dụng hàm trợ giúp `NodeSelectorRequirementsAsSelector` (tận dụng package sẵn có của Kubernetes `k8s.io/apimachinery/pkg/labels` và `k8s.io/apimachinery/pkg/selection`).
 - `labels.Selector` này sau đó được đánh giá dựa trên `node.Labels`.
 - Nếu nhãn của node thỏa mãn bộ chọn cho thuật ngữ ưu tiên đó, `weight` của thuật ngữ đó được thêm vào biến `totalPreferenceScore` đang chạy cho node đó.
- Sau khi đánh giá tất cả các thuật ngữ ưu tiên, `totalPreferenceScore` (có thể dao động từ 0 đến 100 lần số lượng thuật ngữ) được điều chỉnh theo thang điểm 0-10. Logic hiện tại của nhóm em điều chỉnh

điều này bằng cách giả định điểm thô tối đa mong muốn là 100 tương ứng với 10:

```
scaledScore = (float64(totalPreferenceScore) / 100.0)
* float64(maxScoreConstant)
```



`maxScoreConstant` này là 10. Kết quả được giới hạn trong khoảng 0 và 10. Việc điều chỉnh này có nghĩa là `SCORE_WEIGHT_AFFINITY` hoạt động như một hệ số nhân trên thành phần affinity cơ sở 0-10, làm cho tác động của nó trở nên trực quan hơn.

3. Kết hợp các điểm riêng lẻ thành tổng điểm theo trọng số tương ứng:

- Struct `ScoringConfig`, lưu các trọng số của từng loại điểm, được trích xuất từ các biến môi trường (ví dụ: `SCORE_WEIGHT_MEM: "0.6"`, `SCORE_WEIGHT_CPU: "0.4"` và `SCORE_WEIGHT_AFFINITY: "1.0"` trong tệp manifest `scheduler/custom-scheduler-deployment.yaml`).
- Đối với mỗi node, `TotalScore` được tính bằng tổng có trọng số:

```
nodeScore.TotalScore =
    (config.WeightMem * nodeScore.MemScore)
+ (config.WeightCPU * nodeScore.CPUScore)
+ (config.WeightAffinity * nodeScore.AffinityScore);
```



- Kết quả, cùng với các điểm thành phần và tên node, được lưu trữ trong một struct `NodeScore` (được định nghĩa trong `scheduler/scoring.go`).

4. Chọn node tốt nhất dựa trên tổng điểm:

- Gồm một danh sách các đối tượng `NodeScore`, điểm tương ứng cho mỗi node sau khi tính toán thành công.
- Danh sách này sau đó được sắp xếp. Khóa sắp xếp chính là `TotalScore` theo thứ tự giảm dần (điểm cao hơn thì tốt hơn).
- **Tie-breaking:** Nếu hai hoặc nhiều node có cùng `TotalScore` chính xác, một cơ chế tie-breaking được áp dụng để đảm bảo lựa chọn là xác định (deterministic):
 1. Node có `MemScore` cao hơn được ưu tiên. (Lý do: Bộ nhớ thường là tài nguyên quan trọng hơn, ít bị nén hơn CPU trong nhiều khối lượng công việc).

2. Nếu `MemScore` cũng hòa, node có `CPU Score` cao hơn được ưu tiên.
- Node đứng đầu danh sách đã sắp xếp này được chỉ định là “node tốt nhất” cho Pod.

Kết quả Tính điểm và Phương án Dự phòng:

- Hàm `ScoreNodes` trả về `Name` của node tốt nhất này.
- Nếu, vì một lý do nào đó, `ScoreNodes` thất bại (ví dụ: tất cả các lần lấy số liệu đều thất bại một cách thảm khốc) nhưng giai đoạn vị từ **đã** trả về các node tương thích, hàm `schedulePod` (trong `scheduler/processor.go`) sẽ triển khai một phương án dự phòng đơn giản: nó ghi lại lỗi tính điểm và cố gắng lập lịch pod trên node **đầu tiên** từ danh sách tương thích. Đây là một lưới an toàn để ngăn một pod không được lập lịch vô thời hạn nếu việc tính điểm gặp sự cố không mong muốn, mặc dù mục tiêu luôn là để việc tính điểm thành công.

Giai đoạn tính điểm chi tiết này cho phép bộ lập lịch của nhóm em phân biệt chi tiết giữa các node khả thi, vượt ra ngoài việc đáp ứng yêu cầu tài nguyên đơn giản để xem xét tính sẵn có thực tế và các ưu tiên do khối lượng công việc xác định.

3.2.4. Giai đoạn ràng buộc (Binding Phase): Gán Pod cho Node

Sau khi giai đoạn tính điểm đã xác định một cách dứt khoát node tốt nhất duy nhất cho pod, hành động cuối cùng là cam kết quyết định này bằng cách gán chính thức pod cho node đó trong cụm Kubernetes. Điều này được thực hiện thông qua quy trình “ràng buộc”.

Trách nhiệm: Bước quan trọng này được thực hiện bởi hàm `bindPod`, nằm trong `scheduler/kubernetes.go`.

Phân tích cơ chế:

1. **Khái niệm về đối tượng Binding:** Trong Kubernetes, các bộ lập lịch thường không trực tiếp sửa đổi trường `spec.nodeName` của đối tượng Pod mà chúng đang lập lịch. Thay vào đó, cơ chế ưu tiên là tạo một đối tượng API riêng biệt có `Kind: Binding`. Đối tượng `Binding` này hoạt động như một chỉ thị cho máy chủ API Kubernetes, hướng dẫn nó gán một pod cụ thể cho một node cụ thể. Cách tiếp cận này duy trì sự tách biệt rõ ràng về trách nhiệm: bộ lập lịch đề xuất một ràng buộc, và máy chủ API thực thi nó.

2. **Xây dựng đối tượng `v1.Binding`**: Hàm `bindPod` của nhóm em xây dựng một đối tượng `v1.Binding` một cách tỉ mỉ.

- `ObjectMeta.Name`: Được đặt thành `name` của `podToSchedule`.
- `ObjectMeta.Namespace`: Được đặt thành `namespace` của `podToSchedule`. Ràng buộc phải được tạo trong cùng không gian tên với pod.
- `ObjectMeta.Annotations`: Một chú thích quan trọng cần để ý là `kubernetes.io/custom-scheduler: SchedulerName` (trong đó `SchedulerName` là “`custom-scheduler`”), được thêm vào. Điều này đóng vai trò như một bản ghi, có thể xem được qua `kubectl describe binding <pod-name>`, cho biết bộ lập lịch nào đã đưa ra quyết định ràng buộc cụ thể này.
- `Target`: Đây là phần cốt lõi của ràng buộc. Nó là một `v1.ObjectReference` xác định duy nhất node mà pod sẽ được ràng buộc.
 - `Target.APIVersion`: Đặt thành “`v1`”.
 - `Target.Kind`: Đặt thành “`Node`”.
 - `Target.Name`: Đặt thành `nodeName`, là tên của node tốt nhất được xác định bởi giai đoạn tính điểm.

```
// Minh họa cấu trúc từ scheduler/kubernetes.go
binding := &v1.Binding{
    ObjectMeta: metav1.ObjectMeta{
        Name:      podToSchedule.Name,
        Namespace: podToSchedule.Namespace,
        Annotations: map[string]string{
            "kubernetes.io/custom-scheduler": SchedulerName, //
            "custom-scheduler"
        },
    },
    Target: v1.ObjectReference{
        APIVersion: "v1",
        Kind:       "Node",
        Name:       nodeName, // Tên của node tốt nhất
    },
}
```

3. **Thực thi lệnh gọi API Bind**: Bộ lập lịch sau đó sử dụng `clientset` `client-go` của mình để thực hiện một yêu cầu POST đến máy chủ API

Kubernetes để tạo đối tượng `Binding` này. Đường dẫn API cụ thể cho việc này là một tài nguyên con của API Pods:

```
err :=
clientset.CoreV1().Pods(podToSchedule.Namespace).Bind(ctx,
binding, metav1.CreateOptions{})
```



- `ctx` (context) được truyền cho lệnh gọi này, cho phép nó bị hủy nếu bộ lập lịch đang tắt.
- `metav1.CreateOptions{}` được sử dụng vì không cần tùy chọn tạo đặc biệt nào.

4. **Vai trò của Máy chủ API trong việc Binding:** Sau khi nhận và xác thực thành công đối tượng `Binding`, máy chủ API Kubernetes thực hiện việc gán thực tế: nó cập nhật trường `spec.nodeName` của đối tượng `v1.Pod` gốc (được xác định bởi `binding.ObjectMeta.Name` và `binding.ObjectMeta.Namespace`) thành giá trị của `binding.Target.Name`. Khi `spec.nodeName` được đặt, pod được coi là đã được lập lịch.
5. **Kích hoạt Kubelet:** Thành phần Kubelet chạy trên node vừa được gán (tức là node có tên khớp với `spec.nodeName` đã cập nhật của pod) đang theo dõi máy chủ API để tìm các pod được gán cho nó. Khi thấy pod này xuất hiện trong danh sách của mình, nó sẽ tiếp quản trách nhiệm vòng đời của pod trên node đó: kéo các image container cần thiết, tạo sandbox cho container, chạy container, thiết lập mạng và gắn kết volume.
6. **Xử lý lỗi Binding:** Lệnh gọi API `Bind` có thể thất bại vì nhiều lý do (ví dụ: pod đã bị xóa ngay trước khi ràng buộc, sự cố mạng, lỗi máy chủ API, hoặc, ít phổ biến hơn đối với ràng buộc, xung đột tương tranh lạc quan (optimistic concurrency conflicts) nếu một thực thể khác cố gắng cập nhật pod đồng thời, mặc dù điều này hiếm khi xảy ra đối với việc gán `nodeName` thông qua ràng buộc).
 - Nếu `err != nil` từ lệnh gọi `Bind`, hàm `bindPod` của nhóm em sẽ ghi lại một thông báo lỗi chi tiết.
 - Sau đó, nó gọi hàm `postEvent()` để tạo một `Event` Kubernetes với `Reason: "FailedBinding"` và `Type: "Warning"`, bao gồm cả thông báo lỗi. Điều này làm cho lỗi có thể nhìn thấy được đối với quản trị viên. Lỗi sau đó được truyền lên hàm `schedulePod`.

7. **Báo cáo thành công:** Nếu lệnh gọi `Bind` thành công (`err == nil`), `bindPod` sẽ ghi lại thành công và gọi `postEvent` để tạo một `Event` “Scheduled” (Type: “Normal”) cho pod, xác nhận rằng bộ lập lịch tùy chỉnh của nhóm em đã gắn thành công nó cho node đã chọn.

Do đó, giai đoạn ràng buộc là hành động quyết định chuyển đổi quyết định đã tính toán của bộ lập lịch thành một vị trí pod thực tế trong cụm Kubernetes.

3.2.5. Báo cáo sự kiện

Để đảm bảo tính minh bạch và hỗ trợ gỡ lỗi cũng như giám sát hoạt động, bộ lập lịch tùy chỉnh của nhóm em chủ động tạo ra các đối tượng `Event` của Kubernetes tại các điểm chính khác nhau trong quy trình làm việc của nó. Các sự kiện này cung cấp một bản ghi theo trình tự thời gian về các hành động và quyết định của bộ lập lịch liên quan đến các pod cụ thể.

Trách nhiệm: Việc tạo và đăng các sự kiện này được tập trung trong hàm `postEvent` bên trong tệp `scheduler/kubernetes.go`.

Phân tích cơ chế: Hàm `postEvent` được gọi với các tham số chỉ định pod liên quan, một mã `reason` (lý do), một `message` (thông báo) mà con người có thể đọc được, và một `eventType` (loại sự kiện) (“Normal” hoặc “Warning”).

1. **Xây dựng đối tượng `v1.Event`:** Một đối tượng `v1.Event` được xây dựng tỉ mỉ với các trường sau:

- `ObjectMeta.GenerateName`: Đặt thành `pod.Name + "-"`. Điều này hướng dẫn máy chủ API tạo một tên duy nhất cho sự kiện, thường bằng cách nối thêm một hậu tố ngẫu nhiên, điều này cần thiết vì nhiều sự kiện có thể được liên kết với cùng một pod.
- `ObjectMeta.Namespace`: Đặt thành `pod.Namespace`, vì các sự kiện là các đối tượng thuộc không gian tên (namespaced) và nên nằm trong cùng không gian tên với đối tượng mà chúng liên quan đến.
- `InvolvedObject`: Trường quan trọng này là một `v1.ObjectReference` liên kết trực tiếp sự kiện với pod đang được lập lịch. Nó bao gồm:
 - `Kind`: "Pod"
 - `Namespace`: `pod.Namespace`
 - `Name`: `pod.Name`
 - `UID`: `pod.UID` (ID duy nhất của instance pod)

- `APIVersion: "v1"`
 - `ResourceVersion: pod.ResourceVersion` (Liên kết sự kiện với một phiên bản cụ thể của đối tượng pod, hỗ trợ gỡ lỗi nếu đặc tả pod thay đổi).
- `Reason`: Một chuỗi ngắn gọn, máy có thể đọc được cho biết bản chất của sự kiện (ví dụ: “Scheduled”, “FailedScheduling”, “FailedBinding”). Những lý do này có thể được sử dụng để lọc các sự kiện.
- `Message`: Một giải thích chi tiết hơn, con người có thể đọc được về những gì đã xảy ra. Ví dụ, đối với sự kiện “FailedScheduling”, thông báo có thể liệt kê các lỗi vị từ cụ thể.
- `Source`: Một đối tượng `v1.EventSource` xác định thành phần đã tạo ra sự kiện.
 - `Component`: Đặt thành `SchedulerName` (tức là “custom-scheduler”).
- `FirstTimestamp` và `LastTimestamp`: Cả hai đều được đặt thành `metav1.Now()` tại thời điểm tạo sự kiện, vì mỗi lệnh gọi đến `postEvent` tạo ra một instance sự kiện riêng biệt.
- `Count`: Đặt thành `1`.
- `Type`: Đặt thành `"Normal"` (cho các sự kiện thông tin như lập lịch thành công) hoặc `"Warning"` (cho các lỗi hoặc thất bại).
- `ReportingController`: Đặt thành `SchedulerName`. Trường này, cùng với `ReportingInstance`, hữu ích cho việc quy trách nhiệm sự kiện khi nhiều bộ điều khiển (controller) có thể đang hoạt động trên các đối tượng.
- `ReportingInstance`: Đặt thành tên của pod của chính bộ lập lịch. Thông tin này được lấy từ biến môi trường `POD_NAME`, được đưa vào pod của bộ lập lịch bằng cách sử dụng Downward API (như được cấu hình trong `scheduler/custom-scheduler-deployment.yaml` thông qua `valueFrom: fieldRef: fieldPath: metadata.name`). Điều này xác định chính xác instance nào của bộ lập lịch đã tạo ra sự kiện, đặc biệt hữu ích nếu chạy nhiều bản sao (replica) (mặc dù việc triển khai hiện tại của nhóm em sử dụng một bản sao).

2. Đăng sự kiện lên máy chủ API:

- Đối tượng `v1.Event` đã được xây dựng sau đó được tạo trong không gian tên của pod sử dụng `clientset` của `client-go`:

```
clientset.CoreV1().Events(pod.Namespace).Create(eventCtx,
event, metav1.CreateOptions{})
```



- Một `eventCtx` với thời gian chờ ngắn (ví dụ: 10 giây) được sử dụng cho lệnh gọi API này để ngăn bộ lập lịch bị treo vô thời hạn nếu việc tạo sự kiện gặp sự cố.
- Nếu việc tạo sự kiện thất bại (ví dụ: do tải máy chủ API hoặc sự cố mạng), một cảnh báo được ghi lại bởi bộ lập lịch, nhưng lỗi này thường không làm dừng quy trình lập lịch chính cho chính pod đó. Mục tiêu chính là lập lịch cho pod; việc báo cáo sự kiện là thứ yếu, mặc dù quan trọng đối với khả năng quan sát.

Mục đích và Tiện ích của Sự kiện:

- **Gỡ lỗi:** Khi một pod không thể lập lịch, `kubectl describe pod <pod-name>` sẽ hiển thị các sự kiện này, thường cung cấp lý do chính xác (ví dụ: “0/3 nodes are available: 3 Insufficient cpu”).
- **Giám sát:** Quản trị viên cụm có thể theo dõi hoặc truy vấn các sự kiện để giám sát tình trạng và hành vi của bộ lập lịch tùy chỉnh.
- **Dấu vết Kiểm toán:** Các sự kiện đóng vai trò như một bản ghi lịch sử về các quyết định lập lịch.

Bằng cách báo cáo nhất quán các hành động của mình, bộ lập lịch tùy chỉnh của nhóm em tích hợp tốt với các thực tiễn quan sát tiêu chuẩn của Kubernetes, giúp cho hành vi của nó dễ hiểu và có thể chẩn đoán được.

3.3. Cấu trúc mã nguồn tổng thể và các phần chính

Dự án bộ lập lịch Kubernetes tùy chỉnh của nhóm em được tổ chức thành nhiều tệp Go trong thư mục `scheduler/`, cùng với các tệp manifests YAML Kubernetes hỗ trợ trong các thư mục con chuyên dụng. Cấu trúc này thúc đẩy tính mô-đun hóa và tách biệt các mối quan tâm (separation of concerns), giúp mã nguồn dễ quản lý và dễ hiểu hơn. Ngôn ngữ triển khai chính là Go, sử dụng thư viện `client-go` chính thức của Kubernetes để tương tác API.

3.3.1. Logic ứng dụng chính (`scheduler/`)

Các tệp ứng dụng Go chính nằm trực tiếp trong thư mục `scheduler/` :

- `main.go` :

1. **Vai trò:** Tệp này đóng vai trò là điểm vào (entry point) cho ứng dụng bộ lập lịch tùy chỉnh.

2. **Trách nhiệm chính:**

- ▶ Khởi tạo ứng dụng, bao gồm thiết lập ghi log.
- ▶ Phân tích các đối số dòng lệnh hoặc biến môi trường để cấu hình. Quan trọng là, nó gọi `loadScoringConfig` để đọc các trọng số tính điểm (ví dụ: `SCORE_WEIGHT_MEM`, `SCORE_WEIGHT_CPU`, `SCORE_WEIGHT_AFFINITY`) từ các biến môi trường.
- ▶ Khởi tạo Kubernetes `clientset` của `client-go` thông qua `initKubernetesClient`, hàm này cố gắng cấu hình trong cụm (in-cluster).
- ▶ Thiết lập xử lý tín hiệu (cho SIGINT, SIGTERM) để cho phép tắt mềm mại (graceful shutdown) bằng cách sử dụng `context.Context`.
- ▶ Khởi tạo và khởi động Kubernetes shared informer factory (`informers.NewSharedInformerFactory`).
- ▶ Cụ thể, nó thiết lập một Node Informer (`informerFactory.Core().V1().Nodes().Informer()`) với một trình xử lý sự kiện `UpdateFunc`. Trình xử lý này hiện tại ghi log các thay đổi đối với nhãn hoặc taint của node và gọi `scheduleAllUnscheduled` (một hàm đánh giá lại rộng hơn, mặc dù luồng lập lịch chính là dựa trên sự kiện cho mỗi pod). Informer này đảm bảo bộ lập lịch có một cái nhìn nhất quán cuối cùng (eventually consistent) về trạng thái của các node.
- ▶ Chờ cho các bộ đệm cache của informer đồng bộ hóa (`cache.WaitForCacheSync`) trước khi tiếp tục, đảm bảo bộ lập lịch khởi động với thông tin node cập nhật.
- ▶ Gọi `watchUnscheduledPods` (từ `kubernetes.go`) để bắt đầu quá trình bắt đồng bộ theo dõi các pod đủ điều kiện. Hàm này trả về hai kênh:

`podCh` (cho các pod cần lập lịch) và `errCh` (cho các lỗi liên quan đến watch).

- ▶ Triển khai vòng lặp xử lý chính:
 - Sử dụng câu lệnh `select` để lắng nghe trên `podCh`, `errCh`, và context tắt (`ctx.Done()`).
 - Khi một pod được nhận trên `podCh`, nó gọi `schedulePod` (từ `processor.go`) để xử lý logic lập lịch cho pod cụ thể đó.
 - Ghi log bất kỳ lỗi nào nhận được trên `errCh`.
 - Quản lý việc tắt mềm mại bằng cách chờ goroutine xử lý chính hoàn thành bằng cách sử dụng `sync.WaitGroup` khi context bị hủy.

3. Các lệnh gọi bên ngoài chính: `initKubernetesClient`, `loadScoringConfig`, `watchUnscheduledPods`, `schedulePod`.

- `kubernetes.go`:

1. **Vai trò:** Đây là một tệp quan trọng đóng gói hầu hết các tương tác trực tiếp với máy chủ API Kubernetes bằng thư viện `client-go`. Nó cung cấp các lớp trừu tượng hóa trên các lệnh gọi API thô.

2. Trách nhiệm chính:

- ▶ Hằng số `SchedulerName`: Định nghĩa tên (“custom-scheduler”) mà các pod phải chỉ định trong `spec.schedulerName` để được xử lý bởi bộ lập lịch này.
- ▶ `initKubernetesClient()`: Tạo và trả về một `*kubernetes.Clientset`.
- ▶ `watchUnscheduledPods()`: Triển khai logic để theo dõi các pod chưa được lập lịch được gán cho `SchedulerName`, lọc theo `spec.nodeName=""`, và gửi chúng đến một kênh để xử lý. Bao gồm xử lý lỗi và logic thử lại cho watch.
- ▶ `predicateChecks()`: Logic vị từ (predicate) cốt lõi. Nó nhận một pod và clientset, liệt kê các node, và lọc chúng dựa trên yêu cầu tài nguyên, bộ chọn node (node selector), taint/toleration, và trạng thái không thể lập lịch của node. Nó sử dụng các hàm trợ giúp:

- `calculateNodeUsage()` : Tính toán các yêu cầu tài nguyên hiện tại trên các node.
- `calculatePodResourceRequests()` : Tính toán tài nguyên được yêu cầu bởi pod mới.
- `checkNodeResources()` : Kiểm tra xem một node có đủ tài nguyên có thể cấp phát hay không.
- `checkNodeTaints()` : Kiểm tra toleration của pod so với taint của node.
- `tolerationsTolerateTaint()` & `tolerationsTolerateTaints()` : Các hàm trợ giúp cho việc xử lý taint.
- ▶ `bindPod()` : Xây dựng một đối tượng `v1.Binding` và sử dụng clientset để ràng buộc một pod với một node được chỉ định.
- ▶ `postEvent()` : Tạo và đăng các đối tượng `v1.Event` lên API Kubernetes để báo cáo các quyết định lập lịch và lỗi.

3. **Các phụ thuộc chính:** `k8s.io/client-go/kubernetes`, `k8s.io/api/core/v1`, `k8s.io/apimachinery/...`

- `scoring.go` :

1. **Vai trò:** Tập này chứa tất cả logic liên quan đến việc tính điểm các node tương thích, đây là trái tim của chiến lược ưu tiên hóa tùy chỉnh. Nó xử lý giao tiếp với Prometheus để lấy các số liệu thời gian thực.

2. **Các cấu trúc dữ liệu chính:**

- ▶ `ScoringConfig` : Struct để giữ các trọng số cho điểm bộ nhớ, CPU, và affinity, được tải từ các biến môi trường.
- ▶ `NodeScore` : Struct để lưu trữ điểm riêng lẻ và tổng điểm cho một node.
- ▶ `MetricResponse`, `Data`, `Result` : Các struct để giải tuần tự (unmarshal) các phản hồi JSON từ API Prometheus.

3. **Trách nhiệm chính:**

- ▶ Hằng số `prometheusService` : Định nghĩa địa chỉ nội bộ cụm của dịch vụ Prometheus.

- ▶ Các hằng số `memPromQL`, `cpuIdlePromQL`: Định nghĩa các truy vấn PromQL để lấy số liệu bộ nhớ và CPU.
- ▶ `queryPrometheus()`: Gửi các yêu cầu HTTP GET đến Prometheus, thực thi các truy vấn PromQL, và phân tích phản hồi JSON. Bao gồm xử lý lỗi cho giao tiếp với Prometheus.
- ▶ `getNodeNameFromMetric()` & `parseInstanceLabel()`: Các hàm trợ giúp để trích xuất tên node Kubernetes từ các nhãn số liệu Prometheus (cụ thể là nhãn `instance`, được đặt bởi các quy tắc gán lại nhãn trong cấu hình Prometheus).
- ▶ `parseMetricValue()`: Chuyển đổi các giá trị chuỗi số liệu từ Prometheus thành `float64`.
- ▶ `WorkspaceNodeMetrics()`: Điều phối việc lấy cả số liệu bộ nhớ và CPU cho tất cả các node tương thích từ Prometheus.
- ▶ `normalizeScore()`: Chuẩn hóa các giá trị số liệu thô (hoặc tổng affinity) về thang điểm 0-10, cần thiết cho việc tính điểm có trọng số.
- ▶ `calculateAffinityScore()`: Tính toán điểm 0-10 dựa trên các quy tắc affinity node `preferredDuringSchedulingIgnoredDuringExecution` của pod và nhãn của node. Sử dụng `NodeSelectorRequirementsAsSelector` để phân tích `matchExpressions`.
- ▶ `ScoreNodes()`: Hàm tính điểm chính. Nó nhận danh sách các node tương thích, pod, và `ScoringConfig`. Nó gọi `WorkspaceNodeMetrics`, sau đó đối với mỗi node, tính toán điểm bộ nhớ, CPU, và affinity, chuẩn hóa chúng, và kết hợp chúng thành tổng điểm có trọng số. Cuối cùng, nó sắp xếp các node theo tổng điểm (có phá vỡ hòa) và trả về tên của node tốt nhất.

4. **Các phụ thuộc chính:** Các gói HTTP, JSON, math, sort tiêu chuẩn của Go; `k8s.io/api/core/v1` (cho các kiểu pod/node); `k8s.io/apimachinery/pkg/labels`, `k8s.io/apimachinery/pkg/selection`.

- `getBestNode.go`:

1. **Vai trò:** Tập này hoạt động như một bộ điều phối đơn giản hoặc cầu nối giữa giai đoạn vị từ và logic tính điểm chi tiết.

2. Trách nhiệm chính:

- ▶ Hàm `getBestNode()` nhận danh sách các node tương thích (từ `predicateChecks`), pod cần lập lịch, và `ScoringConfig`.
 - ▶ Nó gọi trực tiếp `ScoreNodes()` (từ `scoring.go`) để lấy tên của node tốt nhất.
 - ▶ Sau đó, nó lặp qua danh sách `compatibleNodes` gốc để tìm và trả về đối tượng `v1.Node` đầy đủ tương ứng với `bestNodeName`.
 - ▶ Bao gồm xử lý lỗi cơ bản nếu `ScoreNodes` thất bại hoặc nếu không thể tìm thấy tên node tốt nhất trong danh sách tương thích (điều này sẽ chỉ ra một lỗi logic nội bộ).
- `processor.go`:

1. **Vai trò:** Tập này chứa logic cấp cao nhất để xử lý một pod đơn lẻ qua toàn bộ quy trình lập lịch.

2. **Các cấu trúc dữ liệu chính:** `processorLock` (một `sync.Mutex` ban đầu nhằm ngăn chặn việc xử lý đồng thời cùng một pod, mặc dù ít quan trọng hơn với mô hình theo dõi dựa trên sự kiện hiện tại so với mô hình thăm dò/đối chiếu).

3. Trách nhiệm chính:

- ▶ `schedulePod()`: Đây là hàm chính được `main.go` gọi cho mỗi pod mới.
 - Nó điều phối luồng lập lịch:
 1. Gọi `predicateChecks()` để lấy các node tương thích.
 2. Gọi `getBestNode()` (hàm này lại gọi `ScoreNodes()`) để chọn node tốt nhất từ danh sách tương thích.
 3. Triển khai một phương án dự phòng: nếu `getBestNode()` thất bại nhưng vẫn tồn tại các node tương thích, nó sẽ ghi log vấn đề và có thể chọn node tương thích đầu tiên làm phương án dự phòng để đảm bảo pod được lập lịch nếu có thể.
 4. Gọi `bindPod()` để gán pod cho node đã chọn.

- ▶ Xử lý lỗi từ mỗi bước và đảm bảo ghi log thích hợp.
 - ▶ `scheduleAllUnscheduled()` : Một hàm (hiện được gọi bởi trình xử lý cập nhật của node informer trong `main.go`) liệt kê tất cả các pod chưa được lập lịch cho `custom-scheduler` và cố gắng lập lịch cho chúng. Hàm này hoạt động như một cơ chế đối chiếu, mặc dù luồng lập lịch chính là dựa trên sự kiện.
- `types.go` :
 1. **Vai trò:** Tập này chứa các định nghĩa struct Go (như `Event`, `Pod`, `Node`, `Binding`) phản chiếu các đối tượng API Kubernetes.
 2. **Bối cảnh lịch sử:** Nó được ghi chú rõ ràng trong các bình luận là ban đầu từ dự án bộ lập lịch Kubernetes ví dụ của Kelsey Hightower.
 3. **Trạng thái hiện tại:** Mặc dù có mặt, dự án của nhóm em hiện chủ yếu sử dụng trực tiếp các kiểu API Kubernetes chính thức, có phiên bản từ `k8s.io/api/core/v1` và các gói `k8s.io/apimachinery` khác (ví dụ: `v1.Pod`, `v1.Node`). Do đó, các kiểu tùy chỉnh trong `types.go` phần lớn là dấu tích hoặc đóng vai trò tham khảo lịch sử từ nguồn cảm hứng ban đầu của dự án. Chúng không phải là các cấu trúc dữ liệu chính được sử dụng để tương tác với API Kubernetes trong mã nguồn hiện tại.
- `Dockerfile` :
 1. **Vai trò:** Định nghĩa các chỉ dẫn để xây dựng image container Docker cho bộ lập lịch tùy chỉnh.
 2. **Các tính năng chính:**
 - ▶ Sử dụng một bản dựng đa giai đoạn (multi-stage build):
 - Một image `golang:1.23-alpine` (hoặc tương tự) làm giai đoạn `builder` để biên dịch ứng dụng Go. Giai đoạn này bao gồm việc sao chép `go.mod` và `go.sum` trước để lưu cache bản dựng, sau đó là phần còn lại của mã nguồn, và cuối cùng là chạy `go build`.
 - Lệnh dựng
`CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build -ldflags="-w -s" -o /scheduler` .
 tạo ra một tệp nhị phân Linux được liên kết tĩnh, tối ưu hóa về kích thước (`-w -s` loại bỏ các ký hiệu gỡ lỗi).

- Một image `alpine:latest` tối giản làm giai đoạn cuối cùng.
- Chỉ sao chép tệp nhị phân đã biên dịch (`/scheduler`) từ giai đoạn `builder` vào image cuối cùng.
- Đặt `ENTRYPOINT` thành `["/scheduler"]` để chạy ứng dụng khi container khởi động.

3. Cách tiếp cận này tạo ra một image container cuối cùng nhỏ gọn, an toàn.

3.3.2. Các tệp manifests Kubernetes

Ngoài mã Go, một số tệp YAML định nghĩa các tài nguyên Kubernetes cần thiết để triển khai và chạy bộ lập lịch cũng như các phụ thuộc của nó:

- `scheduler/custom-scheduler-deployment.yaml` :
 1. Định nghĩa `Deployment` Kubernetes cho chính bộ lập lịch tùy chỉnh.
 2. Chỉ định chạy trong không gian tên `kube-system`, số lượng bản sao (thường là 1), bộ chọn (selector), và mẫu pod (pod template).
 3. Quan trọng là, trong đặc tả container:
 - Tham chiếu đến image Docker được xây dựng bởi `Dockerfile` (ví dụ: `gcr.io/YOUR_PROJECT_ID/custom-scheduler:v1`).
 - Đặt `imagePullPolicy: Always` (hữu ích trong quá trình phát triển).
 - Định nghĩa các biến môi trường cho `ScoringConfig` (`SCORE_WEIGHT_MEM`, `SCORE_WEIGHT_CPU`, `SCORE_WEIGHT_AFFINITY`).
 - Đặt tên của pod bộ lập lịch làm biến môi trường `POD_NAME` bằng cách sử dụng Downward API (`fieldRef: metadata.name`), được sử dụng để báo cáo sự kiện.
 - Gán `ServiceAccount` `custom-scheduler-sa`.
- `scheduler/scheduler-rbac.yaml` :
 1. Định nghĩa các quyền Kiểm soát Truy cập Dựa trên Vai trò (RBAC) mà bộ lập lịch tùy chỉnh yêu cầu.
 2. Tạo một `ServiceAccount` tên là `custom-scheduler-sa` trong không gian tên `kube-system`.
 3. Định nghĩa một `ClusterRole` tên là `custom-scheduler-role` cấp các quyền cho:
 - `get, list, watch` trên `Pods` và `Nodes` (toàn cụm).

- ▶ `create` trên `Pods/binding` (hành động lập lịch cốt lõi).
 - ▶ `create, patch` trên `events` (để báo cáo).
- 4. Định nghĩa một `ClusterRoleBinding` tên là `custom-scheduler-rb` ràng buộc `custom-scheduler-role` với `ServiceAccount custom-scheduler-sa`.
- `scheduler/deployments/`: Thư mục con này chứa các tệp manifest YAML ví dụ để triển khai các khối lượng công việc nhằm kiểm thử bộ lập lịch:
 1. `pod-preferred-affinity.yaml`: Một `Pod` minh họa `affinity node preferredDuringSchedulingIgnoredDuringExecution`, sẽ được lập lịch bởi `custom-scheduler`.
 2. `sleep.yaml`: Một `Deployment` yêu cầu lượng bộ nhớ đáng kể, hữu ích để tạo áp lực tài nguyên.
 3. `sysbench.yaml`: Một `Job` có thể chạy các bài kiểm tra hiệu năng chuyên sâu về bộ nhớ.
 4. `testcustom.yaml`: Một `Deployment` Nginx đơn giản được cấu hình để sử dụng `custom-scheduler`.
 5. `testdefault.yaml`: Một `Deployment` Nginx tương tự được cấu hình để sử dụng bộ lập lịch Kubernetes mặc định (bằng cách bỏ qua `schedulerName`).
- `prometheus/`: Thư mục này chứa các tệp manifest để triển khai Prometheus:
 1. `clusterRole.yaml`: RBAC cho Prometheus (`ServiceAccount`, `ClusterRole`, `ClusterRoleBinding`) cho phép nó khám phá các dịch vụ, pod, node, v.v., để thu thập số liệu (scraping).
 2. `config-map.yaml`: Chứa cấu hình Prometheus (`prometheus.yaml`) và các quy tắc cảnh báo (`prometheus.rules`). Tệp `prometheus.yaml` định nghĩa các công việc thu thập số liệu (scrape job), bao gồm cả những công việc cho `kubernetes-apiservers`, `kubernetes-nodes` (kubelet), `kubernetes-cadvisor`, và `kubernetes-pods` (khám phá `node-exporter` thông qua các chú thích). Các quy tắc gán lại nhãn (relabeling rule) ở đây rất cần thiết để đảm bảo các số liệu có các nhãn hữu ích như `instance` (tên node).

3. `prometheus-deployment.yaml`: `Deployment` cho chính máy chủ Prometheus, gán config map và chỉ định một volume cho cơ sở dữ liệu chuỗi thời gian (TSDB) của nó, hiện tại là một `emptyDir` cho đơn giản nhưng được ghi chú để thay thế bằng `PersistentVolumeClaim` trong môi trường sản xuất. Bao gồm các probe liveness/readiness.
4. `prometheus-service.yaml`: Định nghĩa một `Service` `ClusterIP` để phơi bày máy chủ Prometheus trong cụm (ví dụ: trên cổng 8080, nhắm mục tiêu cổng container 9090).
- `node-exporter/node-exporter-daemonset.yml`:
 1. Định nghĩa `DaemonSet` cho Prometheus Node Exporter.
 2. Đảm bảo rằng một pod `node-exporter` chạy trên mọi node trong cụm.
 3. Cấu hình `hostPID`, `hostIPC`, `hostNetwork` để truy cập thông tin cấp máy chủ (host).
 4. Gắn các đường dẫn máy chủ như `/proc`, `/sys`, và `/` (rootfs) ở chế độ chỉ đọc vào container để `node-exporter` có thể thu thập số liệu từ chúng.
- `README.md`: Tập tài liệu chính cho dự án, cung cấp tổng quan, hướng dẫn cài đặt, ví dụ sử dụng, và thông tin chi tiết về kiến trúc.

3.4. Các khía cạnh cấu hình và triển khai

Hoạt động thành công của bộ lập lịch Kubernetes tùy chỉnh của nhóm em không chỉ dựa vào logic ứng dụng Go của nó mà còn vào một tập hợp các tài nguyên Kubernetes được cấu hình cẩn thận. Phần này trình bày chi tiết cách bộ lập lịch xác định các pod để quản lý, các quyền Kiểm soát Truy cập Dựa trên Vai trò (RBAC) mà nó yêu cầu, cấu hình triển khai của chính nó, việc thiết lập ngăn xếp giám sát cần thiết (Prometheus và Node Exporter), và cách các khối lượng công việc ví dụ được định nghĩa để kiểm thử và minh họa.

3.4.1. Nhận dạng Bộ lập lịch: Chỉ định Pod cho Lập lịch Tùy chỉnh

Một khía cạnh cơ bản của việc tích hợp một bộ lập lịch tùy chỉnh là cung cấp một cơ chế để Kubernetes và người dùng chỉ định pod nào sẽ được xử lý bởi nó, thay vì bởi bộ lập lịch Kubernetes mặc định hoặc các bộ lập lịch tùy chỉnh khác có thể có trong cụm.

Cơ chế: Bộ lập lịch tùy chỉnh của nhóm em sử dụng cơ chế Kubernetes tiêu chuẩn cho mục đích này: trường `spec.schedulerName` trong đặc tả của Pod.

- **Cấu hình tệp manifest Pod (Pod Manifest):** Để hướng một pod đến bộ lập lịch tùy chỉnh của nhóm em, tệp manifest YAML của nó phải bao gồm:

```
apiVersion: v1
kind: Pod # hoặc một phần của mẫu Deployment, StatefulSet, Job,
v.v.
metadata:
  name: my-custom-scheduled-pod
spec:
  schedulerName: custom-scheduler # Chuỗi chính xác này là quan
  trọng
  containers:
  - name: my-container
    image: nginx
```

- **Tham chiếu Hằng số:** Chuỗi “`custom-scheduler`” được định nghĩa là hằng số `SchedulerName` trong `scheduler/kubernetes.go`. Cơ chế theo dõi của bộ lập lịch (trong `watchUnscheduledPods`) lọc một cách tường minh các pod có `schedulerName` này và đang ở trạng thái chờ (pending).
- **Hành vi Mặc định:** Nếu `spec.schedulerName` bị bỏ qua trong đặc tả của pod, hoặc nếu nó được đặt thành `default-scheduler`, pod sẽ được xử lý bởi bộ lập lịch mặc định tích hợp sẵn của Kubernetes. Triển khai ví dụ `scheduler/deployments/testdefault.yaml` của nhóm em minh họa điều này bằng cách bỏ qua trường `schedulerName`.

Việc chỉ định tường minh này đảm bảo một hợp đồng rõ ràng: chỉ những pod tường minh chọn tham gia mới được quản lý bởi logic tùy chỉnh của nhóm em, cho phép cùng tồn tại với bộ lập lịch mặc định.

3.4.2. Kiểm soát truy cập dựa trên Vai trò (RBAC)

Cả bộ lập lịch tùy chỉnh của nhóm em và Prometheus đều yêu cầu các quyền cụ thể để tương tác với máy chủ API Kubernetes và thực hiện các chức năng của chúng. Các quyền này được cấp bằng cách sử dụng các tài nguyên RBAC của Kubernetes: `ServiceAccount`, `ClusterRole`, và `ClusterRoleBinding`.

3.4.2.1. RBAC cho bộ lập lịch tùy chỉnh

(**scheduler/scheduler-rbac.yaml**):

Pod bộ lập lịch tùy chỉnh của nhóm em cần các quyền để quan sát trạng thái của cụm và thực thi các quyết định lập lịch của nó.

- **ServiceAccount** (**custom-scheduler-sa**):

1. Một **ServiceAccount** chuyên dụng có tên **custom-scheduler-sa** được tạo trong không gian tên **kube-system**. Pod bộ lập lịch tùy chỉnh chạy dưới danh tính này. Việc triển khai các thành phần của bộ lập lịch vào **kube-system** là một thực tiễn phổ biến đối với các tiện ích bổ sung cấp cụm.

- **ClusterRole** (**custom-scheduler-role**):

1. **ClusterRole** này định nghĩa các quyền cần thiết ở phạm vi cụm:
 - ▶ **apiGroups:** [""], **resources:** ["pods"], **verbs:** ["get", "list", "watch"] : Để tìm và giám sát các pod chưa được lập lịch trên tất cả các không gian tên.
 - ▶ **apiGroups:** [""], **resources:** ["nodes"], **verbs:** ["get", "list", "watch"] : Để lấy thông tin node cho các kiểm tra vị từ và tính điểm (ví dụ: nhãn, taint, tài nguyên có thể cấp phát).
 - ▶ **apiGroups:** [""], **resources:** ["pods/binding"], **verbs:** ["create"] : Đây là quyền cốt lõi cho phép bộ lập lịch gán một pod cho một node bằng cách tạo một đối tượng **Binding**. Ràng buộc được tạo trong không gian tên của pod.
 - ▶ **apiGroups:** [""], **resources:** ["events"], **verbs:** ["create", "patch"] : Để tạo các đối tượng **Event** của Kubernetes nhằm báo cáo các thành công hoặc thất bại trong lập lịch.
2. Các quyền cho **pods/status** (patch, update) được chú thích lại (comment out) vì bộ lập lịch của nhóm em sử dụng tài nguyên con **pods/binding**, đây là phương pháp được ưu tiên.

- **ClusterRoleBinding** (**custom-scheduler-rb**):

1. `ClusterRoleBinding` này liên kết `custom-scheduler-role` (`ClusterRole`) với `ServiceAccount` `custom-scheduler-sa` (trong không gian tên `kube-system`). Việc cấp quyền này áp dụng trên toàn cụm, cho phép bộ lập lịch hoạt động trên các pod và node từ bất kỳ không gian tên nào theo yêu cầu chức năng của nó.

3.4.2.2. RBAC cho Prometheus (`prometheus/clusterRole.yaml`):

Prometheus cũng yêu cầu các quyền để khám phá và thu thập số liệu từ các thành phần Kubernetes khác nhau.

- **ServiceAccount** (`prometheus-sa`):

1. Một `ServiceAccount` chuyên dụng có tên `prometheus-sa` được tạo trong không gian tên `monitoring` (nơi Prometheus được triển khai).

- **ClusterRole** (`prometheus`):

1. Cấp các quyền cho:

- ▶ `apiGroups: [""]`,
`resources: ["nodes", "services", "endpoints", "pods"]`,
`verbs: ["get", "list", "watch"]`: Cần thiết cho việc khám phá dịch vụ Kubernetes (`kubernetes_sd_configs`), cho phép Prometheus tìm các mục tiêu để thu thập số liệu (ví dụ: các pod có chú thích cụ thể, các điểm cuối dịch vụ, các node cho số liệu kubelet/cadvisor).
- ▶ `apiGroups: [""]`, `resources: ["nodes/proxy"]`,
`verbs: ["get", "list", "watch"]`: Cho phép Prometheus truy cập trực tiếp các điểm cuối API của Kubelet thông qua proxy của máy chủ API, được sử dụng bởi các công việc thu thập số liệu như `kubernetes-nodes` và `kubernetes-cadvisor`.
- ▶ `apiGroups: ["networking.k8s.io"]`, `resources: ["ingresses"]`,
`verbs: ["get", "list", "watch"]`: Để khám phá và thu thập số liệu từ các đối tượng Ingress nếu được cấu hình.
- ▶ `nonResourceURLs: ["/metrics"]`, `verbs: ["get"]`: Để thu thập số liệu từ điểm cuối `/metrics` của chính máy chủ API Kubernetes.

- **ClusterRoleBinding** (`prometheus`):

1. Liên kết `ClusterRole` `prometheus` với `ServiceAccount` `prometheus-sa` (trong không gian tên `monitoring`).

Các cấu hình RBAC này tuân thủ nguyên tắc đặc quyền tối thiểu (principle of least privilege) ở những nơi khả thi, chỉ cấp các quyền cần thiết để mỗi thành phần hoạt động chính xác.

3.4.3. Triển khai bộ lập lịch tùy chỉnh

Chính bộ lập lịch tùy chỉnh được triển khai dưới dạng một `Deployment` Kubernetes. tệp manifest này (`scheduler/custom-scheduler-deployment.yaml`) chỉ định cách pod của bộ lập lịch sẽ được cấu hình và chạy.

- **Không gian tên (Namespace):** Được triển khai vào không gian tên `kube-system`, phổ biến cho các dịch vụ cốt lõi của cụm.
- **Số bản sao (Replicas):** Thường được đặt là `1`. Việc chạy nhiều bản sao của một bộ lập lịch tùy chỉnh mà tất cả cùng theo dõi các pod giống nhau có thể dẫn đến tình trạng tranh chấp (race condition) khi nhiều instance cố gắng lập lịch cho cùng một pod. Cơ chế bầu chọn lãnh đạo (leader election) sẽ cần thiết cho một thiết lập có tính sẵn sàng cao, điều này nằm ngoài phạm vi hiện tại của dự án của nhóm em nhưng là một mẫu tiêu chuẩn cho các bộ lập lịch trong môi trường sản xuất.
- **`serviceAccountName: custom-scheduler-sa`:** Đảm bảo pod của bộ lập lịch chạy với các quyền được cấp bởi cấu hình RBAC đã thảo luận ở trên.
- **Image Container:**
 1. `image: gcr.io/YOUR_PROJECT_ID/custom-scheduler:v1` (hoặc đường dẫn thích hợp đến registry container của bạn). Điều này tham chiếu đến image Docker được xây dựng bởi `Dockerfile`.
 2. `imagePullPolicy: Always` được chỉ định, hữu ích trong quá trình phát triển để đảm bảo image mới nhất được kéo về nếu tag được sử dụng lại. Đối với các bản phát hành ổn định, `IfNotPresent` có thể được ưu tiên hơn.
- **Biến môi trường cho cấu hình tính điểm:** Đây là một khía cạnh quan trọng để tinh chỉnh hành vi của bộ lập lịch mà không cần xây dựng lại image:
 1. `SCORE_WEIGHT_MEM`: Định nghĩa trọng số cho điểm bộ nhớ khả dụng (ví dụ: `"0.6"`).

2. `SCORE_WEIGHT_CPU`: Định nghĩa trọng số cho điểm CPU khả dụng (lỗi rảnh) (ví dụ: `"0.4"`).
3. `SCORE_WEIGHT_AFFINITY`: Định nghĩa trọng số (hệ số nhân) cho điểm node affinity (ví dụ: `"1.0"`).
4. Các giá trị này được đọc bởi `loadScoringConfig` trong `scheduler/main.go`.

- **Downward API cho `POD_NAME`:**

1. Một biến môi trường `POD_NAME` được đưa vào container:

```
env:
  - name: POD_NAME
    valueFrom:
      fieldRef:
        fieldPath: metadata.name
```

YAML

2. Điều này cho phép bộ lập lịch biết tên pod của chính nó, sau đó được sử dụng trong trường `ReportingInstance` của các đối tượng `Event` Kubernetes mà nó tạo ra, hỗ trợ việc xác định nguồn gốc của các sự kiện từ bộ lập lịch.

Cấu hình triển khai này đảm bảo rằng bộ lập lịch tùy chỉnh của nhóm em chạy với danh tính, quyền và các tham số hành vi (trọng số tính điểm) chính xác.

3.4.4. Triển khai công cụ giám sát thông số (Prometheus & Node Exporter)

Để bộ lập lịch tùy chỉnh của nhóm em hoạt động (vì nó dựa vào các số liệu thời gian thực), một ngăn xếp giám sát Prometheus phải được triển khai và cấu hình chính xác trong cụm.

3.4.4.1. Node Exporter (`node-exporter/node-exporter-daemonset.yml`):

Node Exporter chịu trách nhiệm thu thập các số liệu chi tiết ở cấp máy chủ (host).

- **Triển khai dưới dạng `DaemonSet`:** Điều này đảm bảo rằng một instance của pod Node Exporter chạy trên mọi node có thể lập lịch trong cụm.
- **Truy cập đặc quyền:** Đặc tả pod bao gồm:
 1. `hostPID: true`, `hostIPC: true`, `hostNetwork: true`: Những cài đặt này cung cấp cho container Node Exporter quyền truy cập vào các không

gian tên của máy chủ, điều này cần thiết để thu thập một số số liệu nhất định.

2. `securityContext: privileged: true`: Mặc dù phạm vi rộng, điều này phổ biến đối với Node Exporter để truy cập tất cả thông tin hệ thống cần thiết.

- **Gắn Kết đường dẫn máy chủ (Host Path Mounts):** Các thư mục máy chủ quan trọng được gắn kết ở chế độ chỉ đọc vào container:

1. `/proc` được gắn tại `/host/proc`
2. `/sys` được gắn tại `/host/sys`
3. `/` (hệ thống tệp gốc) được gắn tại `/rootfs`
4. Các điểm gắn kết này cho phép Node Exporter đọc các tệp từ hệ thống tệp `/proc` và `/sys` của máy chủ, là nguồn cung cấp nhiều số liệu.

- **Đối số (Arguments):** Các đối số cụ thể được truyền cho Node Exporter để cấu hình các bộ thu thập (collector) và đường dẫn của nó, ví dụ: `--path.procfs=/host/proc`, `--path.sysfs=/host/sys`, và các loại trừ cho một số loại hệ thống tệp và điểm gắn kết nhất định để tránh sự cố hoặc dữ liệu dư thừa.

- **Chú thích khám phá Prometheus (Prometheus Discovery Annotations):**

1. `prometheus.io/scrape: "true"`
2. `prometheus.io/port: "9100"`
3. Các chú thích này báo hiệu cho Prometheus (khi được cấu hình với `kubernetes_sd_configs` thích hợp) rằng pod này sẽ được thu thập số liệu trên cổng 9100.

3.4.4.2. Prometheus (thư mục `prometheus/`):

Máy chủ Prometheus tổng hợp và cung cấp các số liệu này.

- **Cấu hình (`prometheus/config-map.yaml`):**

1. ConfigMap này (tên là `prometheus-server-conf`, được triển khai trong không gian tên `monitoring`) chứa tệp cấu hình Prometheus chính (`prometheus.yml`) và các quy tắc cảnh báo ví dụ (`prometheus.rules`).

2. **Điểm nổi bật của `prometheus.yml`:**

- `scrape_interval` và `evaluation_interval` được định nghĩa toàn cục.

- `scrape_configs`: Phần này rất quan trọng. Nó định nghĩa các công việc (job) khác nhau về cách Prometheus khám phá và thu thập số liệu từ các mục tiêu:
 - `kubernetes-apiversers`: Thu thập số liệu từ điểm cuối `/metrics` của máy chủ API Kubernetes.
 - `kubernetes-nodes`: Thu thập số liệu từ Kubelet (`/metrics`).
 - `kubernetes-cadvisor`: Thu thập số liệu cAdvisor từ Kubelet (`/metrics/cadvisor`), cung cấp thông tin sử dụng tài nguyên ở cấp container.
 - `kubernetes-pods`: Công việc này sử dụng khám phá dịch vụ `role: pod`. Nó được cấu hình với `relabel_configs` để:
 - Chỉ giữ lại các pod có chú thích `prometheus.io/scrape: "true"`.
 - Sử dụng các chú thích `prometheus.io/path` và `prometheus.io/port` để tùy chỉnh việc thu thập số liệu.
 - Quan trọng là, đối với `node-exporter`, nó đảm bảo nhãn `instance` trên các số liệu được thu thập được đặt thành tên node Kubernetes (lấy từ `__meta_kubernetes_pod_node_name`), điều mà bộ lập lịch của nhóm em dựa vào để tương quan số liệu với các node.
 - `kubernetes-service-endpoints`: Khám phá và thu thập số liệu từ các dịch vụ được chú thích cho Prometheus.

- **Triển khai Prometheus (`prometheus/prometheus-deployment.yaml`):**

1. Triển khai chính máy chủ Prometheus (sử dụng `prom/prometheus:v2.49.1` hoặc một phiên bản gần đây tương tự) trong không gian tên `monitoring`.
2. Gắn ConfigMap `prometheus-server-conf` vào `/etc/prometheus/`.
3. Cấu hình các đối số dòng lệnh cho Prometheus, ví dụ: `--config.file`, `--storage.tsdb.path`.

4. Sử dụng một volume `emptyDir` cho `--storage.tsdb.path` để đơn giản hóa trong dự án này. **Tệp README ghi chú chính xác rằng đối với môi trường sản xuất, nên sử dụng `PersistentVolumeClaim` (PVC) để duy trì dữ liệu Prometheus qua các lần khởi động lại pod.**
5. Bao gồm các probe liveness (`/-/healthy`) và readiness (`/-/ready`) để hoạt động mạnh mẽ.
6. Gán `ServiceAccount` `prometheus-sa`.

- **Dịch vụ Prometheus (`prometheus/prometheus-service.yaml`):**

1. Phơi bày (expose) việc triển khai Prometheus thông qua một `Service` `ClusterIP` có tên `prometheus-service` trong không gian tên `monitoring`.
2. Lắng nghe trên cổng `8080` trong cụm và nhắm mục tiêu cổng `9090` trên các pod Prometheus (nơi giao diện người dùng web và API của Prometheus được phơi bày). Bộ lập lịch tùy chỉnh của nhóm em sử dụng địa chỉ dịch vụ này (`http://prometheus-service.monitoring.svc.cluster.local:8080`) để truy vấn số liệu.

Việc triển khai và cấu hình cẩn thận ngăn xếp giám sát này là điều kiện tiên quyết cho bộ lập lịch tùy chỉnh của nhóm em, vì các quyết định của nó được điều khiển bởi dữ liệu từ các số liệu mà Prometheus cung cấp.

3.4.5. Kiểm thử và minh họa (`scheduler/deployments/`)

Để xác thực chức năng của bộ lập lịch tùy chỉnh của nhóm em và minh họa hành vi của nó so với bộ lập lịch mặc định, một số tệp manifest ví dụ được cung cấp trong thư mục `scheduler/deployments/`.

- **`sleep.yaml` (`Deployment`):** Triển khai các pod chủ yếu tiêu thụ bộ nhớ (yêu cầu `1600Mi`). Điều này giúp tạo ra một kịch bản trong đó tính khả dụng của bộ nhớ khác nhau giữa các node, cho phép quan sát vị trí đặt pod dựa trên nhận biết bộ nhớ của bộ lập lịch.
- **`sysbench.yaml` (`Job`):** Định nghĩa một `Job` để chạy `sysbench` nhằm kiểm thử bộ nhớ. Nó bao gồm `podAntiAffinity` để cố gắng lập lịch các pod của

nó trên các node khác với các pod của triển khai `sleep`, ảnh hưởng hơn nữa đến việc phân phối tài nguyên để kiểm thử.

- **testdefault.yaml (Deployment):** Triển khai một ứng dụng Nginx đơn giản mà không chỉ định `schedulerName`. Pod này sẽ được lập lịch bởi bộ lập lịch mặc định của Kubernetes, đóng vai trò là cơ sở để so sánh.
- **testcustom.yaml (Deployment):** Triển khai một ứng dụng Nginx tương tự nhưng đặt tường minh `spec.schedulerName: custom-scheduler`. Pod này sẽ được xử lý bởi logic lập lịch tùy chỉnh của nhóm em. So sánh vị trí đặt của nó với pod `testdefault` trong các điều kiện tải cụm khác nhau sẽ minh họa tác động của bộ lập lịch tùy chỉnh.
- **pod-preferred-affinity.yaml (Pod):** Một tệp manifest Pod độc lập được thiết kế để kiểm thử tính năng tính điểm node affinity. Nó bao gồm các quy tắc `spec.affinity.nodeAffinity.preferredDuringSchedulingIgnoredDuringExecution` với các trọng số khác nhau cho các node được gắn nhãn `zone=a` so với `zone=b`. Nó cũng chỉ định `schedulerName: custom-scheduler`. Điều này cho phép quan sát trực tiếp cách các ưu tiên affinity ảnh hưởng đến việc tính điểm và quyết định vị trí đặt cuối cùng khi các nhãn node được đặt một cách thích hợp.
- **pod-node-selector-1.yaml (Deployment):** Tệp manifest này định nghĩa một `Deployment` cũng sử dụng `custom-scheduler`. Mục đích chính của nó là để kiểm thử khả năng lựa chọn node. Nó bao gồm một trường `spec.template.spec.nodeSelector`, yêu cầu các pod chỉ được lập lịch trên các node được gắn nhãn `group: red`. Điều này cho phép xác minh cách bộ lập lịch tùy chỉnh xử lý các ràng buộc lựa chọn node rõ ràng.
- **pod-toleration-1.yaml (Deployment):** Tệp manifest này được thiết kế để kiểm thử cách bộ lập lịch tùy chỉnh xử lý các taint và toleration. Nó chỉ định `schedulerName: custom-scheduler` và bao gồm một mục `spec.template.spec.tolerations`. Toleration cụ thể này cho phép các pod của deployment này được lập lịch trên các node có một taint cụ thể (`key1=value1:NoSchedule`). Điều này rất quan trọng để quan sát xem bộ lập lịch tùy chỉnh có xác định và sử dụng chính xác các node mà nếu không sẽ bị hạn chế bởi các taint hay không, miễn là pod có toleration cần thiết.

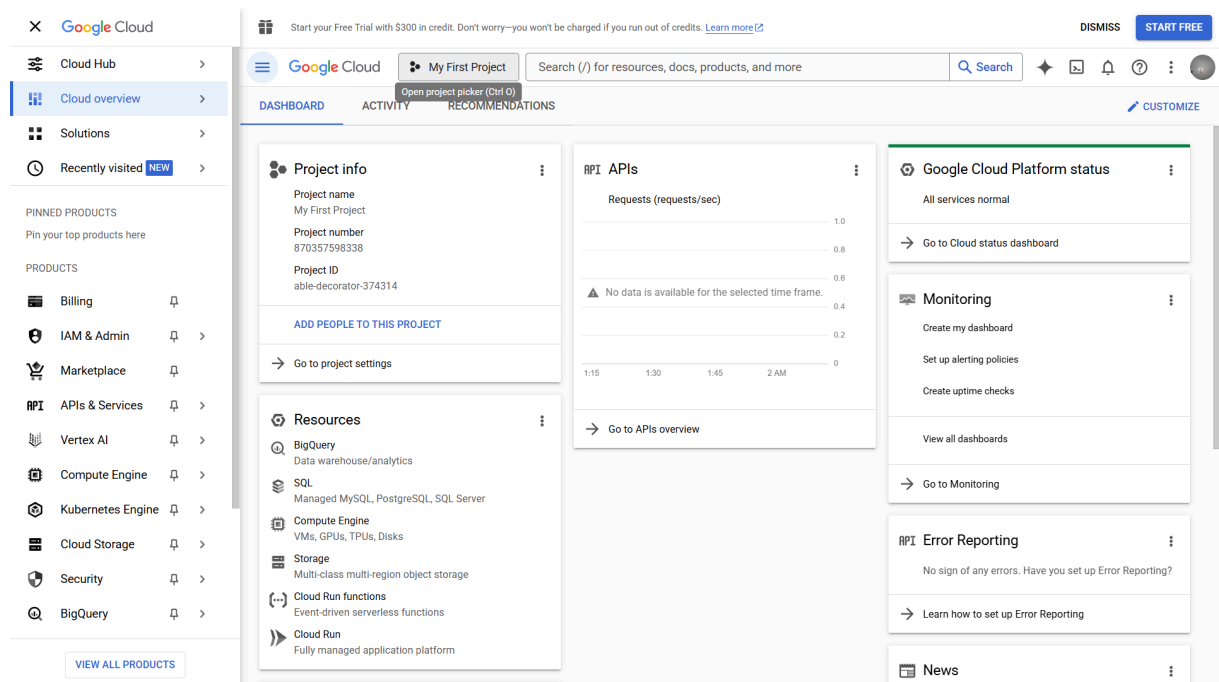
Chương 4

Demo

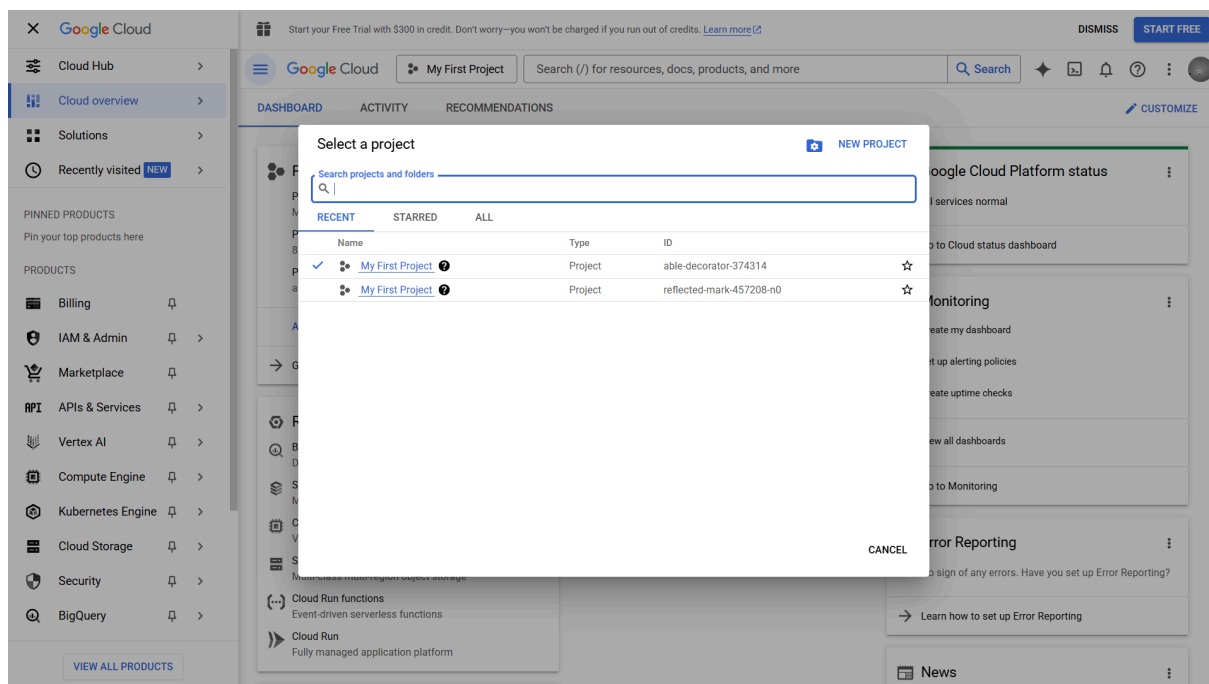
Video demo hoàn chỉnh được đăng tải tại [Github - Custom Kubernetes Scheduler Demo](#). Những lệnh dùng để chạy demo được đăng tải tại [Github - DEMO.md](#).

4.1. Set up Google Kubernetes Engine (GKE)

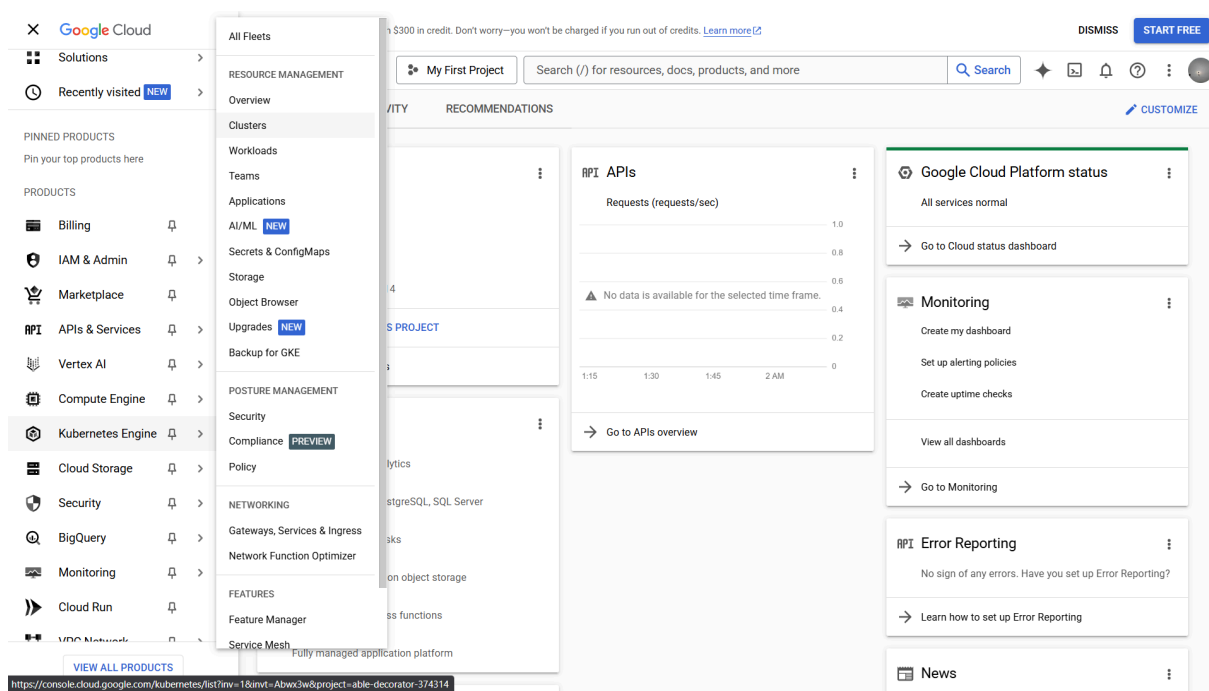
Thực hiện cài đặt GKE theo các bước bên dưới



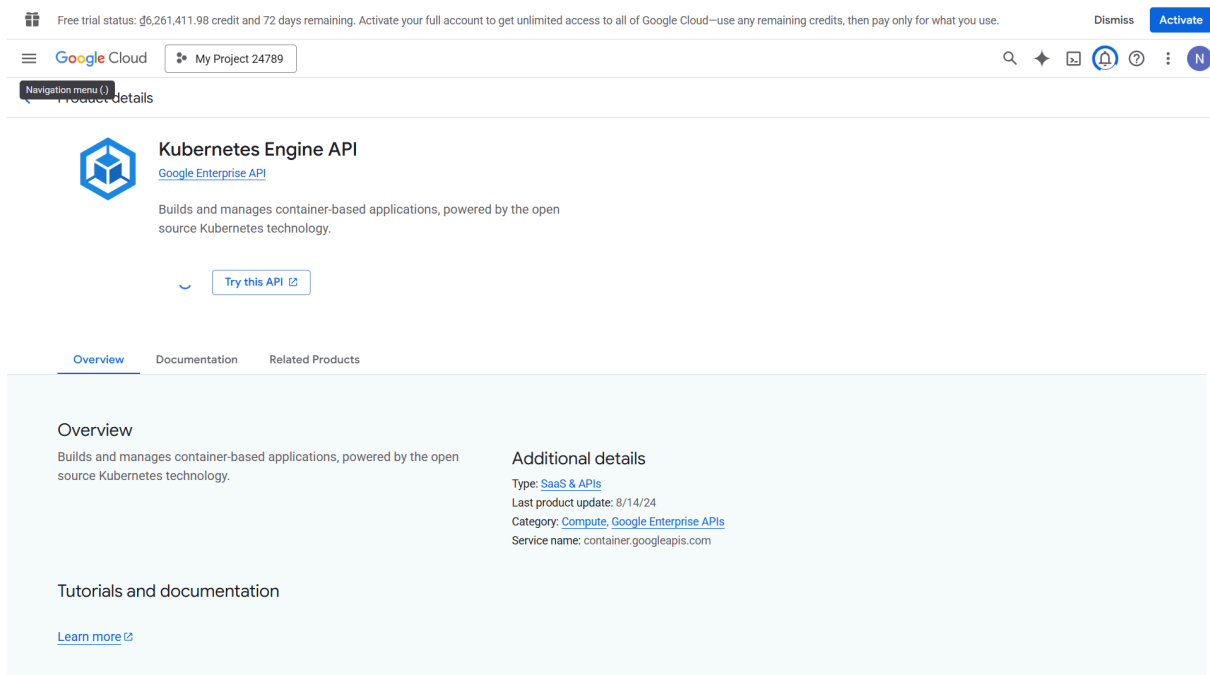
Hình 4.1 — Màn hình Google Cloud



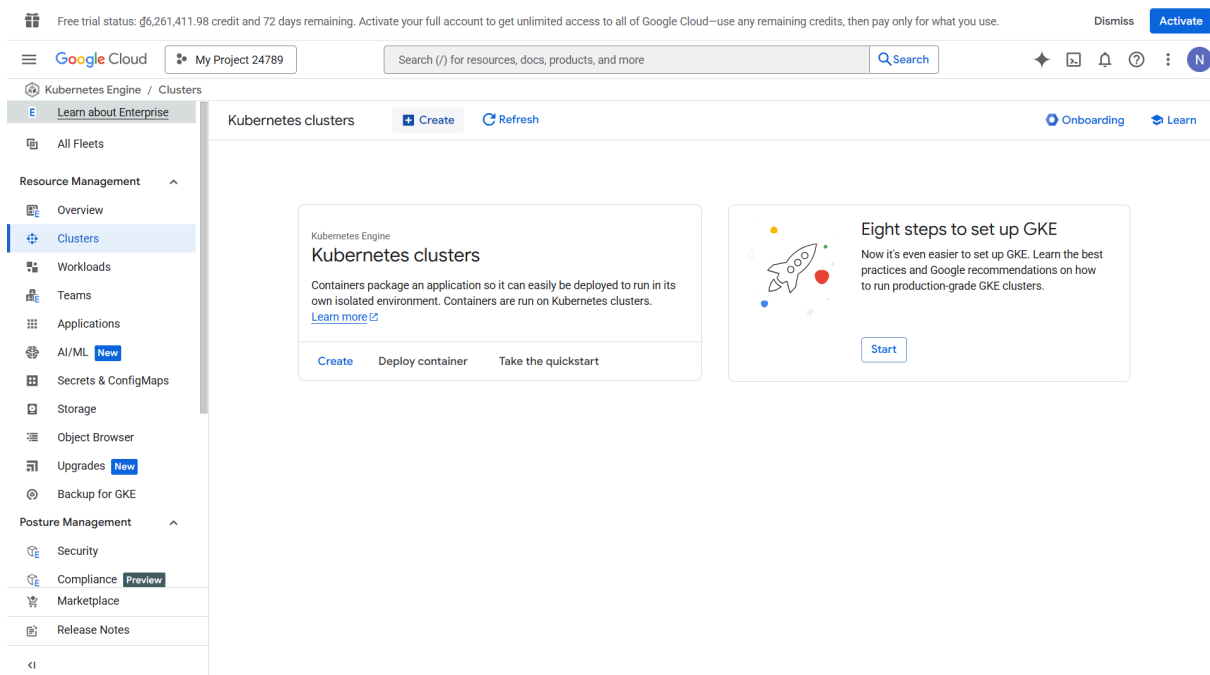
Hình 4.2 — Lựa chọn dự án



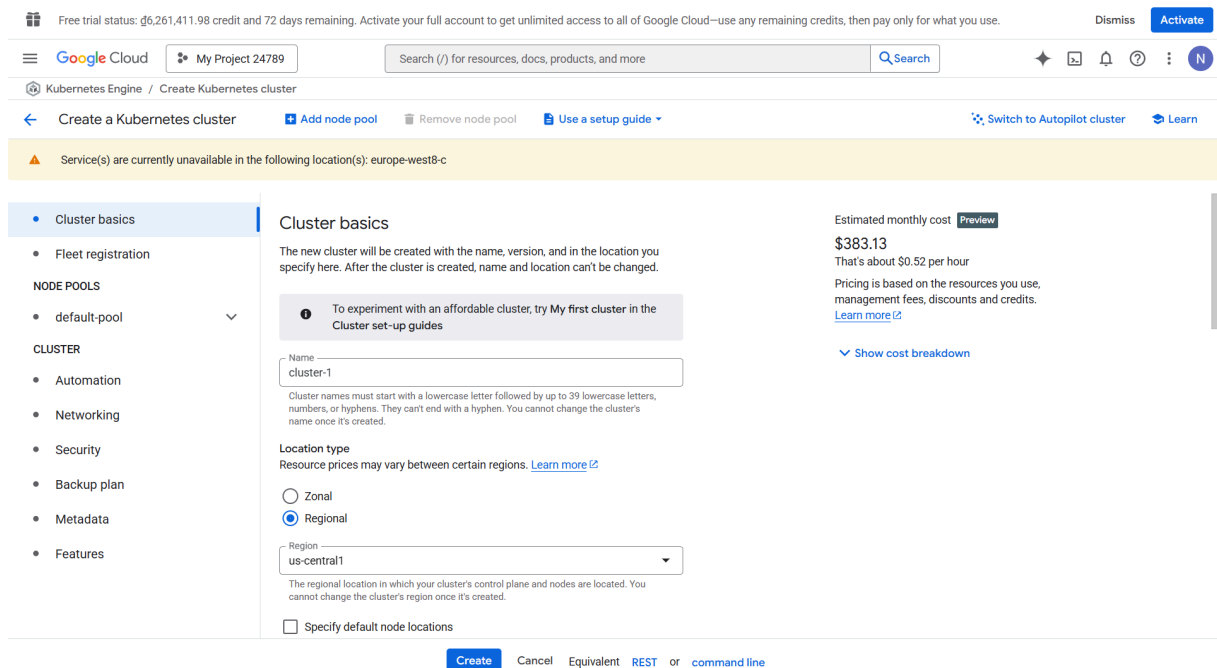
Hình 4.3 — Truy cập vào dịch vụ cluster của GKE



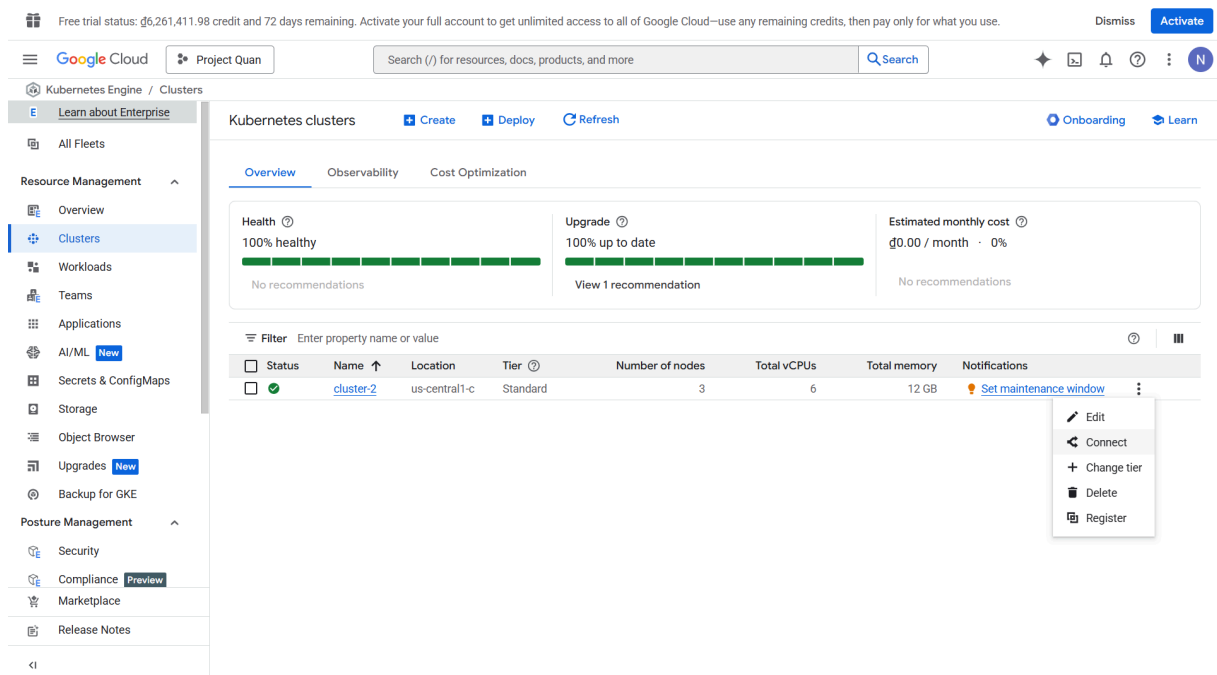
Hình 4.4 — Bật API Kubernetes



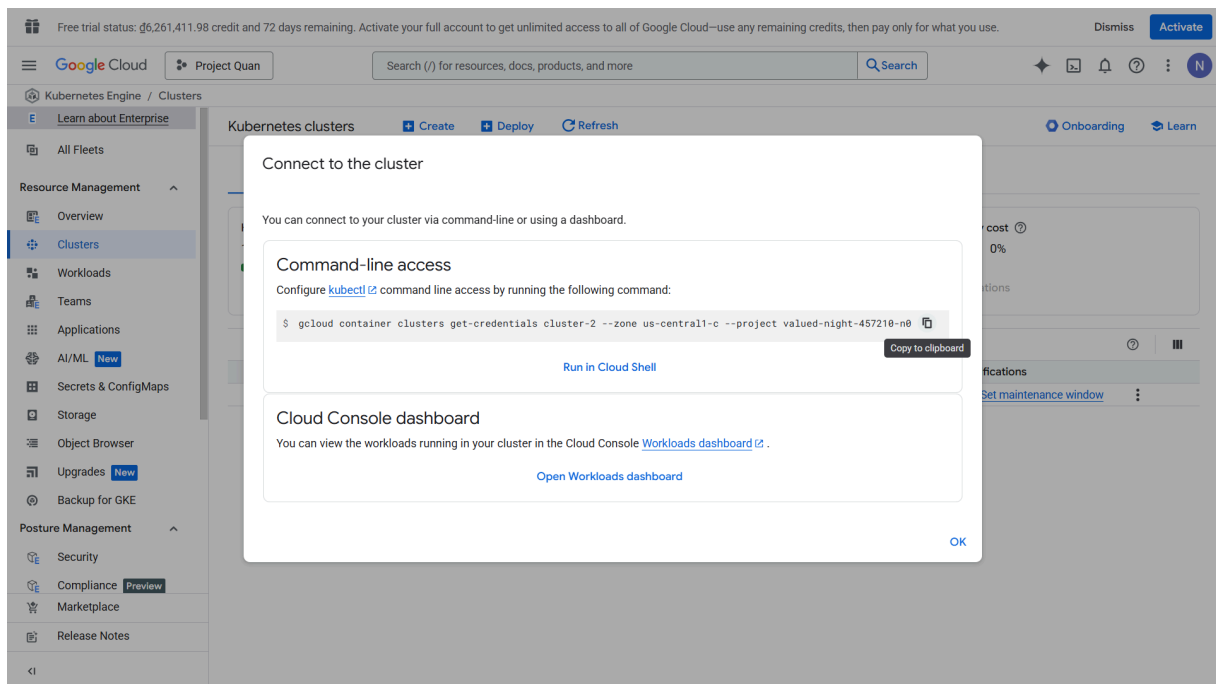
Hình 4.5 — Tạo cluster mới



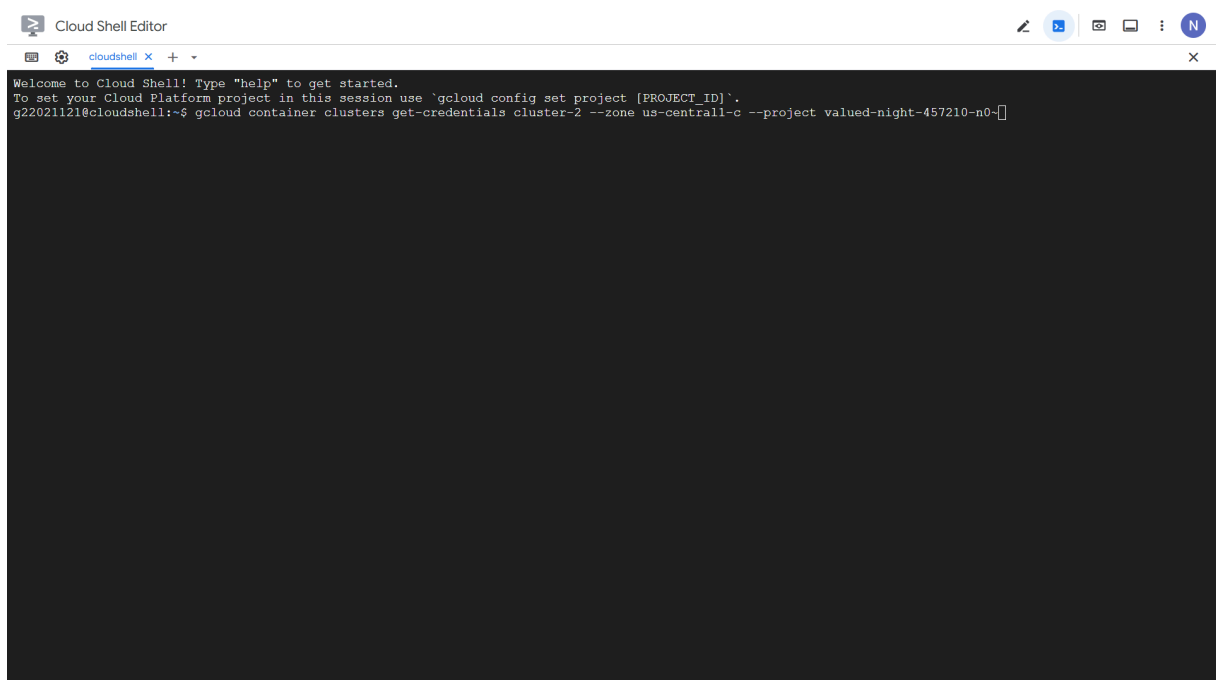
Hình 4.6 — Thiết lập Cluster



Hình 4.7 — Thiết lập kết nối đến master

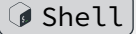


Hình 4.8 — Lấy mã kết nối



Hình 4.9 — Truy cập bằng Google Cloud Shell

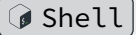
Thực hiện các bước sau trên Google Cloud Shell để clone dự án trình lập lịch tùy chỉnh và build và push image Docker tương ứng của trình lập lịch lên registry của Google Cloud.

```
# Clone the custom scheduler repository (or your fork) 
git clone https://github.com/HynDuf/custom-kubernetes-scheduler.git
cd custom-kubernetes-scheduler

# --- Build and Push Custom Scheduler Docker Image ---
cd scheduler
# The following commands build and push the scheduler image to Google
Container Registry (GCR).
# Ensure PROJECT_ID is correctly captured from your gcloud config.
PROJECT_ID=$(gcloud config get-value project)
docker build -t "gcr.io/${PROJECT_ID}/custom-scheduler:v1" .
gcloud auth configure-docker gcr.io --quiet # Authenticate Docker with
GCR
docker push "gcr.io/${PROJECT_ID}/custom-scheduler:v1"
cd ..
```

Sau đó chỉnh lại đường dẫn của image custom scheduler trong tệp manifest `scheduler/custom-scheduler-deployment.yaml` tương ứng với đường dẫn image ở bên trên.

Tiếp đến, tạo namespace `monitoring` và chạy cái Pods liên quan tới Prometheus và Node Exporter để truy xuất dữ liệu của các nodes:

```
# --- Deploy Monitoring Components (Prometheus & Node
Exporter) --- 
# Create a dedicated namespace for monitoring components
kubectl create namespace monitoring

# Deploy Node Exporter (collects hardware and OS metrics from each
node)
kubectl apply -f node-exporter/node-exporter-daemonset.yaml

# Deploy Prometheus (monitoring system and time series database)
# This includes RBAC, ConfigMap, Deployment, and Service for Prometheus
kubectl apply -f prometheus/clusterRole.yaml
kubectl apply -f prometheus/config-map.yaml -n monitoring
kubectl apply -f prometheus/prometheus-deployment.yaml -n monitoring
kubectl apply -f prometheus/prometheus-service.yaml -n monitoring

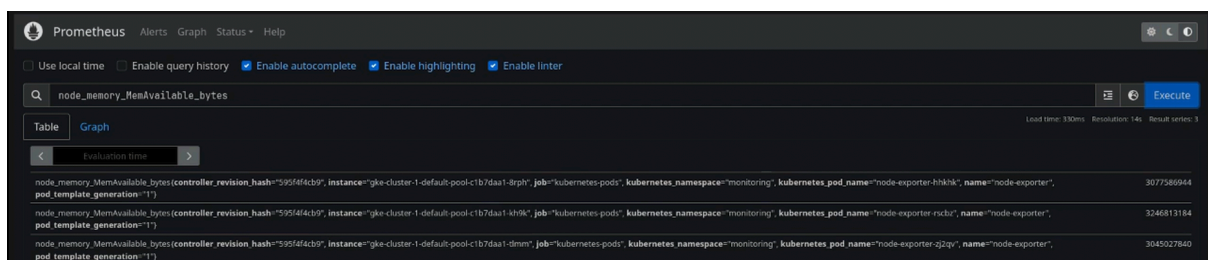
# Verify monitoring components are running
kubectl get pods -o wide -n monitoring
```

```
# You should see one `node-exporter-*` p
```

```
g22021121@cloudshell:~/custom-kubernetes-scheduler (famous-arc-457314-q8)$ kubectl get pods -o wide -n monitoring
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE
node-exporter-hhkhk                 1/1     Running   0          29s   10.128.0.18   gke-cluster-1
-default-pool-c1b7daa1-8rph         <none>   <none>    0          29s   10.128.0.20   gke-cluster-1
node-exporter-rschbz                1/1     Running   0          29s   10.128.0.20   gke-cluster-1
-default-pool-c1b7daa1-kh9k         <none>   <none>    0          29s   10.128.0.19   gke-cluster-1
node-exporter-zj2qv                 1/1     Running   0          29s   10.128.0.19   gke-cluster-1
-default-pool-c1b7daa1-tlmm         <none>   <none>    0          29s   10.128.0.19   gke-cluster-1
prometheus-deployment-57d8bf4c4-2mkw5 0/1     Running   0          16s   10.92.0.4     gke-cluster-1
-default-pool-c1b7daa1-8rph         <none>   <none>    0          29s   10.128.0.20   gke-cluster-1
```

Hình 4.10 — Sau khi chạy cái Pods liên quan tới Prometheus và Node Exporter

Truy cập vào Web UI của Prometheus ta sẽ thấy giao diện và thử truy vấn bộ nhớ còn lại của 3 máy.



Hình 4.11 — Web UI của Prometheus

Sau đó, ta sẽ deploy trình lập lịch tùy chỉnh như sau:

```
# --- Deploy the Custom Scheduler ---
# Apply RBAC rules for the custom scheduler
kubectl apply -f scheduler/scheduler-rbac.yaml
# Deploy the custom scheduler itself (ensure you've updated the image
name in this YAML)
kubectl apply -f scheduler/custom-scheduler-deployment.yaml

# --- Watch Custom Scheduler Logs ---
kubectl logs -n kube-system -l app=custom-scheduler -f --tail=100
```

```

g22021121@cloudshell:~/custom-kubernetes-scheduler (famous-arc-457314-q8)$ kubectl apply -f scheduler/
scheduler-rbac.yaml
serviceaccount/custom-scheduler-sa created
clusterrole.rbac.authorization.k8s.io/custom-scheduler-role created
clusterrolebinding.rbac.authorization.k8s.io/custom-scheduler-rb created
g22021121@cloudshell:~/custom-kubernetes-scheduler (famous-arc-457314-q8)$ kubectl apply -f scheduler/
custom-scheduler-deployment.yaml
deployment.apps/custom-scheduler created
g22021121@cloudshell:~/custom-kubernetes-scheduler (famous-arc-457314-q8)$ kubectl logs -n kube-system
-l app=custom-scheduler -f --tail=100
2025/05/08 08:56:13 Starting custom-scheduler scheduler...
2025/05/08 08:56:13 Loaded Scoring Configuration: MemWeight=0.60, CPUWeight=0.40, AffinityWeight=1.00
2025/05/08 08:56:13 Kubernetes client initialized successfully.
2025/05/08 08:56:13 Waiting for cache sync...
2025/05/08 08:56:13 Node informer synced.
2025/05/08 08:56:13 Scheduler initialized. Waiting for pods...
2025/05/08 08:56:13 Starting watch for unscheduled pods...
2025/05/08 08:56:13 Pod watch started successfully.

```

Hình 4.12 — Xem logs của trình lập lịch tùy chỉnh sau khi vừa deploy

4.2. Default Scheduler vs Custom Scheduler

Sau khi cài đặt thành công thì ta sẽ test 2 trình lập lịch như sau. Ta có `sleep.yaml` sẽ tạo ra 2 Pods yêu cầu 1600Mi bộ nhớ nhưng thật ra không hề sử dụng bộ nhớ, còn `sysbench.yaml` sẽ yêu cầu 0Mi nhưng thực chất lại sử dụng lên đến 2GiB bộ nhớ của node.

```

# --- Demo: Resource-Aware Scheduling ---
# This demo highlights how the custom scheduler considers actual
resource usage,
# unlike the default scheduler which primarily looks at resource
requests.

# Deploy 'sleep.yaml': Requests 1600Mi Memory, but actually uses very
little.
# Deploy 'sysbench.yaml': Requests 0Mi Memory, but actually consumes
nearly 2GiB of Memory.
kubectl apply -f scheduler/deployments/sleep.yaml
kubectl apply -f scheduler/deployments/sysbench.yaml

# Wait for these pods to be scheduled and running.
# Check their status and on which nodes they are placed:
kubectl get pods -o wide

```



```

g22021121@cloudshell:~/custom-kubernetes-scheduler (famous-arc-457314-q8)$ kubectl apply -f scheduler/
deployments/sleep.yaml
deployment.apps/sleep-deployment created
g22021121@cloudshell:~/custom-kubernetes-scheduler (famous-arc-457314-q8)$ kubectl apply -f scheduler/
deployments/sysbench.yaml
job.batch/sysbench-memory-test created
g22021121@cloudshell:~/custom-kubernetes-scheduler (famous-arc-457314-q8)$ kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE
sleep-deployment-859679d47-75mjqr-1-default-pool-c1b7daa1-kh9k  1/1     Running   0           11s   10.92.2.5     gke-clust
sleep-deployment-859679d47-v4vnter-1-default-pool-c1b7daa1-tlmm  1/1     Running   0           11s   10.92.1.13    gke-clust
sysbench-memory-test-v6tj8er-1-default-pool-c1b7daa1-8rph  0/1     ContainerCreating  0           4s    <none>        gke-clust

```

Hình 4.13 — 2 nodes chạy 2 pods `sleep.yaml` và 1 node còn lại chạy pod `sysbench.yaml`

```

(Total limits may be over 100 percent, i.e., overcommitted.)
Resource           Requests           Limits
-----
cpu                 511m (54%)         4100m (436%)
memory              1173704960 (39%)    5710063104 (194%)
ephemeral-storage  0 (0%)             0 (0%)
hugepages-1Gi      0 (0%)             0 (0%)
hugepages-2Mi      0 (0%)             0 (0%)

```

Hình 4.14 — Node chạy `sysbench.yaml` có lượng memory requested thấp

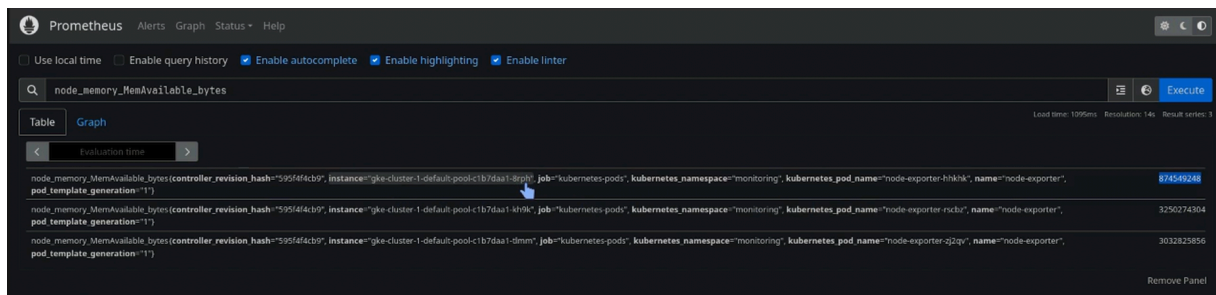
```

(Total limits may be over 100 percent, i.e., overcommitted.)
Resource           Requests           Limits
-----
cpu                 723m (76%)         5143m (547%)
memory              2845969280 (96%)    7517125120 (255%)
ephemeral-storage  0 (0%)             0 (0%)
hugepages-1Gi      0 (0%)             0 (0%)
hugepages-2Mi      0 (0%)             0 (0%)

```

Hình 4.15 — Node chạy `sleep.yaml` có lượng memory requested cao

Tuy vậy, nodes chạy `sleep.yaml` lại có memory trống cao hơn nhiều so với node chạy `sysbench.yaml`.



Hình 4.16 — Thực tế lượng memory trống của các node

Sau đó, ta thử deploy `testdefault.yaml` (sử dụng default scheduler) và `testcustom.yaml` (sử dụng scheduler custom của nhóm).

```
g22021121@cloudshell:~/custom-kubernetes-scheduler (famous-arc-457314-q8)$ kubectl get pods -o wide
```

NAME	READY	STATUS	NOMINATED NODE	READINESS GATES	RESTARTS	AGE	IP	NODE
sleep-deployment-859679d47-75mjq	1/1	Running			0	2m43s	10.92.2.5	gke-clu
ster-1-default-pool-c1b7daa1-kh9k	<none>		<none>					
sleep-deployment-859679d47-v4vnt	1/1	Running			0	2m43s	10.92.1.13	gke-clu
ster-1-default-pool-c1b7daa1-tlmm	<none>		<none>					
sysbench-memory-test-v6tj8	1/1	Running			0	2m36s	10.92.0.6	gke-clu
ster-1-default-pool-c1b7daa1-8rph	<none>		<none>					
testcustom-774b848cdf-gr8mc	0/1	ContainerCreating			0	3s	<none>	gke-clu
ster-1-default-pool-c1b7daa1-kh9k	<none>		<none>					
testdefault-554c5f595f-2kdw5	1/1	Running			0	29s	10.92.0.7	gke-clu
ster-1-default-pool-c1b7daa1-8rph	<none>		<none>					

Hình 4.17 — `testdefault.yaml` được chạy trên máy chạy `sysbench.yaml` (mặc dù máy này đang có rất ít memory trống), còn `testcustom.yaml` được chạy trên máy chạy `sleep.yaml` (máy đang có gần như toàn bộ memory trống)

Xem logs của custom scheduler ta sẽ thấy 3 máy được đánh điểm số như thế nào. Để ý rằng máy chạy `sysbench.yaml` có điểm memory và CPU rất thấp nên scheduler đã chọn máy chạy `sleep.yaml` có điểm số cao hơn nhiều.

```

vent: ADDED)
2025/05/08 08:59:14 Received pod default/testcustom-774b848cdf-gr8mc for scheduling.
2025/05/08 08:59:14 Attempting to schedule pod: default/testcustom-774b848cdf-gr8mc
2025/05/08 08:59:14 Running predicate checks for pod: default/testcustom-774b848cdf-gr8mc
2025/05/08 08:59:14 Pod default/testcustom-774b848cdf-gr8mc requires: CPU=0, Memory=0
2025/05/08 08:59:14 Node gke-cluster-1-default-pool-c1b7daa1-8rph PASSED predicate checks.
2025/05/08 08:59:14 Node gke-cluster-1-default-pool-c1b7daa1-kh9k PASSED predicate checks.
2025/05/08 08:59:14 Node gke-cluster-1-default-pool-c1b7daa1-tlmm PASSED predicate checks.
2025/05/08 08:59:14 Found 3 eligible nodes for pod default/testcustom-774b848cdf-gr8mc (will be scored
).
2025/05/08 08:59:14 Querying Prometheus: http://prometheus-service.monitoring.svc.cluster.local:8080/a
pi/v1/query?query=node_memory_MemAvailable_bytes
2025/05/08 08:59:14 Prometheus query for 'node_memory_MemAvailable_bytes' returned 3 results.
2025/05/08 08:59:14 Querying Prometheus: http://prometheus-service.monitoring.svc.cluster.local:8080/a
pi/v1/query?query=avg+by+%28instance%29+%28rate%28node_cpu_seconds_total%7Bmode%3D%22idle%22%70%5B1m%5
D%29%29
2025/05/08 08:59:14 Prometheus query for 'avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[
1m]))' returned 3 results.
2025/05/08 08:59:14 Max values for normalization - MemAvailable: 3253215232 bytes, CPUIdle(avg cores):
0.917
2025/05/08 08:59:14 Scores for node gke-cluster-1-default-pool-c1b7daa1-8rph: Mem=2.67 (Val=867307520)
, CPU=0.00 (Val=0.000), Affinity=0.00 → Total=1.60
2025/05/08 08:59:14 Scores for node gke-cluster-1-default-pool-c1b7daa1-kh9k: Mem=10.00 (Val=325321523
2), CPU=10.00 (Val=0.917), Affinity=0.00 → Total=10.00
2025/05/08 08:59:14 Scores for node gke-cluster-1-default-pool-c1b7daa1-tlmm: Mem=9.41 (Val=3062095872
), CPU=9.90 (Val=0.908), Affinity=0.00 → Total=9.61
2025/05/08 08:59:14 Selected best node: gke-cluster-1-default-pool-c1b7daa1-kh9k (Total Score: 10.00,
MemScore: 10.00, CPUScore: 10.00, AffinityScore: 0.00)
2025/05/08 08:59:14 Best Node Name determined by scoring: gke-cluster-1-default-pool-c1b7daa1-kh9k
2025/05/08 08:59:14 Found matching node object for gke-cluster-1-default-pool-c1b7daa1-kh9k
2025/05/08 08:59:14 Attempting to bind pod default/testcustom-774b848cdf-gr8mc to node gke-cluster-1-d
efault-pool-c1b7daa1-kh9k
2025/05/08 08:59:14 Successfully bound pod default/testcustom-774b848cdf-gr8mc to node gke-cluster-1-d
efault-pool-c1b7daa1-kh9k
2025/05/08 08:59:14 Unscheduled pod default/testcustom-774b848cdf-gr8mc deleted.
2025/05/08 08:59:14 Unsuccessfully processed scheduling for pod default/testcustom-774b848cdf-gr8mc on n
ode gke-cluster-1-default-pool-c1b7daa1-kh9k

```

Hình 4.18 — Logs của custom scheduler

4.3. Node Selector

Ngoài ra, custom scheduler của nhóm còn có tính năng lập lịch dựa trên node có label cụ thể.

```

# --- Demo: Node Selector ---
# Label a node (replace [CHOSEN_NODE_FOR_LABEL] with an actual node
name from `kubectl get nodes`):
kubectl label node [CHOSEN_NODE_FOR_LABEL] group=red --overwrite

# Deploy a pod that uses a nodeSelector to target the labeled node:
kubectl apply -f scheduler/deployments/pod-node-selector-1.yaml
# Check pod placement and custom scheduler logs:
kubectl get pods -o wide
kubectl logs -n kube-system -l app=custom-scheduler -f --tail=100

```

Khi xem logs, ta sẽ thấy chỉ tìm được 1 node thỏa mãn label `group=red`.

```
2025/05/08 09:07:57 Reconciliation finished, processed 0 pods.
2025/05/08 09:08:29 Node gke-cluster-1-default-pool-clb7daa1-tlmm changed (labels/taints), potentially
  triggering rescheduling...
2025/05/08 09:08:29 Running scheduleAllUnscheduled reconciliation...
2025/05/08 09:08:29 Reconciliation finished, processed 0 pods.
2025/05/08 09:08:36 Found unscheduled pod for custom-scheduler: default/pod-node-selector-1-7948875b8b-
  mrlw7 (Event: ADDED)
2025/05/08 09:08:36 Received pod default/pod-node-selector-1-7948875b8b-mrlw7 for scheduling.
2025/05/08 09:08:36 Attempting to schedule pod: default/pod-node-selector-1-7948875b8b-mrlw7
2025/05/08 09:08:36 Running predicate checks for pod: default/pod-node-selector-1-7948875b8b-mrlw7
2025/05/08 09:08:36 Pod default/pod-node-selector-1-7948875b8b-mrlw7 requires: CPU=0, Memory=0
2025/05/08 09:08:36 Node gke-cluster-1-default-pool-clb7daa1-tlmm PASSED predicate checks.
2025/05/08 09:08:36 Found 1 eligible nodes for pod default/pod-node-selector-1-7948875b8b-mrlw7 (will
  be scored).
2025/05/08 09:08:36 Querying Prometheus: http://prometheus-service.monitoring.svc.cluster.local:8080/a
  pi/v1/query?query=node_memory_MemAvailable_bytes
2025/05/08 09:08:36 Prometheus query for 'node_memory_MemAvailable_bytes' returned 3 results.
2025/05/08 09:08:36 Querying Prometheus: http://prometheus-service.monitoring.svc.cluster.local:8080/a
  pi/v1/query?query=avg+by(%28instance%29+%28rate%28node_cpu_seconds_total%7Bmode%3D%22idle%22%7D%5B1m%5
  D%29%29
2025/05/08 09:08:36 Prometheus query for 'avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[
  1m]))' returned 3 results.
2025/05/08 09:08:36 Max values for normalization - MemAvailable: 3090702336 bytes, CPUIdle(avg cores):
  0.898
2025/05/08 09:08:36 Scores for node gke-cluster-1-default-pool-clb7daa1-tlmm: Mem=10.00 (Val=309070233
  6), CPU=10.00 (Val=0.898), Affinity=0.00 → Total=10.00
2025/05/08 09:08:36 Selected best node: gke-cluster-1-default-pool-clb7daa1-tlmm (Total Score: 10.00,
  MemScore: 10.00, CPUScore: 10.00, AffinityScore: 0.00)
2025/05/08 09:08:36 Best Node Name determined by scoring: gke-cluster-1-default-pool-clb7daa1-tlmm
2025/05/08 09:08:36 Found matching node object for gke-cluster-1-default-pool-clb7daa1-tlmm
2025/05/08 09:08:36 Attempting to bind pod default/pod-node-selector-1-7948875b8b-mrlw7 to node gke-cl
  uster-1-default-pool-clb7daa1-tlmm
2025/05/08 09:08:36 Successfully bound pod default/pod-node-selector-1-7948875b8b-mrlw7 to node gke-cl
  uster-1-default-pool-clb7daa1-tlmm
2025/05/08 09:08:36 Unscheduled pod default/pod-node-selector-1-7948875b8b-mrlw7 deleted.
2025/05/08 09:08:36 Successfully processed scheduling for pod default/pod-node-selector-1-7948875b8b-m
  rlw7 demo_final.mp4
00:10:39 12:02
```

Hình 4.19 — Logs của custom scheduler

4.4. Taint and Tolerants

Custom scheduler của nhóm cũng có tính năng lập lịch dựa taints và tolerations.

```
# --- Demo: Taints and Tolerations ---
# Taint two different nodes (replace placeholders with actual node
  names from `kubectl get nodes`):
kubectl taint nodes [CHOSEN_NODE_1_FOR_TAINT] key1=value1:NoSchedule --
  overwrite
kubectl taint nodes [CHOSEN_NODE_2_FOR_TAINT] key2=value2:NoSchedule --
  overwrite

# Deploy 'pod-toleration-1.yaml'. This pod only tolerates
  'key1=value1:NoSchedule'.
```



```
# It should be scheduled on [CHOSEN_NODE_1_FOR_TAINT] or the remaining
node.
# [CHOSEN_NODE_2_FOR_TAINT] will not pass the predicate check because
the pod doesn't tolerate its taint.
kubectyl apply -f scheduler/deployments/pod-toleration-1.yaml
```

Khi xem logs, ta sẽ thấy chỉ tìm được 2 node thỏa mãn, là node có taint `key1=value1` và node không có taint nào.

```
2025/05/08 09:10:58 Found unscheduled pod for custom-scheduler: default/pod-toleration-1-5c59d7d86c-455cj (Event: ADDED)
2025/05/08 09:10:58 Received pod default/pod-toleration-1-5c59d7d86c-455cj for scheduling.
2025/05/08 09:10:58 Attempting to schedule pod: default/pod-toleration-1-5c59d7d86c-455cj
2025/05/08 09:10:58 Running predicate checks for pod: default/pod-toleration-1-5c59d7d86c-455cj
2025/05/08 09:10:58 Pod default/pod-toleration-1-5c59d7d86c-455cj requires: CPU=0, Memory=0
2025/05/08 09:10:58 Node gke-cluster-1-default-pool-c1b7daa1-8rph PASSED predicate checks.
2025/05/08 09:10:58 Node gke-cluster-1-default-pool-c1b7daa1-kh9k PASSED predicate checks.
2025/05/08 09:10:58 Node gke-cluster-1-default-pool-c1b7daa1-tlmm FAILED predicate checks: Pod does not tolerate taint key2=value2:NoSchedule
2025/05/08 09:10:58 Found 2 eligible nodes for pod default/pod-toleration-1-5c59d7d86c-455cj (will be scored).
2025/05/08 09:10:58 Querying Prometheus: http://prometheus-service.monitoring.svc.cluster.local:8080/api/v1/query?query=node_memory_MemAvailable_bytes
2025/05/08 09:10:58 Prometheus query for 'node_memory_MemAvailable_bytes' returned 3 results.
2025/05/08 09:10:58 Querying Prometheus: http://prometheus-service.monitoring.svc.cluster.local:8080/api/v1/query?query=avg+by+%28instance%29+%28rate%28node_cpu_seconds_total%7Bmode%3D%22idle%22%7D%5B1m%5D%29%29
2025/05/08 09:10:58 Prometheus query for 'avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[1m]))' returned 3 results.
2025/05/08 09:10:58 Max values for normalization - MemAvailable: 3244228608 bytes, CPUIdle(avg cores): 0.923
2025/05/08 09:10:58 Scores for node gke-cluster-1-default-pool-c1b7daa1-8rph: Mem=2.61 (Val=846852096), CPU=0.00 (Val=0.000), Affinity=0.00 → Total=1.57
2025/05/08 09:10:58 Scores for node gke-cluster-1-default-pool-c1b7daa1-kh9k: Mem=10.00 (Val=3244228608), CPU=10.00 (Val=0.923), Affinity=0.00 → Total=10.00
2025/05/08 09:10:58 Selected best node: gke-cluster-1-default-pool-c1b7daa1-kh9k (Total Score: 10.00, MemScore: 10.00, CPUScore: 10.00, AffinityScore: 0.00)
2025/05/08 09:10:58 Best Node Name determined by scoring: gke-cluster-1-default-pool-c1b7daa1-kh9k
2025/05/08 09:10:58 Found matching node object for gke-cluster-1-default-pool-c1b7daa1-kh9k
2025/05/08 09:10:58 Attempting to bind pod default/pod-toleration-1-5c59d7d86c-455cj to node gke-cluster-1-default-pool-c1b7daa1-kh9k
2025/05/08 09:10:58 Unscheduled pod default/pod-toleration-1-5c59d7d86c-455cj deleted.
2025/05/08 09:10:58 Successfully bound pod default/pod-toleration-1-5c59d7d86c-455cj to node gke-cluster-1-default-pool-c1b7daa1-kh9k
2025/05/08 09:10:58 Successfully processed scheduling for pod default/pod-toleration-1-5c59d7d86c-455cj on node gke-cluster-1-default-pool-c1b7daa1-kh9k
```

Hình 4.20 — Logs của custom scheduler

4.5. Node Affinity

Ngoài ra, điểm số của mỗi node cũng được ưu tiên dựa trên node affinity. Ở đây ta sẽ đánh dấu 2 node, một node `zone=a` (được ưu tiên `weight=80` và node `zone=b` được ưu tiên `weight=20`).

```
# --- Demo: Node Affinity ---
```

 Shell

```
# Label two different nodes (replace placeholders with actual node
names from `kubectl get nodes`):
kubectl label node [CHOSEN_NODE_1_FOR_AFFINITY_LABEL] zone=a
kubectl label node [CHOSEN_NODE_2_FOR_AFFINITY_LABEL] zone=b

# Deploy 'pod-preferred-affinity.yaml'. This pod has a
preferredNodeSchedulingIgnoredDuringExecution
# affinity for nodes with label 'zone=a' (weight: 80) or
'zone=b' (weight: 20).
kubectl apply -f scheduler/deployments/pod-preferred-affinity.yaml
```

Ta sẽ thấy node có `zone=a` được tăng thêm điểm Affinity là 8 và node có `zone=b` được tăng thêm điểm Affinity là 2.

```
2025/05/08 09:13:29 Scores for node gke-cluster-1-default-pool-clb7daa1-8rph: Mem=2.72 (Val=878833664)
, CPU=0.00 (Val=0.000), Affinity=8.00 → Total=9.63
2025/05/08 09:13:29 Scores for node gke-cluster-1-default-pool-clb7daa1-kh9k: Mem=10.00 (Val=323036364
8), CPU=10.00 (Val=0.913), Affinity=0.00 → Total=10.00
2025/05/08 09:13:29 Node gke-cluster-1-default-pool-clb7daa1-tlmm affinity score (raw sum: 20, scaled
0-10: 2.00)
2025/05/08 09:13:29 Scores for node gke-cluster-1-default-pool-clb7daa1-tlmm: Mem=9.47 (Val=3057807360
), CPU=10.00 (Val=0.912), Affinity=2.00 → Total=11.68
```

Hình 4.21 — Logs của custom scheduler

Tài liệu tham khảo

- [GopherCon 2016: Kelsey Hightower - Building a custom Kubernetes scheduler](#)
- [Hightower Toy Scheduler Source Code](#)
- [Kubernetes Documentation: Scheduler](#)
- [Prometheus Documentation](#)
- [Pod Lifecycle, Kubernetes docs](#)
- [What is a Container? | Docker](#)
- [Overview, Kubernetes docs](#)
- [Kubernetes, CNCF](#)
- [Kubernetes: Scheduling the Future at Cloud Scale \(Free eBook\)](#)
- [G. Google Cloud Monitoring. Cloud Monitoring](#)
- [D. Spatharakis, et al. Distributed Resource Autoscaling in Kubernetes Edge Clusters](#)
- [J. Campbell. Kubernetes vs. docker](#)
- [Google kubernetes engine documentation](#)
- [Node Exporter](#)
- [A Deep Dive into Kubernetes Metrics](#)
- [The Kubernetes Authors. Managing Compute Resources for Containers](#)